

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



NGHIÊN CỨU PHÁT TRIỂN MÔ HÌNH CHATBOT TƯ VẤN
HỌC VỤ VÀ TUYỂN SINH CỦA TRƯỜNG ĐẠI HỌC

Bùi Tiến Phát

2051052096

Lê Văn Chiến

2051010032

Giảng viên hướng dẫn: ThS. DƯƠNG HỮU THÀNH

KHÓA LUẬN TỐT NGHIỆP
NGÀNH KHOA HỌC MÁY TÍNH

TP. HỒ CHÍ MINH, 2024

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



NGHIÊN CỨU PHÁT TRIỂN MÔ HÌNH CHATBOT TƯ VẤN
HỌC VỤ VÀ TUYỂN SINH CỦA TRƯỜNG ĐẠI HỌC

Bùi Tiến Phát

2051052096

Lê Văn Chiến

2051010032

Giảng viên hướng dẫn: ThS. DƯƠNG HỮU THÀNH

KHÓA LUẬN TỐT NGHIỆP
NGÀNH KHOA HỌC MÁY TÍNH

TP. HỒ CHÍ MINH, 2024

LỜI CẢM ƠN

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

ABSTRACT

MỤC LỤC

LỜI CẢM ƠN	1
NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN.....	2
ABSTRACT	1
DANH MỤC BẢNG BIỂU	6
DANH MỤC TỪ VIẾT TẮT	8
DANH MỤC HÌNH VẼ	11
MỞ ĐẦU	13
Chương 1. GIỚI THIỆU ĐỀ TÀI.....	15
1.1. Tổng quan tình hình nghiên cứu thuộc lĩnh vực đề tài	15
1.2. Lý do lựa chọn đề tài	17
1.3. Mục tiêu đề tài	17
1.4. Cách tiếp cận, phương pháp nghiên cứu.....	19
1.4.1. Open-book QA	19
1.4.2. Close-book QA	19
1.5. Đối tượng và phạm vi nghiên cứu.....	20
1.5.1. Đối tượng nghiên cứu	20
1.5.2. Phạm vi nghiên cứu	21
Chương 2. CƠ SỞ LÝ THUYẾT	23
2.1. Xử lý ngôn ngữ tự nhiên	23
2.2. Bài toán Open Domain Question Answering	24
2.2.1. Giới thiệu về Open Domain Question Answering.....	24
2.2.2. Một số khái niệm cần nắm	27
2.2.3. Word-embedding.....	30

2.2.4.	Retriever Model	33
2.2.5.	Reader model	34
2.3.	Các kỹ thuật huấn luyện mô hình	36
2.3.1.	Fine-tuning	36
2.3.2.	Prompt engineering	39
2.3.3.	Few-shot learning	40
2.4.	Pipeline	41
2.5.	Các phương pháp đánh giá.....	41
2.5.1.	Cosine similarity	41
2.5.2.	Exact Match	42
2.5.3.	ROUGE Metric	42
2.5.4.	Sentences similarity	43
2.6.	Framework Next.js.....	44
Chương 3.	KIẾN TRÚC HỆ THỐNG CHATBOT.....	46
3.1.	Phân tích kiến trúc một chatbot	46
3.1.1.	Kiến trúc tổng thể	47
3.1.2.	Chuẩn bị dữ liệu	49
3.1.3.	Tiền xử lý dữ liệu	50
3.1.4.	Phân chia dữ liệu	59
3.1.5.	Lưu trữ dữ liệu.....	60
3.1.6.	Truy vết dữ liệu	64
3.1.7.	Tổng hợp dữ liệu phản hồi.....	79
3.2.	Tổng kết	84
Chương 4.	CÁC TIẾP CẬN HỆ THỐNG VÀ KẾT QUẢ THỰC NGHIỆM..	92

4.1.	Tiền xử lý dữ liệu.....	93
4.1.1.	Thực nghiệm “Clause is all you need”	93
4.1.2.	Remove acronyms	97
4.2.	Lưu trữ dữ liệu bằng phương pháp Information Augmentation	98
4.3.	Truy vết dữ liệu.....	105
4.3.1.	Rewrite question according the Agent và Summary Conversation 105	
4.3.2.	Vector retriever.....	107
4.4.	Thực nghiệm trên PhoBERT	109
4.4.1.	Huấn luyện mô hình	109
4.4.2.	Kết quả đánh giá mô hình	111
4.4.3.	Phân tích kết quả và thử nghiệm PhoBERT.....	112
Chương 5.	PHÂN TÍCH KIẾN TRÚC HỆ THỐNG	114
5.1.	Phân tích yêu cầu	114
5.1.1.	Yêu cầu người dùng.....	114
5.1.2.	Yêu cầu hệ thống	115
5.2.	Kiến trúc hệ thống.....	116
5.2.1.	Mô hình kiến trúc tổng quan	116
5.2.2.	Mô tả các thành phần hệ thống.....	120
5.3.	Thiết kế CSDL	121
5.3.1.	Mô hình	122
5.3.2.	Mô tả cấu trúc bảng	123
5.4.	Thiết kế giao diện và nghiệp vụ.....	126
5.4.1.	Giao diện chính.....	126

Chương 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	133
6.1. Kết luận.....	133
6.2. Hạn chế của nghiên cứu.....	133
6.3. Hướng phát triển trong tương lai	135
6.4. Ứng dụng thực tiễn	135
TÀI LIỆU THAM KHẢO	137
PHỤ LỤC	138
1. FrontEnd (Next.js).....	139

DANH MỤC BẢNG BIỂU

Table 1 : Prompt giúp chuyển các câu đa nghĩa thành một câu mệnh đề	55
Table 2 : Bảng các phương pháp khả thi cho việc xử lý từ viết tắt	58
Table 3 : Bảng ví dụ cho cuộc trò chuyện giữa người và máy khi dùng đại từ quan hệ	67
Table 4 : Bảng ví dụ về đoạn hội thoại dài có dùng đại từ thay thế	70
Table 5 : Prompt giúp tóm tắt hội thoại.....	72
Table 6 : Bảng hội thoại ví dụ để thử nghiệm “Prompt giúp tóm tắt hội thoại”	72
Table 7 : Prompt giúp tạo lại câu hỏi mới từ lịch sử cuộc hội thoại trước đó	73
Table 8 : Ví dụ và kết quả khi sử dụng “Prompt giúp tạo lại câu hỏi mới từ lịch sử cuộc hội thoại trước đó”	73
Table 9 : Prompt tổng hợp câu trả lời từ PhoBERT và đoạn văn đã truy vết.....	82
Table 10 : Bảng tổng kết quy trình chuẩn bị dữ liệu	86
Table 11 : Bảng tổng kết quy trình tiền xử lý dữ liệu	87
Table 12 : Bảng tổng kết quy trình phân chia dữ liệu	87
Table 13 : Bảng tổng kết quy trình lưu trữ dữ liệu.....	88
Table 14 : Bảng tổng kết quá trình truy vết dữ liệu.....	90
Table 15 : Bảng tổng kết quá trình tổng hợp dữ liệu và phản hồi.....	91
Table 16 : Bảng ví dụ về data test cho thực nghiệm “Clause is all you need”	93
Table 17 : Bảng kết quả thực nghiệm cho kỹ thuật “Clause is all you need” (1).....	94
Table 18 : Bảng kết quả thực nghiệm cho kỹ thuật “Clause is all you need” (2).....	96
Table 19 : Bảng tỷ lệ trả lời đúng khi thay thế từ viết tắt so với không thay thế từ viết tắt	98
Table 20 : Bảng kết quả khi sử dụng phương pháp Information Augmentation.....	100
Table 21 : Bảng ví dụ cho dataset test của phương pháp IA	101
Table 22 : Bảng kết quả truy vết top 1 khi dùng Information Augmentation	102
Table 23 : Bảng kết quả truy vết top 2 khi dùng Information Augmentation	103
Table 24 : Tỷ lệ truy vết dữ liệu đúng mệnh đề với top $k = 3$	104

Table 25 : Ví dụ cho bộ dataset test kỹ thuật “Rewrite question according the Agent và Summary Conversation”.....	105
Table 26 : Bảng kết quả sử dụng kỹ thuật "Rewrite question according the Agent và Summary Conversation"	106
Table 27 : Bảng kết quả khi sử dụng phương pháp MultiQueryRetriever	107
Table 28 : Bảng kết quả khi sử dụng phương pháp Parent Document Retriever	109
Table 29 : Cấu trúc tập dữ liệu huấn luyện	110
Table 30 : Giá trị dữ liệu huấn luyện.....	111
Table 31 : Bảng kết quả Exact match.....	111
Table 32 : Bảng kết quả đánh giá bằng ROUGE metric	112

DANH MỤC TỪ VIẾT TẮT

STT	Ký hiệu	Diễn giải
1	AI	Artificial Intelligence - Trí tuệ nhân tạo
2	ALBERT	A Lite BERT - BERT nhẹ
3	API	Application Programming Interface-là một bộ các giao diện lập trình ứng dụng cho phép các ứng dụng khác truy cập vào các dịch vụ và chức năng của một ứng dụng hoặc hệ thống.
4	BERT	Bidirectional Encoder Representations from Transformers - Biểu diễn bộ mã hóa hai chiều từ Transformers
5	BM25	BM25 (Best Matching 25) là một thuật toán đánh giá độ tương đồng giữa văn bản và truy xuất thông tin.
6	ELECTRA	Efficiently Learning an Encoder that Classifies Token Replacements Accurately - Học hiệu quả một bộ mã hóa phân loại thay thế mã thông báo một cách chính xác
7	FP	False Positive (FP), hay Dự đoán sai, là một thuật ngữ được sử dụng phổ biến trong Đánh giá mô hình học máy, đặc biệt là trong các bài toán phân loại. Nó thể hiện số lượng dự đoán sai trong một tập dữ liệu cụ thể.
8	GLUE	General Language Understanding Evaluation - Đánh giá hiểu biết ngôn ngữ chung,
9	GPT	Generative Pre-trained Transformer - Máy biến áp được đào tạo trước,
10	GPU	GPU là viết tắt của "Graphics Processing Unit", là một loại bộ xử lý dành riêng cho việc xử lý đồ họa và tính

		toán song song trong máy tính.
11	IA	Information Augmentation là khái niệm mới được nhóm tác giả đề xuất, nói đến việc thêm thông tin dữ liệu vào một vector.
12	ID	ID là một thuật ngữ mang nhiều ý nghĩa khác nhau, phụ thuộc vào ngữ cảnh sử dụng. Nó có thể là mã số định danh, tên gọi tắt hoặc tên viết tắt của một đối tượng, tổ chức, sản phẩm, dịch vụ hay khái niệm cụ thể.
13	L2	L2 (chuẩn Euclidean) trong công thức cosine_similarity đóng vai trò quan trọng trong việc chuẩn hóa các vector, giúp tính toán độ tương đồng cosine một cách chính xác và hiệu quả.
14	LCS	Longest Common Subsequence (LCS) là chuỗi con dài nhất xuất hiện trong cả hai chuỗi ban đầu theo thứ tự từng ký tự mà không yêu cầu các ký tự liên tiếp.
15	LLM	Large Language Model (LLM) là một loại mô hình ngôn ngữ mạnh mẽ được huấn luyện trên một lượng lớn dữ liệu văn bản, thường là hàng tỷ hoặc thậm chí hàng trăm tỷ từ.
16	NLG	Natural language generation - Tạo ngôn ngữ tự nhiên
17	NLP	Natural Language Processing - Xử lý ngôn ngữ tự nhiên
18	ODQA	ODQA là viết tắt của "Open-Domain Question Answering" (Hỏi-đáp trong miền mở). Đây là một lĩnh vực trong xử lý ngôn ngữ tự nhiên (NLP) tập trung vào việc trả lời các câu hỏi mà không giới hạn đến một miền hay chủ đề cụ thể.

19	QA	Question answering - Trả lời câu hỏi
20	RC	RC là viết tắt của "Reading Comprehension" (Hiểu đọc). Đây là một tác vụ trong xử lý ngôn ngữ tự nhiên (NLP) tập trung vào việc đọc và hiểu nội dung của một đoạn văn bản và trả lời câu hỏi dựa trên thông tin đã đọc.
21	RoBERTa	Robustly Optimized BERT Pretraining Approach - Phương pháp tiếp cận đào tạo trước BERT được tối ưu hóa mạnh mẽ
22	ROUGE	ROUGE là một tập hợp các phép đo được sử dụng để đánh giá chất lượng của các hệ thống tạo ra tóm tắt văn bản tự động.
23	SGK	Sách giáo khoa
24	SNN	Siamese Neural Networks là một công cụ mạnh mẽ trong NLP và CV, cho phép so sánh các đầu vào cùng loại một cách hiệu quả.
25	SQuAD	Stanford Question Answering Dataset - là một tập dữ liệu thường được sử dụng trong Xử lý ngôn ngữ tự nhiên (NLP) để đào tạo và đánh giá mô hình trả lời câu hỏi tự động
26	T4	T4-Tesla là một dạng GPU (Graphics Processing Unit) do NVIDIA sản xuất, được thiết kế đặc biệt để hỗ trợ hiệu suất cao cho các ứng dụng tính toán song song như xử lý dữ liệu AI và máy học.
27	T5	T5 là một mô hình ngôn ngữ tự nhiên và hệ thống xử lý ngôn ngữ tự nhiên (NLP) được phát triển bởi Google

		AI. T5 viết tắt của "Text-To-Text Transfer Transformer", mô hình này là một kiến trúc mạng nơ-ron sử dụng Transformer, giống như các mô hình ngôn ngữ khác như GPT của OpenAI và BERT của Google.
28	TP	True Positive (TP) là một thuật ngữ được sử dụng phổ biến trong Đánh giá mô hình học máy, đặc biệt là trong các bài toán phân loại. Nó thể hiện số lượng dự đoán chính xác trong một tập dữ liệu cụ thể.

DANH MỤC HÌNH VẼ

Hình 1 : Ba mô hình chính của Domain QA(https://lilianweng.github.io)	27
Hình 2 : Vector word-embedding	30
Hình 3 : Vector word-embedding	30
Hình 4 : Biểu diễn từ trong không gian	31
Hình 5 : Ví dụ về tách token.....	32
Hình 6 : Cấu tạo của các token được tách	32
Hình 7 : Minh họa fine-tuning BERT(https://www.researchgate.net/)	37
Hình 8 : Ví dụ về cấu tạo câu khi tiến hành embedding (https://www.researchgate.net/).....	37
Hình 9 : Cấu trúc lớp input (https://www.researchgate.net/)	38
Hình 10 : Cấu trúc lớp output (https://www.researchgate.net/)	39
Hình 11 : Kiến trúc Next.js	45
Hình 12 : Sơ đồ quy trình RAG cơ bản (https://tinyurl.com/ys7srps5)	48
Hình 13 : Quy trình biến đổi và truy vết câu hỏi người dùng	68
Hình 14 : Quy trình sử dụng phương pháp Rewrite question according the Agent và Summary Conversation	74
Hình 15 : Quy trình tạo ra và tìm đoạn văn bản liên quan (MultiQueryRetriever).....	76
Hình 16 : Quy trình của Parent Document Retriever	78
Hình 17 : Quy trình tổng hợp dữ liệu đưa ra cho người dùng.....	83

Hình 18 : Ví dụ về câu trả lời được trả về từ PhoBERT	84
Hình 19 : Tổng hợp quy trình từ chuẩn bị dữ liệu đến khi đưa ra kết quả cho người dùng	85
Hình 20 : Biểu đồ so sánh tỉ lệ ngữ nghĩa khi dùng kỹ thuật “Clause is all you need”.94	
Hình 21 : Biểu đồ so sánh số lượng câu hỏi không thể trả lời khi dùng kỹ thuật “Clause is all you need”	95
Hình 22 : Biểu đồ so sánh tứ phân vị phần trăm ngữ nghĩa khi dùng kỹ thuật “Clause is all you need”	97
Hình 23 : Biểu đồ thể hiện độ chính xác khi thay thế và không thay thế từ viết tắt	98
Hình 24 : Quy trình cho phương pháp Information Augmentation.....	99
Hình 25 : Tỷ lệ giữa phương pháp truy xuất truyền thống và IA cho Mệnh đề / IA cho Mệnh đề và Câu hỏi.....	100
Hình 26 : Tỷ lệ truy vết dữ liệu đúng mệnh đề với top k = 1	102
Hình 27 : Tỷ lệ truy vết dữ liệu đúng mệnh đề với top k = 2.....	103
Hình 28 : Tỷ lệ truy vết dữ liệu đúng mệnh đề với top k = 3.....	104
Hình 29 : Kết quả sử dụng kỹ thuật "Rewrite question according the Agent và Summary Conversation"	106
Hình 30 : Biểu đồ thể hiện kết quả khi dùng và không dùng phương pháp MultiQueryRetriever	108
Hình 31 : Biểu đồ thể hiện kết quả khi dùng và không dùng phương pháp Parent Document Retriever	109
Hình 32 : Kiến trúc tổng quan hệ thống	117
Hình 33 : Cấu tạo chức năng hệ thống	117
Hình 34 : Kiến trúc hệ thống chi tiết.....	119
Hình 35 : Trang chủ Chatbot.....	126

MỞ ĐẦU

Trong thời gian gần đây, công nghệ xử lý ngôn ngữ tự nhiên (NLP) đã có những bước tiến vượt bậc, đặc biệt là trong lĩnh vực Hỏi-Đáp (Question Answering - QA). Đây là một lĩnh vực nghiên cứu quan trọng với mục tiêu phát triển các hệ thống tự động trả lời câu hỏi của người dùng một cách chính xác và hiệu quả. Các phương pháp truyền thống trong QA chủ yếu dựa trên các luật ngữ pháp và ngữ nghĩa, tuy nhiên, những phương pháp này thường gặp phải hạn chế về ngôn ngữ và không đủ mạnh để giải quyết các câu hỏi phức tạp. Sự ra đời của các phương pháp học sâu (Deep Learning) đã mang lại một bước đột phá lớn trong lĩnh vực này. Các mô hình học sâu sử dụng kiến trúc mạng nơ-ron sâu để học cách trả lời câu hỏi bằng cách phân tích và rút trích thông tin từ văn bản đầu vào, đem lại kết quả vượt trội so với các phương pháp truyền thống.

Một yếu tố quan trọng trong việc phát triển các hệ thống QA hiệu quả chính là dữ liệu huấn luyện. Việc sử dụng các tập dữ liệu lớn và đa dạng đã được chứng minh là có thể cải thiện đáng kể hiệu suất của các hệ thống QA. Tuy nhiên, mặc dù đã có những cải thiện đáng kể về độ chính xác, vẫn còn nhiều thách thức phức tạp cần phải đối mặt, chẳng hạn như xử lý đa nghĩa, ngữ cảnh và khả năng đưa ra các câu trả lời chính xác cho các câu hỏi phức tạp.

Một ứng dụng sâu hơn của các mô hình QA là phát triển các hệ thống chatbot, nhằm mục đích cung cấp các câu trả lời chính xác trong nhiều lĩnh vực cụ thể hoặc đóng vai trò là người trò chuyện, đưa ra lời khuyên. Các nghiên cứu về chatbot hiện nay chủ yếu tập trung vào việc sử dụng các mô hình ngôn ngữ lớn (LLM) như GPT để tạo ra các chatbot có khả năng giao tiếp trôi chảy và sáng tạo hơn. Tuy nhiên, lĩnh vực chatbot tại Việt Nam vẫn đang trong giai đoạn phát triển ban đầu và đối mặt với nhiều khó khăn như thiếu dữ liệu, nguồn nhân lực và hạ tầng công nghệ.

Trong bối cảnh đó, nghiên cứu của chúng em tập trung vào việc phát triển một mô hình dựa trên PhoBERT - một mô hình ngôn ngữ tiếng Việt dựa trên kiến

trúc BERT - để giải quyết bài toán trả lời câu hỏi học vụ của sinh viên. Mục tiêu của dự án là cải thiện hiệu suất của các mô hình hiện có và phát triển một hệ thống hoàn chỉnh hỗ trợ sinh viên trong việc tiếp cận thông tin về học tập, lịch thi và học vụ. Chúng em cũng sẽ tối ưu hóa khả năng trích xuất thông tin từ văn bản, sử dụng các mô hình ngôn ngữ lớn như GPT để diễn đạt thông tin đã trích xuất, đảm bảo rằng chatbot chỉ trả lời từ những thông tin được cung cấp.

Phần còn lại của báo cáo sẽ trình bày chi tiết về tổng quan tình hình nghiên cứu, lý do chọn đề tài, mục tiêu và phương pháp nghiên cứu, đối tượng nghiên cứu, kết quả nghiên cứu, và cuối cùng là kết luận và kiến nghị. Chúng em hy vọng nghiên cứu này sẽ đóng góp vào sự phát triển của lĩnh vực NLP và ứng dụng AI trong giáo dục, mang lại những lợi ích thiết thực cho sinh viên và cộng đồng học thuật.

Chương 1. GIỚI THIỆU ĐỀ TÀI

Chúng em đã nghiên cứu và phát triển một hệ thống hỏi đáp tự động dựa trên các mô hình xử lý ngôn ngữ tự nhiên (NLP) tiên tiến. Trước hết, chúng em đã tổng quan tình hình nghiên cứu trong lĩnh vực NLP và ứng dụng QA, nêu bật sự phát triển từ các phương pháp truyền thống dựa trên luật ngữ pháp đến các mô hình học sâu hiện đại. Chúng em đã lựa chọn mô hình PhoBERT để giải quyết bài toán trả lời câu hỏi học vụ do tính hiệu quả và khả năng xử lý ngôn ngữ tiếng Việt tốt. Mục tiêu của chúng em là phát triển một mô hình QA hiệu quả, có thể cung cấp câu trả lời chính xác cho các câu hỏi liên quan đến học vụ, giúp sinh viên dễ dàng truy cập thông tin mà không cần tra cứu thủ công. Để đạt được điều này, chúng em đã tinh chỉnh PhoBERT và sử dụng các phương pháp như TF-IDF, Sentences-Transformer để cải thiện khả năng trích xuất thông tin và so sánh văn bản. Hệ thống chatbot được phát triển sẽ hỗ trợ sinh viên trong việc tìm kiếm thông tin học tập, định hướng nghề nghiệp, và xây dựng lộ trình học tập. Chúng em cũng đề xuất các phương pháp mới để nâng cao hiệu suất truy vết văn bản và đánh giá mô hình một cách chính xác. Phạm vi nghiên cứu bao gồm việc áp dụng các mô hình học sâu như PhoBERT, GPT và các kỹ thuật truy vết văn bản để xây dựng một hệ thống hỏi đáp tự động hiệu quả, đáp ứng nhu cầu thông tin của sinh viên.

1.1. Tổng quan tình hình nghiên cứu thuộc lĩnh vực đề tài

Trong vực xử lý ngôn ngữ tự nhiên (NLP), Question Answering (QA) là một chủ đề nghiên cứu quan trọng, trong đó mục tiêu là phát triển các hệ thống tự động trả lời câu hỏi của người dùng. Dưới đây là một số thông tin tổng quan về tình hình nghiên cứu trong lĩnh vực NLP Question Answering.

Các phương pháp truyền thống: Trong quá khứ, các phương pháp truyền thống cho QA dựa trên các luật ngữ pháp và semantic, trong đó ta định nghĩa các quy tắc để tìm kiếm câu trả lời trong văn bản. Tuy nhiên, các phương pháp này thường rất phụ thuộc vào ngôn ngữ và không đủ mạnh để giải quyết các câu hỏi phức tạp.

Học sâu (Deep Learning): Trong những năm gần đây, các phương pháp học sâu đã được áp dụng rộng rãi cho các nhiệm vụ QA. Các mô hình này sử dụng các kiến trúc mạng nơ-ron sâu để học cách trả lời các câu hỏi bằng cách phân tích và rút trích thông tin từ văn bản đầu vào. Các mô hình học sâu cho QA đã cho kết quả tốt hơn so với các phương pháp truyền thống.

Dữ liệu huấn luyện: Một yếu tố quan trọng trong việc phát triển các hệ thống QA hiệu quả là dữ liệu huấn luyện. Các nghiên cứu đã chỉ ra rằng việc sử dụng các tập dữ liệu lớn và đa dạng có thể cải thiện hiệu suất của các hệ thống QA.

Độ chính xác: Tuy nhiên, mặc dù các mô hình học sâu cho QA đã cải thiện đáng kể độ chính xác của các hệ thống, nhưng vẫn có những thách thức phức tạp cần được giải quyết, như đa nghĩa, ngữ cảnh và khả năng đưa ra các câu trả lời chính xác cho các câu hỏi phức tạp.

Nhìn chung, tình hình nghiên cứu trong lĩnh vực NLP Question Answering đang phát triển nhanh chóng, với việc sử dụng các phương pháp học sâu và tập dữ liệu lớn để cải thiện hiệu suất của các hệ thống. Tuy nhiên, vẫn còn nhiều thách thức cần được giải quyết để đưa ra các hệ thống QA chính xác và hiệu quả hơn.

Ứng dụng sâu hơn của một mô hình QA là phát triển một hệ thống chatbot, nhằm mục đích đưa ra những kết quả trả lời chính xác nhất về một hay nhiều lĩnh vực cụ thể nào đó, hoặc cũng có thể đóng vai trò là một người trò chuyện hay đưa ra lời khuyên v.v... Một số nghiên cứu chính về chatbot có thể kể đến như RL - Cho phép chatbot học hỏi từ kinh nghiệm tương tác với người dùng, tự động điều chỉnh hành vi để tối ưu hóa hiệu quả giao tiếp, LLM - Sử dụng các mô hình ngôn ngữ lớn như GPT để tạo ra các chatbot có khả năng giao tiếp trôi chảy và sáng tạo hơn, hoặc những chatbot giúp hiểu được cảm xúc của người dùng để giao tiếp đồng cảm và hiệu quả hơn. Lĩnh vực chatbot tại Việt Nam đang được đẩy mạnh phát triển và có nhiều tiềm năng, nhưng số lượng vẫn còn hạn chế, thiếu dữ liệu, nguồn nhân lực và hạ tầng công nghệ.

1.2. Lý do lựa chọn đề tài

Lĩnh vực xử lý ngôn ngữ tự nhiên (NLP) đã phát triển nhanh chóng trong những năm gần đây do nhu cầu máy móc hiểu và xử lý ngôn ngữ của con người ngày càng tăng. Với sự bùng nổ của dữ liệu kỹ thuật số và việc sử dụng rộng rãi các phương tiện truyền thông xã hội và các nền tảng trực tuyến khác, có một lượng lớn dữ liệu văn bản phi cấu trúc được tạo ra mỗi ngày. Nghiên cứu NLP nhằm mục đích phát triển các thuật toán và mô hình có thể giúp máy móc hiểu được dữ liệu này và trích xuất những hiểu biết và thông tin có ý nghĩa từ nó. Để thỏa mãn mục đích tìm hiểu của bản thân cũng như mong muốn đóng góp một ứng dụng hữu ích cho mọi người nên chúng em đã quyết định lựa chọn nghiên cứu về NLP. Mặt khác, là một sinh viên được nhận sự dạy dỗ từ nhà trường, với sứ mệnh mong muốn được cống hiến, trả ơn công lao của nhà trường, thầy, cô đã dạy dỗ, cũng như nhận thấy nhu cầu phát triển chatbot về học vụ, tuyển sinh là cần thiết. Nên đề tài đã được chúng em quyết tâm thực hiện một cách chính chu nhất.

1.3. Mục tiêu đề tài

Tại bước đầu tiên trong dự án nghiên cứu của chúng em, chúng em đã phát triển một mô hình dựa trên PhoBERT là một mô hình ngôn ngữ tiếng Việt dựa trên kiến trúc của BERT (Bidirectional Encoder Representations from Transformers) để giải quyết bài toán trả lời các câu hỏi của sinh viên về vấn đề học vụ. Mục tiêu của dự án là tìm cách cải thiện hiệu suất của các mô hình đã được phát triển trước đó và đạt được kết quả tốt hơn. Mục tiêu cuối cùng của bước đầu là đào tạo ra một mô hình có thể trả lời bất kỳ câu hỏi nào liên quan đến kiến thức thực tế có thể dẫn đến nhiều ứng dụng hữu ích và thực tế nói chung cũng như kiến thức về vấn đề học vụ trường học nói riêng, chẳng hạn như làm chatbot hoặc trợ lý AI. Với mô hình được phát triển, chúng em hy vọng sẽ giúp sinh viên có thể dễ dàng tiếp cận với các thông tin về học tập, lịch thi, học vụ... mà không cần phải tra cứu thủ công từ các văn bản, quyết định hoặc thông báo (thường có độ dài lớn).

Tại bước tiếp cận tiếp theo, chúng em đã tối ưu hóa khả năng trích xuất

thông tin từ một văn bản, thông tin này sẽ đóng vai trò là quyền sách, mượn sức mạnh của các mô hình ngôn ngữ lớn mạnh như GPT, Gemini... để làm bộ não nhằm diễn đạt thông tin đã trích xuất từ tài liệu chỉ định. Việc này sẽ giới hạn lại miền kiến thức của các LLMs, đảm bảo chatbot chỉ có thể trả lời từ những thông tin ta cung cấp.

Từ mô hình đã được đào tạo, chúng em sẽ phát triển một hệ thống chatbot toàn diện nhằm hỗ trợ sinh viên trên nhiều phương diện. Chatbot này sẽ đóng vai trò như một trợ lý ảo đặc lực, sẵn sàng cung cấp thông tin học vụ chi tiết về khóa học, ngành học, chương trình học, cũng như định hướng nghề nghiệp và các nguồn tài liệu hữu ích. Bên cạnh đó, chatbot còn có khả năng cá nhân hóa trải nghiệm học tập của mỗi sinh viên. Bằng cách thu thập thông tin về sở thích, kỹ năng và mục tiêu của người dùng, chatbot sẽ đưa ra những đề xuất về lộ trình học tập phù hợp, giúp sinh viên xác định rõ hướng đi và nâng cao hiệu quả học tập.

Không chỉ dừng lại ở việc cung cấp thông tin, chatbot còn hỗ trợ sinh viên trong việc xây dựng lộ trình học tập cụ thể. Chatbot sẽ giúp người dùng lựa chọn các khóa học phù hợp, sắp xếp thứ tự học một cách khoa học, đặt mục tiêu rõ ràng và theo dõi tiến độ học tập, từ đó đảm bảo sinh viên luôn đi đúng hướng và đạt được kết quả tốt nhất.

Ngoài ra, chatbot còn đóng vai trò như một người bạn đồng hành đáng tin cậy, luôn nhắc nhở sinh viên về những mốc thời gian quan trọng như hạn nộp bài, lịch học, sự kiện học thuật... Bất cứ khi nào có thắc mắc về học thuật, sinh viên có thể đặt câu hỏi cho chatbot và nhận được những giải đáp chi tiết, từ việc giải thích thuật ngữ, khái niệm cho đến cung cấp thông tin chuyên sâu về các chủ đề học tập.

Đặc biệt, chatbot còn có khả năng tạo ra một môi trường học tập tương tác hấp dẫn thông qua các câu hỏi, bài tập thực hành đa dạng. Điều này không chỉ giúp sinh viên củng cố kiến thức đã học mà còn rèn luyện kỹ năng tư duy, giải quyết vấn đề một cách chủ động và hiệu quả.

1.4. Cách tiếp cận, phương pháp nghiên cứu

Về vấn đề đào tạo chatbot trả lời câu hỏi về học vụ, ta có khái niệm Open-domain Question Answering (ODQA). ODQA là một loại nhiệm vụ ngôn ngữ, yêu cầu một mô hình đưa ra câu trả lời cho các câu hỏi thực tế bằng ngôn ngữ tự nhiên.

Ví dụ: Câu hỏi: Sinh viên đại học Mở vào tiết 1 lúc mấy giờ?

Trả lời: 7h30.

Phần “Open Domain” đề cập đến việc thiếu bối cảnh (context) liên quan cho bất kỳ câu hỏi thực tế nào được hỏi một cách tùy tiện. Trong trường hợp trên, mô hình chỉ lấy câu hỏi làm đầu vào (“Sinh viên đại học Mở vào tiết 1 lúc mấy giờ?”) mà không đưa ra bài viết về “Giờ giấc học tập của sinh viên”, trong đó có khả năng dữ liệu về “thời điểm tiết 1 bắt đầu” được nhắc đến. Trong trường hợp cả câu hỏi và ngữ cảnh đều được cung cấp, nhiệm vụ này được gọi là Reading Comprehension (RC). Một mô hình ODQA có thể được phép truy cập (hoặc không) vào một miền dữ liệu ngoài (ví dụ: Wikipedia, kaggle...), mỗi điều kiện được truy cập sẽ lần lượt được định nghĩa là Open-book QA (Mở) và Close-book QA (Đóng).

1.4.1. Open-book QA

Open book QA cho phép truy cập vào một nguồn thông tin rộng lớn để tìm kiếm và trích xuất câu trả lời cho câu hỏi. Trong trường hợp này, chatbot có thể sử dụng các nguồn dữ liệu bên ngoài như sách, bài báo, trang web, v.v. để tìm kiếm thông tin chính xác nhằm cung cấp câu trả lời cho câu hỏi từ người dùng. Chatbot sẽ sử dụng kỹ thuật trích xuất thông tin từ nguồn dữ liệu và sắp xếp lại thông tin đó thành một câu trả lời hoặc một tập hợp các câu trả lời có ý nghĩa.

1.4.2. Close-book QA

Close book QA chỉ sử dụng thông tin có sẵn trong mô hình chatbot và không truy cập vào nguồn thông tin bên ngoài. Trong trường hợp này, chatbot được huấn luyện trên dữ liệu hạn chế và chỉ biết những thông tin được cung cấp trong quá trình huấn luyện. Chatbot sử dụng kiến thức có sẵn trong mô hình để trả lời câu hỏi từ

người dùng. Phương pháp này có thể đạt hiệu suất tốt trong các tác vụ hẹp và đặc biệt hữu ích khi nguồn thông tin bên ngoài không thể truy cập hoặc không khả thi.

1.5. Đối tượng và phạm vi nghiên cứu

1.5.1. Đối tượng nghiên cứu

Các mô hình học sâu và cấu trúc của chúng được sử dụng để trả lời câu hỏi: Nghiên cứu các phương pháp học sâu như mạng nơ-ron, mạng Transformer, mạng LSTM, mạng CNN, và cấu trúc của chúng để tối ưu hóa hiệu suất của các mô hình QA. Nghiên cứu cách áp dụng các mô hình này cho các nhiệm vụ QA khác nhau như trả lời câu hỏi đa câu trả lời, trả lời câu hỏi dựa trên đoạn văn bản, trả lời câu hỏi dựa trên tri thức.

Các phương pháp tiếp cận khác nhau (dựa trên tri thức, trích xuất thông tin, học sâu) và cách tích hợp chúng để đạt được kết quả tốt nhất: Nghiên cứu các phương pháp tiếp cận khác nhau để tìm kiếm câu trả lời cho các câu hỏi. Các phương pháp này bao gồm phương pháp dựa trên tri thức, trích xuất thông tin, học sâu, và cách kết hợp chúng để tìm kiếm câu trả lời. Nghiên cứu cách tích hợp tri thức vào các mô hình học sâu để tăng cường khả năng trả lời câu hỏi.

Các tập dữ liệu và ứng dụng của QA: Nghiên cứu các tập dữ liệu QA, bao gồm các tập dữ liệu chuyên môn và các tập dữ liệu thực tế, để đánh giá hiệu quả của các mô hình QA và cải tiến chúng. Nghiên cứu ứng dụng thực tế của QA, bao gồm chatbot, virtual assistant, hệ thống QA cho lĩnh vực y tế và tài chính.

Trong nỗ lực nâng cao hiệu suất truy vết thông tin của chatbot từ miền tri thức giới hạn, nghiên cứu này tập trung vào việc khám phá các phương pháp so sánh văn bản mới. Mục tiêu chính là khắc phục những hạn chế của các mô hình ngôn ngữ lớn hiện nay, vốn thường "tự sinh" thông tin và thiếu chính xác trong các lĩnh vực chuyên môn.

Các phương pháp tiếp cận đa dạng, từ dựa trên đặc trưng đến dựa trên khoảng cách, đã được xem xét kỹ lưỡng. Phương pháp dựa trên đặc trưng tập trung

vào việc tính toán độ tương đồng giữa các đặc trưng của văn bản, sử dụng cả các mô hình học máy truyền thống như mạng nơ-ron, cây quyết định và Naive Bayes, cũng như các kỹ thuật rút trích đặc trưng như LSA và NMF. Mặt khác, phương pháp dựa trên khoảng cách lại đo lường sự khác biệt giữa các văn bản thông qua các đại lượng như khoảng cách Euclid, Manhattan hoặc cosine, đồng thời tận dụng các kỹ thuật giảm chi phí tính toán và mô hình học máy để phân loại văn bản.

Đặc biệt, nghiên cứu này còn khai thác tiềm năng của Sentences-Transformer, một thư viện Python mạnh mẽ, để tạo ra các vector biểu diễn cho câu hoặc đoạn văn bằng cách sử dụng các mô hình transformer tiên tiến trong lĩnh vực xử lý ngôn ngữ tự nhiên. Từ đó, việc tính toán độ tương đồng văn bản trở nên thuận lợi hơn thông qua các phương pháp như cosine similarity. Thêm vào đó, các mô hình nhúng từ (embedding) của GPT cũng được kết hợp để nâng cao hơn nữa khả năng so sánh và xác định sự tương đồng giữa các văn bản.

1.5.2. Phạm vi nghiên cứu

1.5.2.1. PhoBERT

Bước đầu của nghiên cứu sẽ tập trung vào việc nghiên cứu và phát triển các phương pháp sử dụng mô hình PhoBERT để giải quyết bài toán trả lời câu hỏi. Mô hình PhoBERT: Xem xét các khía cạnh của mô hình PhoBERT, bao gồm cơ chế pre-training, cấu trúc mô hình và các đặc trưng của nó.

Tinh chỉnh PhoBERT cho bài toán trả lời câu hỏi có nhiều lựa chọn: Tìm hiểu cách sử dụng tinh chỉnh mô hình PhoBERT để thích ứng với bài toán trả lời câu hỏi. Nghiên cứu các phương pháp khác nhau để tinh chỉnh mô hình, bao gồm fine-tuning và multi-task learning.

1.5.2.2. Các phương pháp truy vết văn bản

Ứng dụng Term Frequency-Inverse Document Frequency (TF-IDF) và những biến thể của nó, TF-IDF là một phương pháp được sử dụng trong xử lý ngôn ngữ tự nhiên (NLP) để đánh giá sự quan trọng của một từ trong một văn bản. Nó

được sử dụng rộng rãi trong các tác vụ như truy vấn thông tin, phân loại văn bản, và tìm kiếm liên quan. Từ đó chuyển vào input cho PhoBert hoặc GPT để xử lý văn bản trả lời.

Sử dụng các mô hình biểu diễn văn bản thành không gian vector như Sentences-Transformer, mBERT để chuyển các đoạn thông tin thành không gian vector. Sau đó, dùng các phương pháp so sánh tương đồng như độ tương đồng cosine để so sánh thông tin đó với câu hỏi mà người dùng đưa ra, tạo miền tri thức cho chatbot trả lời.

Chúng em cũng tiến hành cải tiến và đề xuất các phương pháp mới, mục tiêu nhằm cải thiện hiệu suất truy vết văn bản cho mô hình cũng như tăng cường ngữ nghĩa cho văn bản. Các phương pháp mới sẽ đã được chính mình là có hiệu quả tốt khi đã thu được những kết quả đáng mong đợi.

Xây dựng tập dữ liệu và đánh giá mô hình: Thiết kế và xây dựng các tập dữ liệu để đánh giá hiệu suất của mô hình trả lời câu hỏi có nhiều lựa chọn. Sử dụng các phương pháp đánh giá chính xác và khách quan để đánh giá hiệu suất của mô hình, bao gồm các phương pháp đánh giá định lượng và định tính.

Chương 2. CƠ SỞ LÝ THUYẾT

Ở chương này, chúng em trình bày cơ sở lý thuyết về Xử lý ngôn ngữ tự nhiên (NLP) và các ứng dụng của nó, cũng như các quá trình chuyển đổi từ văn bản phi cấu trúc sang cấu trúc gồm các bước như Tokenizer, Stemming, Lemmatization, và Part of Speech Tagging. Trong nghiên cứu này, chúng em tập trung vào nhiệm vụ Open Domain Question Answering (ODQA), giúp tìm kiếm và cung cấp câu trả lời chính xác từ nguồn thông tin rộng lớn. ODQA đòi hỏi sự hiểu biết ngữ cảnh và kỹ thuật truy vấn thông tin để trích xuất và đánh giá câu trả lời chính xác. Các thách thức trong ODQA bao gồm hiểu đúng ý đồ người dùng, tìm kiếm thông tin phù hợp từ nguồn dữ liệu không đồng nhất, và đánh giá độ tin cậy của câu trả lời. Các mô hình học sâu như BERT và PhoBERT đã cải thiện khả năng xử lý ngôn ngữ tự nhiên, và chúng em đã trình bày lý thuyết và áp dụng các phương pháp như Fine-tuning để tinh chỉnh mô hình cho nhiệm vụ cụ thể. Cuối cùng, chúng em sử dụng các phương pháp đánh giá như Cosine Similarity, Exact Match, và ROUGE để đo lường hiệu quả của hệ thống ODQA.

2.1. Xử lý ngôn ngữ tự nhiên

Xử lý ngôn ngữ tự nhiên (NLP) là lĩnh vực của trí tuệ nhân tạo (AI) giúp máy tính hiểu và xử lý ngôn ngữ tự nhiên của con người như văn bản và giọng nói. NLP kết hợp ngôn ngữ học tính toán, mô hình hóa ngôn ngữ con người và học máy để giúp máy tính hiểu và biểu đạt lại ý định hoặc tình cảm của con người. Quá trình NLP bắt đầu từ văn bản phi cấu trúc, tức những gì chúng ta nói hoặc viết hàng ngày. Máy tính cần chuyển đổi những văn bản này thành cấu trúc để xử lý. Ví dụ, câu "Hãy mua cho tôi một lon bò húc và một cái bánh mì" cần được cấu trúc lại để máy tính hiểu và phản hồi chính xác. NLP đóng vai trò trung gian giữa văn bản có cấu trúc và văn bản phi cấu trúc. Việc chuyển từ văn bản không cấu trúc sang văn bản có cấu trúc gọi là Natural Language Understanding (NLU), và ngược lại gọi là Natural Language Generation (NLG). Trong hơn 50 năm, NLP đã được ứng dụng rộng rãi trong y tế, kinh doanh, tìm kiếm, tương tác trực tuyến, v.v....

Một ứng dụng phổ biến của NLP là dịch máy (Machine Translation), như Google Dịch, giúp dịch văn bản từ ngôn ngữ này sang ngôn ngữ khác dựa trên ngữ cảnh. Phân tích tình cảm (Sentiment Analysis) giúp đánh giá cảm xúc từ văn bản. NLP cũng hỗ trợ phát hiện thư rác bằng cách nhận diện các từ ngữ đặc trưng của spam. NLP có thể tóm tắt văn bản và khai thác thông tin quan trọng, như xác định sản phẩm được tìm kiếm nhiều nhất để xây dựng chiến lược kinh doanh. Trong Question Answering, NLP giúp truy xuất câu trả lời từ đoạn văn bản lớn, với lĩnh vực Open Domain Question Answering (ODQA) tìm kiếm câu trả lời từ nguồn dữ liệu rộng lớn.

Đầu vào của NLP là văn bản phi cấu trúc, sau đó được chuyển thành văn bản có cấu trúc qua các bước như:

- Tokenizer: Chia văn bản thành các đơn vị nhỏ hơn (tokens).
- Stemming và Lemmatization: Xác định từ gốc của các tokens.
- Part of Speech Tagging: Gán nhãn ngữ pháp cho các tokens.
- Entity Recognition: Nhận dạng các thực thể trong văn bản.

Những bước này giúp NLP chuyển đổi văn bản phi cấu trúc thành cấu trúc mà máy tính có thể hiểu và sử dụng trong các ứng dụng AI hiện đại.

2.2. Bài toán Open Domain Question Answering

Trong phạm vi nghiên cứu đề tài này, chúng em đã chọn đào sâu vào một nhiệm vụ cụ thể của NLP đó là Open Domain Question Answering.

2.2.1. Giới thiệu về Open Domain Question Answering

Open Domain Question Answering (ODQA) là một lĩnh vực trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP) nhằm tìm kiếm và cung cấp câu trả lời chính xác cho các câu hỏi mở trong một nguồn thông tin rộng lớn, không bị ràng buộc bởi ngữ cảnh hoặc tài liệu cụ thể. ODQA có vai trò quan trọng trong việc giúp con người truy cập vào thông tin một cách nhanh chóng và hiệu quả, đồng thời mang lại những

tiền bộ đáng kể trong lĩnh vực trí tuệ nhân tạo.

Mục tiêu chính của ODQA là tìm kiếm thông tin từ các nguồn dữ liệu mở như bách khoa toàn thư, báo chí, trang web, tài liệu khoa học, v.v..., và trích xuất câu trả lời chính xác dựa trên câu hỏi của người dùng. ODQA đòi hỏi sự kết hợp giữa nhiều kỹ thuật NLP như hiểu ngữ cảnh, xử lý ngôn ngữ tự nhiên, truy vấn thông tin và rút trích thông tin để đạt được kết quả tốt.

ODQA cũng đối mặt với nhiều thách thức phức tạp. Một trong những thách thức chính là hiểu và đánh giá chính xác ngữ cảnh và ý nghĩa của câu hỏi. Câu hỏi mở thường đa dạng và không bị giới hạn bởi ngữ cảnh hoặc tài liệu cụ thể. Do đó, việc hiểu đúng ý đồ của người dùng và tìm hiểu câu hỏi là điều quan trọng để đưa ra câu trả lời phù hợp. Một thách thức quan trọng khác trong ODQA là tìm kiếm thông tin phù hợp từ các nguồn dữ liệu rộng lớn và không đồng nhất. Các nguồn thông tin thường có định dạng và cấu trúc khác nhau, đòi hỏi các phương pháp truy vấn thông tin và xếp hạng thông tin để tìm ra những tài liệu quan trọng và liên quan nhất.

Sau khi tìm kiếm được các tài liệu phù hợp, ODQA cần rút trích thông tin từ các nguồn dữ liệu phức tạp và không đồng nhất. Việc rút trích thông tin chính xác từ các văn bản, định dạng khác nhau đòi hỏi sự kết hợp giữa các phương pháp trích xuất thông tin và xử lý ngôn ngữ tự nhiên.

Một thách thức quan trọng nữa trong ODQA là đánh giá độ tin cậy và tính chính xác của câu trả lời. Câu trả lời phải được đánh giá dựa trên sự phù hợp với câu hỏi, tính chính xác của thông tin được trích xuất và độ tin cậy của nguồn thông tin.

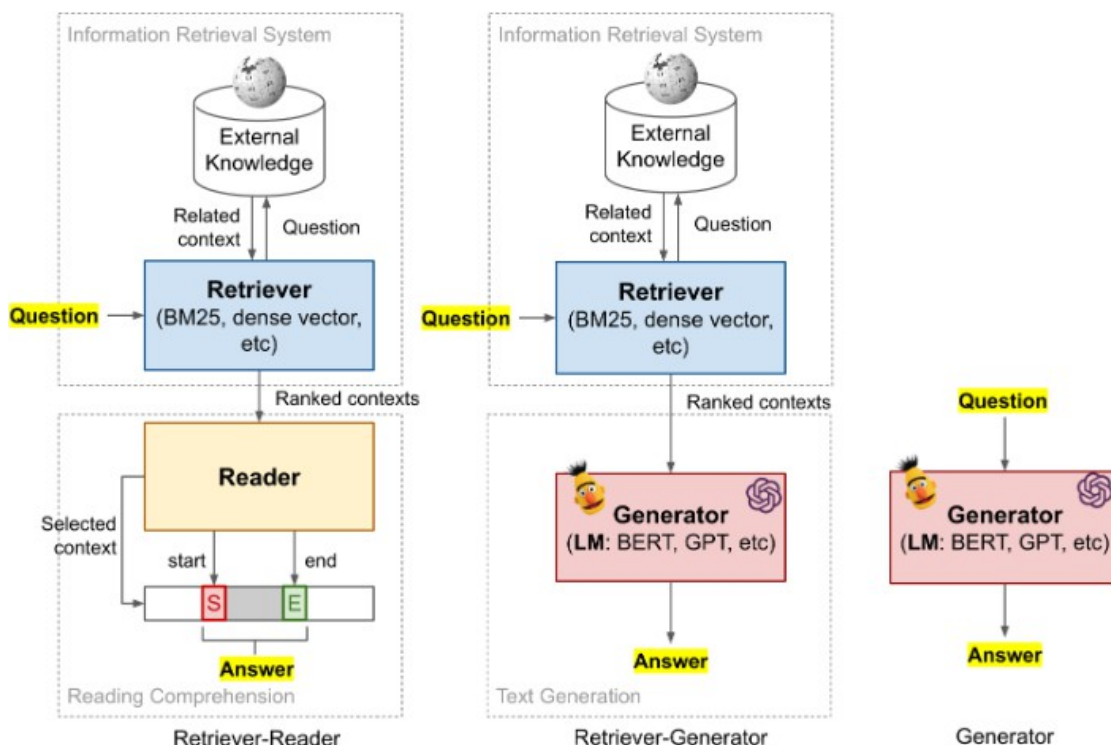
Trong thời gian gần đây, các mô hình học sâu và kiến trúc transformer như BERT, T5 và ELECTRA đã được áp dụng thành công trong ODQA. Những tiến bộ này đã cải thiện khả năng hiểu và xử lý ngôn ngữ tự nhiên của hệ thống và mang lại kết quả tốt hơn trong việc trích xuất thông tin và cung cấp câu trả lời chính xác cho các câu hỏi mở. Vì vậy, ứng dụng của ODQA rất đa dạng. Trong các hệ thống trợ lý ảo, ODQA giúp cung cấp câu trả lời tức thì cho người dùng dựa trên các nguồn

thông tin mở. Các công cụ tìm kiếm thông tin cũng có thể sử dụng ODQA để cung cấp câu trả lời chính xác cho các câu hỏi mở của người dùng. Các hệ thống hỏi-đáp cũng có thể truy vấn vào ODQA để trích xuất thông tin và trả lời các câu hỏi mở một cách tự động.

ODQA cũng có ứng dụng trong nhiều lĩnh vực khác nhau như giáo dục, y tế, tư vấn, và nghiên cứu khoa học. Trong giáo dục, ODQA có thể giúp học sinh và sinh viên tìm kiếm thông tin và cung cấp câu trả lời chính xác cho các câu hỏi liên quan đến bài học. Trong lĩnh vực y tế, ODQA có thể hỗ trợ các chuyên gia y tế trong việc tra cứu thông tin và tìm kiếm câu trả lời cho các câu hỏi về bệnh tật và điều trị. Trong nghiên cứu khoa học, ODQA có thể hỗ trợ các nhà nghiên cứu trong việc tìm kiếm và trích xuất thông tin từ các tài liệu khoa học và báo cáo.

Có nhiều loại ODQA khác nhau, nhưng ta có thể phân làm ba loại chính như sau:

- Mô hình ODQA có thể ghi nhớ chính xác những gì đã có trong quá trình đào tạo.
- Mô hình ODQA có thể trả lời câu hỏi mới dựa vào thông tin cụ thể được cung cấp và xác định phần trả lời từ câu hỏi (Mục tiêu đang nhắm đến).
- Mô hình ODQA có thể trả lời câu hỏi mà nó chưa từng được học.



Hình 1: Ba mô hình chính của Domain QA(<https://lilianweng.github.io>)

Tuy nhiên, còn nhiều thách thức cần được vượt qua để đạt được ODQA tối ưu. Các hệ thống ODQA cần được cải thiện để hiểu và xử lý ngôn ngữ tự nhiên một cách thông minh hơn, tìm kiếm thông tin hiệu quả từ các nguồn dữ liệu phức tạp và không đồng nhất, và đảm bảo tính chính xác và đáng tin cậy của câu trả lời. Các nghiên cứu và phát triển trong lĩnh vực này vẫn đang tiếp tục để đưa ra những tiến bộ mới trong ODQA và tăng cường khả năng tương tác thông minh giữa con người và máy tính trong việc truy xuất thông tin và trả lời câu hỏi mở.

2.2.2. Một số khái niệm cần nắm

Downstream task (Nhiệm vụ sau) : Downstream task là những tác vụ học có giám sát được cải tiến từ các pretrained model. Trong xử lý ngôn ngữ tự nhiên (NLP), các Downstream task đề cập đến các ứng dụng hoặc tác vụ cụ thể sử dụng các mô hình ngôn ngữ được đào tạo trước làm điểm bắt đầu. Các Downstream task thường cụ thể hơn so với tác vụ pretrain khi đào tạo và yêu cầu tinh chỉnh mô hình

trước khi đào tạo trên một tập dữ liệu nhỏ hơn, dành riêng cho nhiệm vụ. Mục tiêu của việc tinh chỉnh là điều chỉnh mô hình được đào tạo trước với ngôn ngữ và ngữ cảnh cụ thể của nhiệm vụ sau, từ đó cải thiện hiệu suất của tác vụ. Downstream task rất quan trọng trong NLP vì chúng cho phép áp dụng các mô hình ngôn ngữ đã được đào tạo trước cho các vấn đề cụ thể trong thế giới thực, do đó cải thiện độ chính xác và hiệu quả của các ứng dụng NLP.

General Language Understanding Evaluation (GLUE): Đánh giá hiểu biết ngôn ngữ chung (GLUE) là một tiêu chuẩn để đánh giá hiệu suất của các mô hình xử lý ngôn ngữ tự nhiên (NLP) trên nhiều nhiệm vụ khác nhau. GLUE bao gồm một tập hợp gồm chín nhiệm vụ NLP đa dạng, bao gồm phân tích tình cảm, trả lời câu hỏi và trình bày văn bản, được thiết kế để kiểm tra khả năng hiểu và vận dụng ngôn ngữ tự nhiên của các mô hình. Mục tiêu của GLUE là cung cấp một khung đánh giá được tiêu chuẩn hóa để so sánh hiệu suất của các mô hình NLP khác nhau trên một loạt các nhiệm vụ. Các nhiệm vụ GLUE được chọn để đại diện cho một loạt các nhiệm vụ hiểu ngôn ngữ tự nhiên và các mô hình được đánh giá dựa trên hiệu suất trung bình của chúng trên tất cả các nhiệm vụ.

Textual Entailment (Quan hệ văn bản): Là một nhiệm vụ xử lý ngôn ngữ tự nhiên (NLP) liên quan đến việc xác định xem một đoạn văn bản (giả thuyết) có thể được suy ra từ một đoạn văn bản khác (tiền đề) hay không. Nhiệm vụ này được thiết kế để kiểm tra khả năng của người mẫu trong việc hiểu mối quan hệ giữa hai đoạn văn bản và đưa ra các suy luận logic dựa trên sự hiểu biết đó. Natural Language Inference: Nó giúp xác định xem một giả thuyết là đúng (entailment) hoặc sai (Contradiction) hoặc trung lập (neutral). so với một mệnh đề trước đó.

Contextual (Ngữ cảnh): Trong xử lý ngôn ngữ tự nhiên (NLP), ngữ cảnh đề cập đến ý tưởng rằng ý nghĩa của một từ hoặc cụm từ có thể phụ thuộc vào ngữ cảnh mà nó xuất hiện. Điều này có nghĩa là cùng một từ hoặc cụm từ có thể có nghĩa hoặc cách hiểu khác nhau dựa trên các từ, câu hoặc diễn ngôn xung quanh. Hiểu ngữ cảnh rất quan trọng đối với nhiều nhiệm vụ NLP, chẳng hạn như dịch

máy, phân tích tình cảm và phân loại văn bản. Trong những nhiệm vụ này, ý nghĩa của một từ hoặc cụm từ có thể tác động đáng kể đến việc diễn giải tổng thể văn bản. Các mô hình có khả năng nắm bắt và sử dụng hiệu quả thông tin theo ngữ cảnh thường chính xác và mạnh mẽ hơn trong hoạt động của chúng.

Transformers là một lớp kiến trúc mạng thần kinh có tác động đáng kể đến lĩnh vực xử lý ngôn ngữ tự nhiên (NLP). Không giống như các mô hình NLP truyền thống, sử dụng các tính năng thủ công và chủ yếu dựa vào các quy tắc ngôn ngữ, các mô hình máy biến áp sử dụng các cơ chế tự chú ý để nắm bắt các phụ thuộc tầm xa trong văn bản và đã đạt được hiệu suất cao nhất trên nhiều loại NLP nhiệm vụ. Cải tiến quan trọng của các mô hình Transformers là cơ chế tự chú ý, cho phép mô hình chú ý đến các phần khác nhau của chuỗi đầu vào khi tính toán biểu diễn của từng mã thông báo. Điều này cho phép mô hình nắm bắt các quan hệ phụ thuộc trong phạm vi dài và đã được chứng minh là có hiệu quả đối với các tác vụ như dịch máy, phân tích cảm tính và nhận dạng thực thể được đặt tên.

Mô hình ngôn ngữ ẩn (Masked Language Model) là một loại mô hình mạng thần kinh được sử dụng trong xử lý ngôn ngữ tự nhiên (NLP) để dự đoán các từ còn thiếu trong câu. Trong Mô hình ngôn ngữ được che giấu, một số từ trong câu được che giấu ngẫu nhiên và mô hình sau đó được đào tạo để dự đoán các từ bị che giấu dựa trên ngữ cảnh của câu.

Một ví dụ nổi tiếng về Mô hình ngôn ngữ ẩn là BERT (Bidirectional Encoder Representations from Transformers), một mô hình NLP được đào tạo trước do Google phát triển. BERT được đào tạo trên một kho văn bản lớn và bằng cách đó, nó học cách hiểu mối quan hệ giữa các từ và ngữ cảnh mà chúng được sử dụng. Điều này cho phép BERT thực hiện tốt nhiều nhiệm vụ NLP, bao gồm phân tích tình cảm, phân loại văn bản và trả lời câu hỏi.

Mô hình ngôn ngữ ẩn đặc biệt hữu ích cho các nhiệm vụ liên quan đến hiểu và tạo ngôn ngữ tự nhiên, vì chúng có thể giúp cải thiện độ chính xác và trôi chảy của văn bản được tạo. Chúng cũng là một bước phát triển quan trọng trong lĩnh vực

NLP, vì chúng đã đạt được hiệu suất cao nhất trên nhiều bộ dữ liệu chuẩn và đã mở đường cho sự phát triển của các mô hình NLP tiên tiến và mạnh mẽ hơn.

2.2.3. Word-embedding

Hãy thử nhìn vào đầu vào của mô hình BERT, chúng ta cần phải từng từ và chuyển chúng thành những từ nhúng (word embedding). Khi một mạng lưới thần kinh (neural network) hoạt động trên một đoạn văn bản, chúng không thực sự tính toán các ký tự riêng lẻ trong chuỗi, mà thay vào đó chúng nhúng từ đó thành một giá trị khác. Do đó, word embedding cơ bản chỉ là một biểu diễn vector đặc trưng, đại diện cho một từ cụ thể. Và vector đặc trưng này nên được hiểu chỉ là một danh sách các số thực có âm và dương.

Word Embeddings

“Word Embedding” = Feature Vector representation of a word

$\langle 0.4125, -1.6098, 0.6047, \dots, -1.4257, -1.2321 \rangle$

Hình 2: Vector word-embedding

Một mô hình sẽ bao gồm một tập hợp các từ nhúng đã được học, mỗi model sẽ có một bảng tra từ nhúng trong đó mỗi hàng tương ứng của bảng sẽ đại diện cho một từ khác nhau.

String word



String id

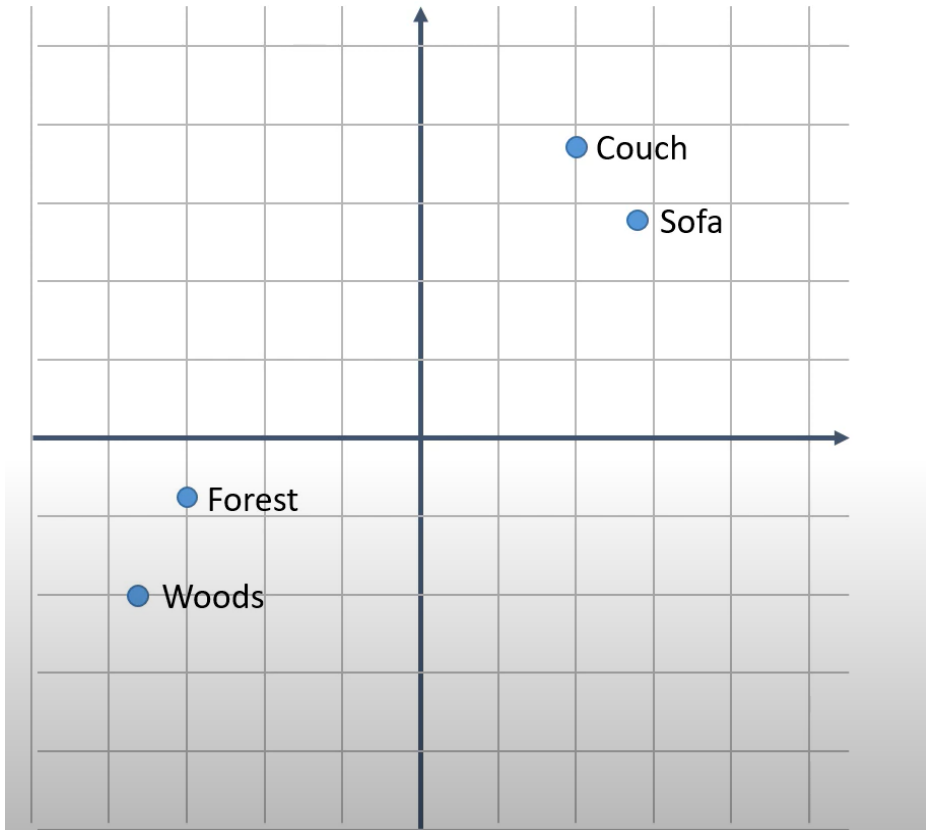
Word embedding look up table

	0	1	2	...	767
0					
...					
29999					

Hình 3: Vector word-embedding

BERT bao gồm 30000 token và mỗi token có 768 cột nhúng. Khi ta có một đoạn văn bản, ta cần phải ánh xạ từ dạng chuỗi sang id của nó bằng cách tra bảng.

Word Similarity (Từ tương đồng): Hãy tưởng tượng rằng từ nhúng của chúng ta nằm trong một mảng 2 chiều. Ta có thể trực quan hóa chúng như sau:

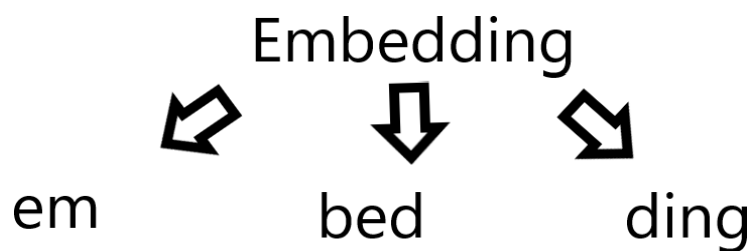


Hình 4: Biểu diễn từ trong không gian

Có thể thấy các từ mang ý nghĩa gần giống nhau sẽ được đặt ở những vị trí gần nhau (Couch, Sofa) và (Forest, Woods). Tương tự giữa Couch và Forest là hai từ không liên quan đến nhau sẽ ở xa nhau. Từ đó có thể thấy khoảng cách của các từ nhúng phản ánh độ trùng khớp giữa các từ. Vậy nên các mạng lưới thần kinh sẽ tạo ra những giá trị rất tương đồng giữa những từ khác nhau, nó là một điều rất tốt bởi vì nó có nghĩa là với tất cả những kiến thức đã học được từ chữ Couch, nó cũng sẽ áp dụng được vào chữ Sofa.

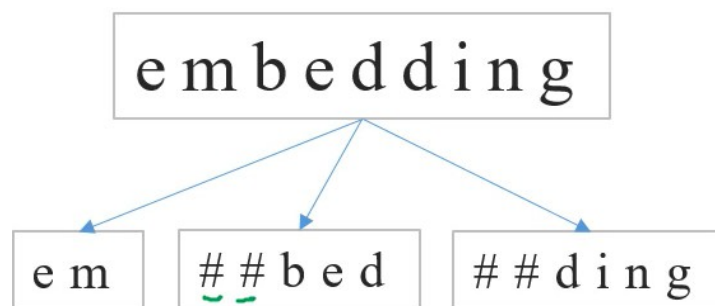
BERT's Vocabulary: BERT là một mô hình pretrained của Google và điều đầu tiên Google phải làm là thiết lập một bảng từ vựng và toàn bộ model sẽ được train xung quanh tập hợp các dữ liệu đó. Vì vậy, BERT có một bảng từ vựng cố định và chúng ta không thể thay đổi nó.

Việc không thể thay đổi nghe có vẻ khá bất tiện nếu ta gặp phải những từ không có trong bảng dữ liệu. Vậy BERT đã giải quyết như thế nào ?. Ví dụ như chúng ta có từ “embedding” là từ chưa có trong bảng từ vựng của BERT, mô hình sẽ sử dụng một mô hình phân mảnh từ và nó sẽ tách từ “embedding” thành nhiều từ nhỏ hơn. Mặc dù không có token của “embedding” nhưng nó sẽ có token của “em”, “bed” và “ding”. Do đó nếu ta có một chuỗi 10 từ vựng bao gồm từ “embedding”, ta sẽ nhận được 12 tokens thay vì 10, bởi “embedding” đã bị tách thành 3 tokens con.



Hình 5: Ví dụ về tách token

Tất cả các subword sẽ bắt đầu bằng “##”, ngoại trừ từ đầu tiên:



Hình 6: Cấu tạo của các token được tách

2.2.4. Retriever Model

Mô hình Retriever là một trong những thành phần quan trọng trong hệ thống xử lý ngôn ngữ tự nhiên (NLP) và có vai trò quan trọng trong việc tìm kiếm và truy xuất thông tin từ một nguồn dữ liệu lớn. Trong một ứng dụng NLP, khi người dùng đưa ra một câu truy vấn, mô hình Retriever sẽ xác định và trả về các văn bản có liên quan trong ngữ cảnh của câu truy vấn đó.

Một trong những ứng dụng phổ biến của mô hình Retriever là trong hệ thống hỏi-đáp (question-answering). Khi người dùng đặt một câu hỏi, mô hình Retriever sẽ truy xuất các văn bản chứa thông tin có thể đáp lại câu hỏi đó. Nhờ vào khả năng tìm kiếm và truy xuất nhanh chóng, mô hình Retriever giúp cung cấp các kết quả truy vấn chính xác và đáng tin cậy.

Mô hình Retriever sử dụng các kỹ thuật truy xuất thông tin để đánh giá độ tương đồng giữa câu truy vấn và các văn bản trong nguồn dữ liệu. Một trong những phương pháp phổ biến là sử dụng TF-IDF (Term Frequency-Inverse Document Frequency), BM25 (Best Matching 25). Các phương pháp này tính toán độ tương đồng dựa trên tần suất xuất hiện của các từ trong câu truy vấn và trong các văn bản. Các văn bản có điểm tương đồng cao sẽ được ưu tiên trả về làm kết quả truy vấn.

Mô hình Retriever cũng có thể sử dụng các phương pháp khác như phân loại văn bản để xác định xem một văn bản có thuộc vào danh mục nào hay không. Điều này có thể hữu ích trong các tác vụ như phân loại tin tức, phân loại sản phẩm và nhiều tác vụ NLP khác. Trên thực tế, mô hình Retriever thường được kết hợp với các thành phần khác trong hệ thống xử lý ngôn ngữ tự nhiên. Ví dụ, sau khi mô hình Retriever đã truy xuất các văn bản có liên quan, các văn bản này có thể được đưa vào một mô hình Reader để đọc và trả lời câu hỏi cụ thể. Mô hình Retriever có thể giúp giới hạn không gian tìm kiếm cho mô hình Reader, từ đó làm giảm đáng kể thời gian và tài nguyên tính toán.

2.2.5. Reader model

2.2.5.1. BERT

BERT (Bidirectional Encoder Representations from Transformers) là mô hình ngôn ngữ được đào tạo trước do Google phát triển cho các tác vụ xử lý ngôn ngữ tự nhiên (NLP). BERT dựa trên kiến trúc máy biến áp và được đào tạo trên lượng lớn dữ liệu văn bản theo cách không giám sát, sử dụng mục tiêu mô hình hóa ngôn ngữ ẩn (masked language modeling) và mục tiêu dự đoán câu tiếp theo (next sentence prediction).

BERT được Google cho ra mắt từ tháng 10 năm 2018 và nó đã đạt được nhiều kết quả ấn tượng đối với một số tác vụ rất khó trong NLP. Tại thời điểm đó BERT đã được công bố có những tác vụ vượt qua hiệu suất của con người.

BERT là một khái niệm học tập chuyển giao, vì vậy BERT là một kiến trúc rất lớn vì vậy nó cần rất nhiều thời gian và dữ liệu để train. Đó là bất khả thi nếu chúng ta phải train lại mô hình này từ đầu. Thật may mắn vì Google đã xuất bản mô hình BERT pretrain và nó là khả thi để tạo ra một mô hình của riêng chúng ta bằng cách fine-tuned lại theo đầu ra mình mong muốn

Sau khi đào tạo trước (pre-training), BERT có thể được tinh chỉnh (fine-tune) trên một loạt các nhiệm vụ NLP xuôi dòng (downstream task), chẳng hạn như phân tích tình cảm, nhận dạng thực thể được đặt tên và trả lời câu hỏi. Tinh chỉnh liên quan đến việc thêm một lớp dành riêng cho nhiệm vụ lên trên mô hình BERT đã được đào tạo trước và đào tạo toàn bộ mô hình trên một tập dữ liệu dành riêng cho nhiệm vụ nhỏ hơn. Ở thời điểm hiện tại BERT đã được sử dụng cho rất nhiều ngôn ngữ trong đó có cả tiếng Việt với model PhoBERT và đạt được những kết quả tốt cho những tác vụ NLP.

BERT cũng đã truyền cảm hứng cho sự phát triển của các mô hình ngôn ngữ được đào tạo trước khác, chẳng hạn như RoBERTa, ALBERT và ELECTRA, những mô hình này đã cải thiện hơn nữa công nghệ tiên tiến nhất trong nghiên cứu và ứng dụng NLP. Ngoài ra, BERT còn được thiết kế để huấn luyện trước word-

embedding mà ta sẽ tìm hiểu ở phần sau.

2.2.5.2. PhoBERT

PhoBERT là một mô hình ngôn ngữ tự nhiên được phát triển dựa trên kiến trúc transformer và huấn luyện trên dữ liệu tiếng Việt. Mô hình này được xây dựng dựa trên mô hình ngôn ngữ BERT (Bidirectional Encoder Representations from Transformers) của Google nhưng được điều chỉnh và tinh chỉnh để phù hợp với ngôn ngữ và ngữ cảnh tiếng Việt. Mô hình có khả năng hiểu và biểu diễn ngôn ngữ tiếng Việt một cách hiệu quả. Nó có thể xử lý nhiều tác vụ ngôn ngữ như phân loại văn bản, trích xuất thông tin, dịch máy, và hỏi-đáp. Mô hình này đã được huấn luyện trên một lượng lớn dữ liệu tiếng Việt từ các nguồn trực tuyến, bao gồm cả báo chí, trang web, diễn đàn, và văn bản từ các lĩnh vực khác nhau.

PhoBERT sử dụng kiến trúc transformer để tổ chức thông tin và học cách biểu diễn ngôn ngữ. Transformer cho phép mô hình có khả năng chú ý đa lớp và xử lý cùng lúc thông tin từ cả hai phía của một câu, giúp mô hình hiểu được ngữ cảnh và sự phụ thuộc giữa các từ trong văn bản. Điều này giúp PhoBERT đạt được hiệu suất cao trong việc hiểu và xử lý ngôn ngữ tự nhiên tiếng Việt. PhoBERT đã trở thành một công cụ hữu ích trong lĩnh vực xử lý ngôn ngữ tự nhiên tiếng Việt. Nó đã được áp dụng trong nhiều ứng dụng thực tế như chatbot, hệ thống hỏi-đáp tự động, phân loại văn bản, và dịch máy. Sự phát triển và sử dụng PhoBERT đã đóng góp đáng kể vào việc nâng cao khả năng xử lý ngôn ngữ tự nhiên tiếng Việt và mở ra nhiều cơ hội trong việc phát triển các ứng dụng NLP tiếng Việt.

2.2.5.3. GPT

GPT (Generative Pre-trained Transformer) là một mô hình ngôn ngữ được phát triển bởi OpenAI. Nó sử dụng kiến trúc Transformer và được huấn luyện trên một tập dữ liệu khổng lồ gồm văn bản và mã. GPT có thể thực hiện nhiều tác vụ xử lý ngôn ngữ tự nhiên (NLP) như tạo văn bản, dịch ngôn ngữ, trả lời câu hỏi, tóm tắt văn bản...

GPT sử dụng kiến trúc Transformer để học mối quan hệ giữa các từ trong

một câu. Transformer là một kiến trúc mạng nơ-ron được Google Research phát triển vào năm 2017. Nó đã đạt được kết quả ấn tượng trong nhiều tác vụ NLP. Kiến trúc được huấn luyện trên một tập dữ liệu khổng lồ gồm văn bản và mã. Tập dữ liệu này bao gồm các văn bản từ sách, bài báo, trang web và mã từ các kho lưu trữ GitHub. Vì vậy đây là một mô hình ngôn ngữ mạnh mẽ và tốt cho vai trò là reader model.

2.3. Các kỹ thuật huấn luyện mô hình

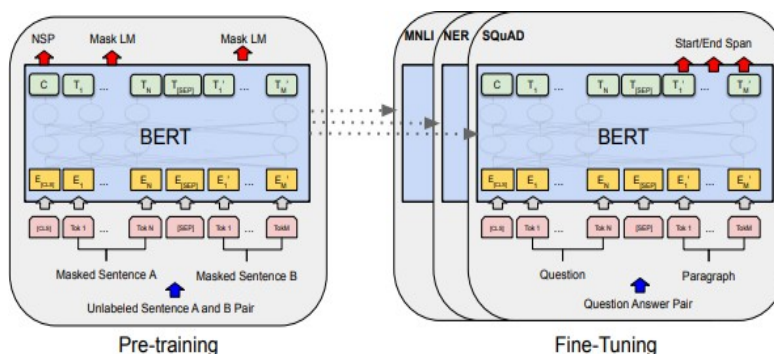
2.3.1. Fine-tuning

Fine-tuning có nghĩa là sử dụng một model đã được train sẵn trên một tập dữ liệu lớn, ta chỉ cần làm lại thêm một chút trong sản phẩm của chúng ta. BERT đã được train trên một tập dữ liệu rất lớn, vì vậy chúng ta đã có một model hiểu khá nhiều về ngôn ngữ của chúng ta.

Vì lý do trên, chúng ta không dùng BERT ngay lập tức, mà chúng ta phải bỏ thời gian ra để tinh chỉnh lại model một chút cho phù hợp với đầu ra của bài toán ta đang hướng đến. So với tạo lại model từ rác thì việc tinh chỉnh lại model đã có sẵn giúp tiết kiệm rất nhiều thời gian phát triển.

Để có được một cơ sở dữ liệu lớn, ta phải tốn rất nhiều thời gian và tiền bạc để phát triển. Nhờ BERT, ta có thể train với số lượng dữ liệu ít hơn. BERT Transformer hiện tại đã có rất nhiều mô hình đã được tinh chỉnh nhằm phù hợp với từng tác vụ cụ thể. Nhờ đó, ta có thể thu được kết quả tốt hơn.

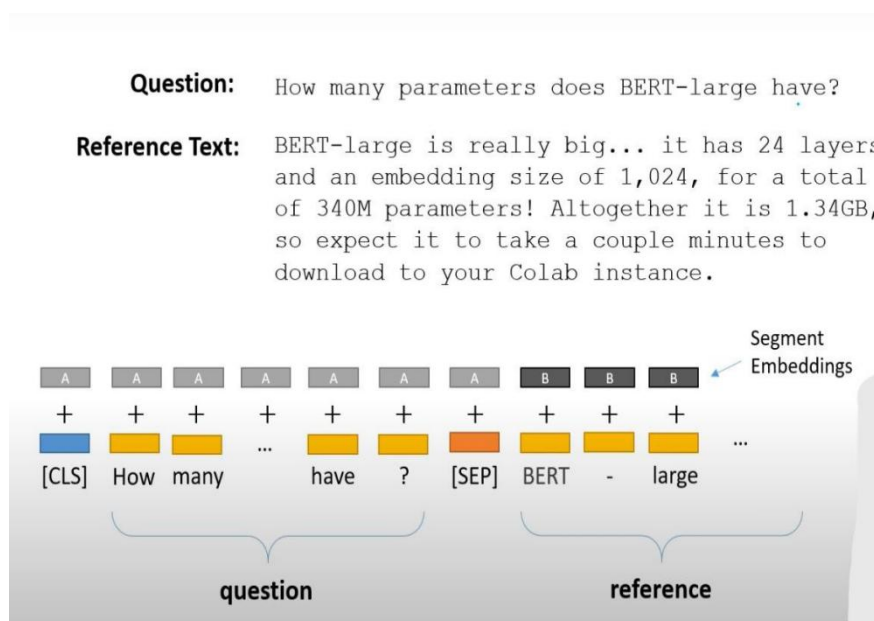
Fine-tuning question and answering model:



Hình 7: Minh họa fine-tuning BERT(<https://www.researchgate.net/>)

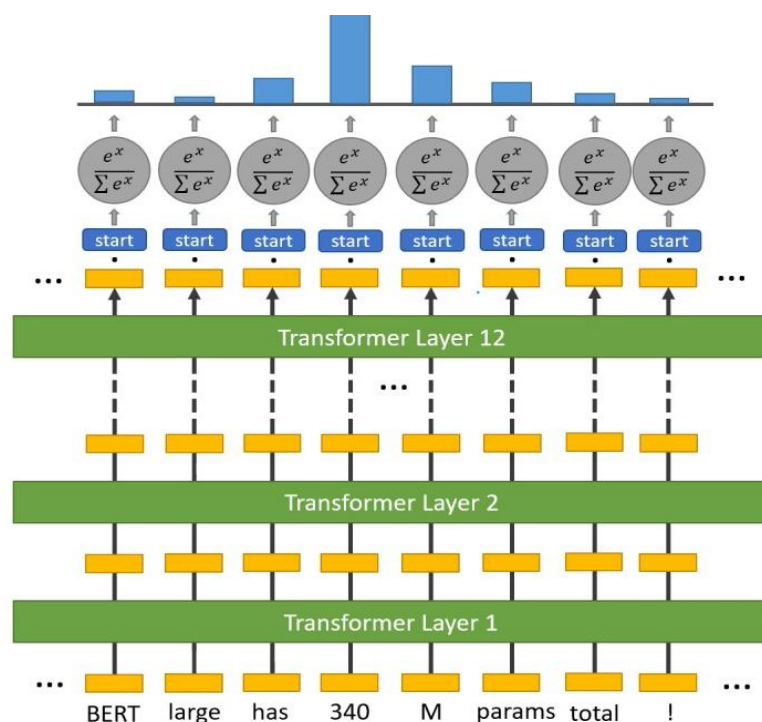
Hình trên cho ta thấy tiến trình fine-tuning của BERT. Ta sẽ có các bước tiến hành fine-tuning như sau:

Trước tiên, ta luôn bắt đầu với một [CLS] token và một [SEP] token ở giữa nhằm phân tách giữa câu hỏi và đoạn văn bản đọc hiểu đó chính là embedding.



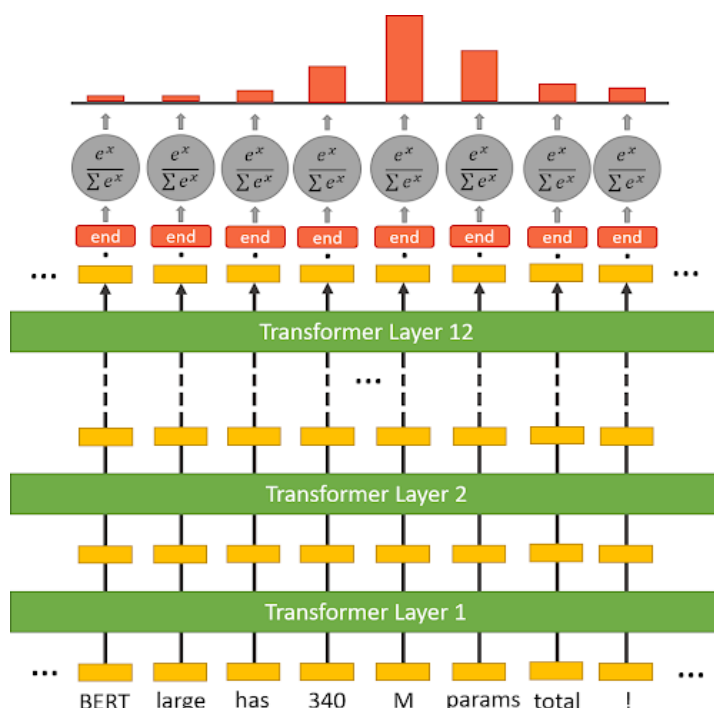
Hình 8: Ví dụ về cấu tạo câu khi tiến hành embedding (<https://www.researchgate.net/>)

Tiếp theo cho mỗi lần embedding, qua 12 lớp ta sẽ thu được một vector output ở cuối quá trình.



Hình 9: Cấu trúc lớp input (<https://www.researchgate.net/>)

Cuối cùng để dự đoán xác suất cho vị trí đầu và cuối của câu trả lời. Ta lấy tỷ trọng của từng token chia cho tổng tỉ trọng và softmax để phân phối xác suất cho các output tương ứng.



Hình 10: Cấu trúc lớp output (<https://www.researchgate.net/>)

2.3.2. Prompt engineering

Prompt engineering là một kỹ thuật quan trọng trong việc tương tác và khai thác hiệu quả các Mô hình ngôn ngữ lớn (LLM) hiện đại như GPT-3, PaLM hoặc Claude. Kỹ thuật này bao gồm việc thiết kế và tạo ra các lời nhắc một cách thông minh để hướng dẫn mô hình đưa ra câu trả lời hoặc hoàn thành nhiệm vụ theo cách mong muốn.

Viết prompt hiệu quả đòi hỏi sự hiểu biết sâu sắc về cách hoạt động của mô hình LLM và những hạn chế của chúng. Các nhà nghiên cứu và kỹ sư phải tìm cách trình bày bối cảnh, bối cảnh và câu hỏi một cách rõ ràng, ngắn gọn nhưng đầy đủ thông tin để đưa mô hình đi đúng hướng. Điều này đòi hỏi kỹ năng sử dụng ngôn ngữ cũng như kiến thức chuyên môn về lĩnh vực cần giải quyết.

Một số kỹ thuật và chiến lược được sử dụng trong kỹ thuật nhắc nhở bao gồm: cung cấp ví dụ (few-shot learning), sử dụng từ khóa hoặc câu hướng dẫn (instructional prompting), đưa ra các giới hạn và hướng dẫn rõ ràng (constrained

prompting), kết hợp nhiều lời nhắc chain-of-thought prompting) , thậm chí còn dạy mẫu viết lời nhắc một cách tự động (prompt tutoring). Ngoài ra, thiết kế prompt cũng cần xem xét các yếu tố như an toàn, công bằng và đạo đức để tránh tạo ra nội dung có hại, phân biệt đối xử hoặc vi phạm đạo đức. Điều này đòi hỏi cách diễn đạt cẩn thận, sử dụng ngôn ngữ trung lập, tránh rập khuôn hoặc những bình luận không phù hợp.

Kỹ thuật prompt engineering đóng một vai trò quan trọng trong việc khai thác toàn bộ tiềm năng của các mô hình LLM, giúp chúng thực hiện các nhiệm vụ phức tạp và cung cấp đầu ra chất lượng cao. Nó không chỉ giúp cải thiện hiệu suất mô hình mà còn mở ra cơ hội cho các ứng dụng mới trong nhiều lĩnh vực khác nhau như trợ lý ảo, tóm tắt văn bản, dịch máy, phân tích dữ liệu, v.v. Khi các mô hình LLM ngày càng trở nên mạnh mẽ hơn, kỹ thuật prompt engineering sẽ trở thành một kỹ năng thiết yếu để khai thác triệt để khả năng của chúng.

2.3.3. Few-shot learning

Few-shot learning là một kỹ thuật đào tạo các mô hình trí tuệ nhân tạo để học từ một số lượng mẫu dữ liệu rất hạn chế, thường chỉ từ vài đến vài chục ví dụ (few shots). Đây là một cách tiếp cận quan trọng để giải quyết thách thức xây dựng các mô hình AI có thể học hỏi và thích ứng nhanh chóng, giống như cách con người học.

Few-shot learning cho phép mô hình AI học và thực hiện các nhiệm vụ mới từ một số ít mẫu huấn luyện, thay vì cần hàng nghìn hoặc hàng triệu mẫu như trong học máy truyền thống. Khi được cung cấp một số ví dụ về một nhiệm vụ mới, mô hình có thể suy ra và nắm bắt các nguyên tắc và mô hình cơ bản để thực hiện nhiệm vụ đó.

Few-shot learning thường bao gồm meta-learning (học cách học), đào tạo đa tác vụ, trích xuất kiến thức từ các mô hình được đào tạo trước lớn (pre-train model) và sử dụng các kỹ thuật như thêm ví dụ, điều chỉnh độ dốc (gradient) hoặc tối ưu hóa meta. Việc thiết kế cấu trúc mô hình phù hợp và khai thác kiến thức nền cũng

đóng vai trò quan trọng.

Few-shot learning là một bước quan trọng để xây dựng hệ thống AI thực sự thông minh và linh hoạt, có khả năng học hỏi và thích ứng nhanh như con người. Nó làm giảm nhu cầu về dữ liệu đào tạo lớn, cho phép lập mô hình hình học các nhiệm vụ và kiến thức mới một cách nhanh chóng và hiệu quả, điều này rất quan trọng đối với nhiều ứng dụng trong thế giới thực. Mặc dù vẫn còn nhiều thách thức cần giải quyết, những phương pháp học vài lần đang trở thành một lĩnh vực nghiên cứu sôi động và được kỳ vọng sẽ dẫn đến những bước tiến lớn trong việc xây dựng trí tuệ nhân tạo thông minh hơn và linh hoạt hơn. trong tương lai gần.

2.4. Pipeline

Thư viện Hugging Face Transformers cung cấp giao diện cấp cao để xây dựng và triển khai các mô hình xử lý ngôn ngữ tự nhiên (NLP) cho nhiều tác vụ khác nhau, chẳng hạn như phân loại văn bản, trả lời câu hỏi và tạo văn bản. Một trong những tính năng chính của thư viện là API đường dẫn, cho phép người dùng dễ dàng áp dụng các mô hình được đào tạo trước cho dữ liệu mới để suy luận. API pipeline cung cấp một giao diện đơn giản để thực hiện các tác vụ NLP phổ biến, chẳng hạn như phân loại văn bản và nhận dạng thực thể được đặt tên, sử dụng các mô hình được đào tạo trước.

2.5. Các phương pháp đánh giá

2.5.1. Cosine similarity

Độ tương tự cosine là thước đo độ tương tự giữa hai vector khác 0 của không gian tích bên trong, chẳng hạn như không gian vector của từ hoặc tài liệu trong xử lý ngôn ngữ tự nhiên (NLP). Nó đo cosin của góc giữa hai vector, nằm trong khoảng từ -1 đến 1, với các giá trị càng cao cho thấy độ tương tự càng lớn.

Trong NLP, độ tương tự cosine thường được sử dụng để tính toán độ tương tự giữa hai tài liệu hoặc câu dựa trên biểu diễn vector của chúng. Một phương pháp phổ biến để tính toán các biểu diễn vector của văn bản là sử dụng các từ nhúng, ánh

xạ từng từ trong từ vựng thành một vector chiều cao. Sau đó, biểu diễn vector của một tài liệu hoặc câu có thể được tính là giá trị trung bình của các vector cho các từ cấu thành của nó.

$$\text{cosine_similarity}(A, B) = (A \cdot B) / (\|A\| * \|B\|)$$

Trong đó $A \cdot B$ là tích vô hướng của hai vector và $\|A\|$ và $\|B\|$ là các chuẩn L2 của hai vector.

Độ tương tự cosine là một thước đo độ tương tự hữu ích trong NLP, vì nó tương đối đơn giản để tính toán và có thể nắm bắt được các mối quan hệ quan trọng giữa các từ và tài liệu.

2.5.2. Exact Match

Exact Match hay là mức độ khớp chính xác là một khái niệm chỉ trạng thái giống nhau hoàn toàn trong xử lý ngôn ngữ tự nhiên. Khi dùng exact match để đánh giá kết quả, hai câu phải sao chép y nhau để có thể gọi là exact.

Khi ta áp dụng exact match trong việc so sánh tác vụ QA, văn bản được model sinh ra phải khớp hoàn toàn với đáp án cho trước. Đây là một cách đánh giá kết quả trực tiếp nhất về mô hình xử lý QA nhưng exact match lại là thang đo quá chặt chẽ và không linh hoạt nếu một câu hỏi có nhiều câu trả lời, hoặc câu trả lời có nhiều từ đồng nghĩa. Ví dụ “Tôi đi học” và “Tôi đi học” là exact. Còn “Tôi bị ho” và “Tôi bị hơ” là non-exact và σ và o là không tương đồng. Do sự quá chi tiết nên Exact match không được sử dụng nhiều mà chỉ đóng vai trò nhỏ trong việc đánh giá một mô hình reader QA.

2.5.3. ROUGE Metric

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) là một tập hợp các phép đo được sử dụng để đánh giá chất lượng của các hệ thống tạo ra tóm tắt văn bản tự động. ROUGE tạo ra giúp tính độ tương đồng giữa câu tóm tắt với câu trả lời cho trước. Vì vậy, ROUGE được ưu tiên sử dụng hơn Exact match.

Thay vì, so sánh cứng từng vị trí như Exact Match, ROUGE tiếp cận bằng

cách so sánh độ khớp (overlap) giữa các cụm từ để tham chiếu. Ta có một số đo ROUGE phổ biến:

ROUGE-N: Đo độ tương đồng dựa trên số lượng cụm từ chung (n-grams) giữa hai tập hợp câu. Ví dụ: ROUGE-1 tính số lượng từ đơn chung, ROUGE-2 tính số lượng cặp từ chung, và ROUGE-3 tính số lượng bộ ba từ chung.

ROUGE-L: Đo độ tương đồng dựa trên độ dài của các dãy con chung (longest common subsequences) giữa hai tập hợp câu. ROUGE-L giúp đo lường sự tương đồng dựa trên cấu trúc câu và thứ tự các từ.

ROUGE-S: Đo độ tương đồng dựa trên số lượng cụm từ chung và các khoảng cách giữa các cụm từ trong câu. ROUGE-S giúp đo lường sự tương đồng dựa trên cấu trúc câu và sự tương đương ngữ nghĩa.

ROUGE là thang đo được sử dụng rộng rãi và ứng dụng nhiều trong lĩnh vực đánh giá mô hình ngôn ngữ tự nhiên.

2.5.4. Sentences similarity

Sentences similarity hay còn gọi là sự tương đồng giữa các câu là một tác vụ quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP), trong đó mô hình được tạo ra nhằm so sánh mức độ tương đồng giữa hai đoạn văn bản dựa vào đặc trưng riêng của từng ngôn ngữ, ngữ pháp hay ngữ cảnh khác nhau. Có nhiều phương pháp để so sánh sự tương đồng giữa các câu:

- Distance-based methods (Tính toán dựa trên khoảng cách): Cách tính toán này dựa trên việc đo khoảng cách vector giữa các câu dựa trên các phép tính như khoảng cách Euclidean, khoảng cách Cosine (sẽ đề cập trong phần sau),...
- Textual similarity metrics (Chỉ số tương đồng) : Sử dụng các phép đo tương đương như TF-IDF (Term Frequency - Inverse Document Frequency), BM25 (Best Matching 25) hoặc Word2Vec để tính độ tương đồng. Các phép tính này được đề tài ứng dụng ở mô hình truy vết.
- Deep learning models: Sử dụng các mô hình học sâu như BERT, SNN

(Siamese Neural Networks) nhằm đưa ra độ tương đồng sâu hơn về ngữ nghĩa.

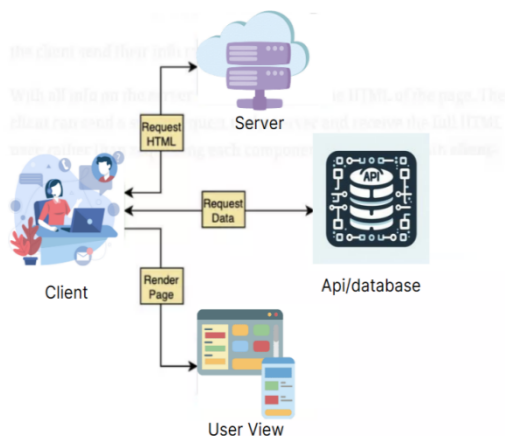
Mục đích của việc tính toán độ tương đồng câu là để xác định mức độ tương đồng giữa các câu, từ đó có thể áp dụng cho nhiều ứng dụng như tìm kiếm thông tin, phân loại văn bản, dịch máy, và hệ thống hỏi đáp.

2.6. Framework Next.js

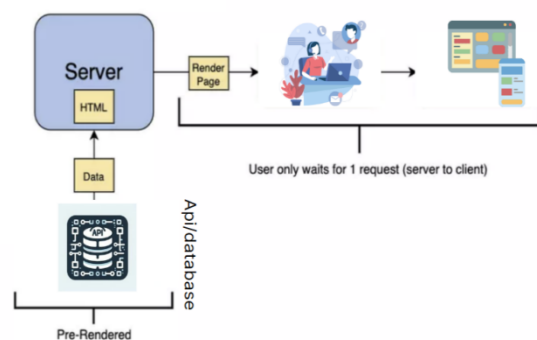
Next.js, một framework phát triển web dựa trên React, đã trở thành công cụ không thể thiếu trong việc xây dựng các ứng dụng web đa năng. Với khả năng tối ưu hóa tự động, Next.js đẩy nhanh quá trình tải trang thông qua server-side rendering (SSR), đồng thời cải thiện đáng kể khả năng tìm kiếm trên các công cụ tìm kiếm. Điều này không chỉ giúp nâng cao trải nghiệm người dùng mà còn tối ưu hóa hiệu suất của trang web. Ngoài ra, Next.js còn tích hợp sẵn các công cụ tối ưu hóa hình ảnh, cho phép các hình ảnh được tải nhanh hơn và hiển thị mượt mà hơn. Sự linh hoạt trong phát triển là một trong những ưu điểm nổi bật của Next.js, hỗ trợ các nhà phát triển triển khai nhanh chóng và dễ dàng các tính năng mới.

Về mặt kiến trúc, Next.js áp dụng mô hình monorepo, cho phép quản lý và phát triển đồng thời nhiều gói mã nguồn trong một kho lưu trữ Git duy nhất. Điều này tạo điều kiện cho việc bảo trì và cập nhật mã nguồn trở nên thuận tiện và hiệu quả hơn.

Client-Side Rendering



Server-Side Rendering



Hình 11: Kiến trúc Next.js

Lựa chọn Next.js cho việc phát triển chatbot hỗ trợ trả lời học vụ không chỉ dựa trên hiệu suất và khả năng tối ưu hóa SEO mà còn bởi khả năng phát triển nhanh chóng và dễ dàng bảo trì. Đây là những yếu tố quan trọng để đảm bảo rằng ứng dụng có thể cung cấp thông tin chính xác và truy cập nhanh chóng, đáp ứng nhu cầu của người dùng.

Đặc biệt, SSR tích hợp trong Next.js giúp tăng cường hiệu suất và SEO bằng cách xử lý thông tin trên server và tạo ra HTML của trang. Điều này giúp giảm thời gian tải trang so với việc render từng thành phần riêng lẻ trên client. Hơn nữa, Next.js còn hỗ trợ code splitting tự động, tối ưu hóa hiệu suất khi tải trang, và cho phép dễ dàng xây dựng các API nội bộ thông qua API routes tích hợp sẵn. Sự hỗ trợ mạnh mẽ cho các công nghệ như CSS, JSX và TypeScript cùng khả năng tùy chỉnh linh hoạt thông qua plugin làm cho Next.js trở thành lựa chọn lý tưởng cho việc phát triển website Chatbot hỗ trợ học vụ.

Chương 3. KIẾN TRÚC HỆ THỐNG CHATBOT

Chúng em đã phân tích kiến trúc và quy trình xây dựng chatbot cho hệ thống hỏi đáp tự động, tập trung vào kiến trúc Retrieval-Augmented Generation (RAG). Trong đó, RAG kết hợp giữa việc truy xuất thông tin và tạo sinh văn bản để tạo ra các câu trả lời tự nhiên và chính xác. Chúng em đã phát triển các phương pháp như “Clause is all you need” và “Information Augmentation” để cải thiện hiệu suất truy vết và độ chính xác của câu trả lời. RAG bao gồm năm phần chính: nhận truy vấn từ người dùng, xử lý đầu vào, tìm kiếm dữ liệu từ các nguồn kiến thức, đánh giá các câu trả lời tiềm năng, và sử dụng các mô hình ngôn ngữ tiên tiến như PhoBERT và GPT để tạo ra các câu trả lời tự nhiên. Quá trình chuẩn bị dữ liệu, tiền xử lý, phân chia dữ liệu và lưu trữ dữ liệu đều được tối ưu hóa để đảm bảo chất lượng và hiệu suất của hệ thống. Chúng em cũng đã đề xuất và triển khai các phương pháp mới như sử dụng dictionary để loại bỏ các từ viết tắt, tổng hợp lịch sử cuộc trò chuyện, và kỹ thuật MultiQueryRetriever và Parent Document Retriever để cải thiện khả năng truy vết và độ chính xác của câu trả lời. Cuối cùng, PhoBERT và GPT được sử dụng để tổng hợp và sinh ra câu trả lời cuối cùng, đảm bảo tính nhất quán, mạch lạc và chính xác của thông tin.

3.1. Phân tích kiến trúc một chatbot

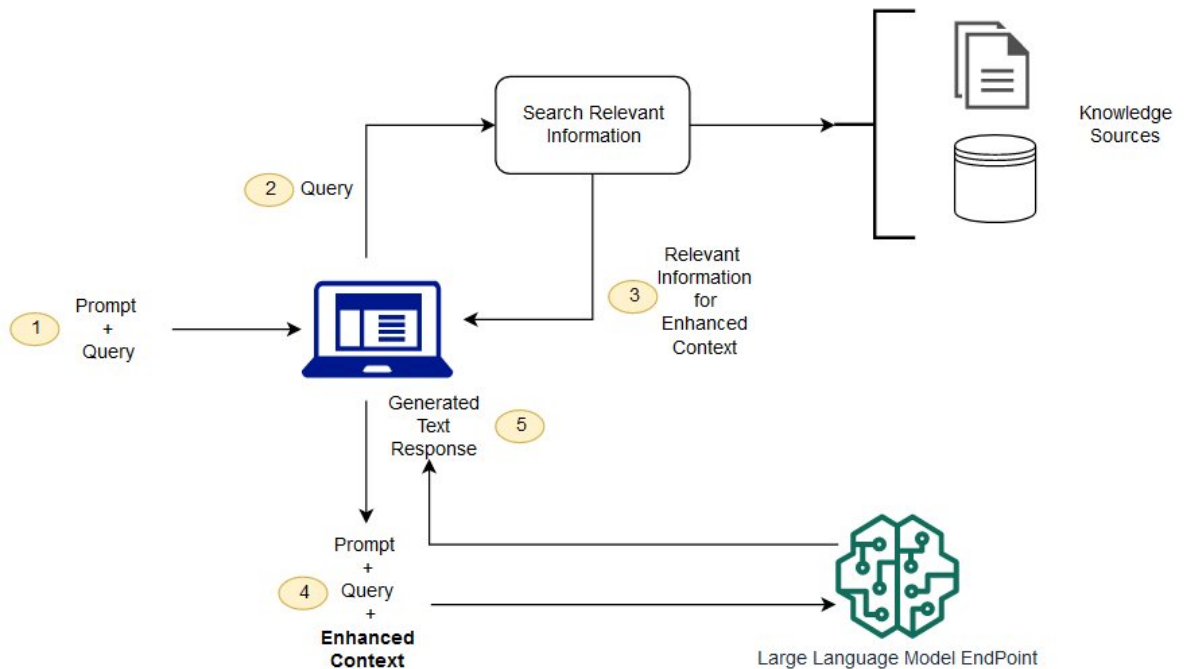
Như đã giới thiệu ở Chương 2, bài toán Open Domain Question Answering, một chatbot có thể chia làm ba loại chính: Retriever - Reader, Retriever - Generator hoặc Full Generator. Phương pháp Retriever - Reader và Retriever - Generator yêu cầu một bộ truy vết (Retriever) đủ tốt để làm tiền đề cho việc tìm ra câu hỏi, điểm khác biệt ở đây là quá trình đưa ra câu trả lời. Reader thường là những mô hình đọc hiểu (Reading Comprehension model), ở đó mô hình đưa ra output là vị trí khởi đầu - vị trí kết thúc của một đoạn văn bản (văn bản ở đây là output của Retriever model). Một số mô hình Reader có thể kể đến như BERT, RoBERTa, Longformer v.v. Mặc dù đã được đào tạo trên một tập dữ liệu lớn nhưng việc diễn giải ngôn ngữ chưa tốt, do mô hình chỉ đưa ra được câu trả lời có sẵn từ đoạn văn bản. Generator là một

phương pháp cải tiến hơn, nó giúp diễn giải câu trả lời gần với cuộc trò chuyện tự nhiên nhất. Không cần dữ liệu truy vết, Full Generator có thể tạo ra các câu trả lời phù hợp với dữ liệu đã train, nhưng nhược điểm lớn nhất của Full Generator là đòi hỏi lượng dữ liệu đặc thù lớn và cơ chế thiên vị trong dữ liệu đào tạo, dẫn đến các câu trả lời có thể sai hoặc hiểu lầm. Một số bài báo nghiên cứu về vấn đề so sánh Retriever - Reader hoặc Retriever - Generator với Full Generator (<https://openreview.net/pdf?id=fB0hRu9GZUS>), mặc dù một số báo cáo chỉ ra rằng Full Generator có một số lợi thế nhưng việc đòi hỏi một lượng dữ liệu lớn là cản trở lớn nhất để thực hiện. Vì vậy, chúng em đề cử kiến trúc RAG - một kiến trúc thuộc Retriever - Generator, nhằm đào tạo một chatbot phục vụ cho nhu cầu tuyển sinh, học vụ trong trường.

Chatbot RAG (Retrieval Augmented Generation) là một mô hình ngôn ngữ tiên tiến sử dụng kiến trúc kết hợp giữa truy xuất thông tin và tạo sinh văn bản để tạo ra các cuộc trò chuyện tự nhiên và hấp dẫn. Chúng em đã tiến hành xây dựng quy trình và kiến trúc mô hình RAG, đồng thời chúng em cũng phát triển những phương pháp mới nhằm tối ưu hiệu suất và cải thiện chất lượng trả lời. Các phương pháp mới mà chúng em đề xuất là “Clause is all you need” và “Information Augmentation” sẽ được chúng em trình bày ở phần sau.

3.1.1. Kiến trúc tổng thể

Một chatbot RAG có thể được chia làm 5 phần chính sau:



Hình 12: Sơ đồ quy trình RAG cơ bản (<https://tinyurl.com/ys7srps5>)

RAG nhận truy vấn từ người dùng, truy vấn có thể dưới dạng văn bản hoặc giọng nói. (2) Hệ thống nhận diện và xử lý đầu vào để hiểu ý nghĩa của nó, hệ thống áp dụng các kỹ thuật NLP để phân tích cú pháp, ngữ nghĩa và ngữ cảnh của truy vấn. Mục tiêu là xác định các thông tin quan trọng như chủ đề, ý định và thực thể trong truy vấn. (3) Từ truy vấn đó, mô hình sẽ tiến hành tìm kiếm dữ liệu từ “Knowledge Sources”, các kỹ thuật tìm kiếm được sử dụng để xác định các câu trả lời có liên quan đến truy vấn của người dùng. Các yếu tố như độ tin cậy, tính phù hợp và tính trôi chảy được sử dụng để đánh giá chất lượng của các câu trả lời. (4) Hệ thống đánh giá các câu trả lời tiềm năng dựa trên các tiêu chí như độ tin cậy, tính phù hợp và tính trôi chảy. Một tập con nhỏ các câu trả lời có điểm đánh giá cao nhất được lựa chọn để sử dụng trong giai đoạn tiếp theo. (5) Hệ thống sử dụng các mô hình ngôn ngữ tiên tiến như BERT hoặc Transformer để tạo ra các câu trả lời trôi chảy và tự nhiên. Mục tiêu là tạo ra các câu trả lời phù hợp với ngữ cảnh và đáp ứng nhu cầu của người dùng.

3.1.2. Chuẩn bị dữ liệu

Như đã đề cập ở phần trên “Knowledge Sources” đóng vai trò then chốt trong RAG. Knowledge Sources là một tập hợp các tài liệu, nội dung văn bản chứa thông tin, dữ liệu và kiến thức đa dạng về nhiều lĩnh vực và chủ đề khác nhau. Nó đóng vai trò là kho lưu trữ tri thức bên ngoài mà mô hình RAG có thể truy cập và khai thác để tìm kiếm câu trả lời phù hợp cho các câu hỏi đầu vào. Sự phong phú và chất lượng của nguồn tri thức có tác động trực tiếp đến hiệu suất và hiệu quả của mô hình RAG, là yếu tố quyết định khả năng trả lời các câu hỏi phức tạp đòi hỏi kiến thức sâu rộng ngoài khả năng trả lời các câu hỏi phức tạp. Khả năng của hệ thống mô hình chỉ dựa vào kiến thức được đào tạo trước. Nguồn kiến thức toàn diện, đa dạng và đáng tin cậy cho phép mô hình RAG đưa ra câu trả lời chính xác và hợp lý cho nhiều câu hỏi thuộc nhiều lĩnh vực khác nhau, từ kiến thức Kiến thức chuyên ngành chuyên sâu cho các vấn đề thời sự và kiến trúc chung.

Xây dựng nguồn tri thức lý tưởng là một thách thức đáng kể trong việc triển khai mô hình RAG vì nó đòi hỏi phải thu thập, tích hợp và tổ chức một lượng lớn dữ liệu văn bản đa dạng từ nhiều nguồn khác nhau. Quá trình này đòi hỏi phải phân loại và tổ chức cẩn thận để đảm bảo thông tin được trình bày một cách có hệ thống, dễ dàng truy cập và cập nhật liên tục. Các chiến lược lọc, chuẩn hóa và loại bỏ dữ liệu trùng lặp và sai lệch cũng rất quan trọng để duy trì chất lượng và tính nhất quán của các nguồn kiến thức. Ngoài ra, vấn đề bảo mật, nhạy cảm dữ liệu cũng là yếu tố cần được quan tâm nghiêm túc trong quá trình xây dựng và quản lý nguồn lực tri thức. Về khía cạnh kỹ thuật và hiệu suất, nguồn tri thức cần đáp ứng yêu cầu về quy mô lớn, phân tán và truy cập nhanh để hỗ trợ mô hình RAG có thể thực hiện truy vấn và trích xuất thông tin. một cách hiệu quả và kịp thời. Điều này đòi hỏi các nguồn tri thức phải được lưu trữ và tổ chức bằng cách sử dụng các cấu trúc và công nghệ dữ liệu phù hợp, cho phép truy xuất, tìm kiếm và lọc nội dung nhanh chóng. Các kỹ thuật như đối sánh vector mật độ (Dense Retrieval), Lập chỉ mục đảo ngược (Inverted Indexing), Cây Trie hoặc lưu trữ cơ sở dữ liệu phân tán đóng vai trò quan trọng trong việc đảm bảo hiệu suất truy xuất. thông tin từ các nguồn tri thức cho mô

hình RAG.

Để xây dựng nguồn tri thức nói riêng cho vấn đề học vụ và tuyển sinh, chúng em đã lựa chọn tài nguyên từ những nguồn tin cậy, được xác định của trường như: ou.edu.vn, tienichsv.ou.edu.vn, it.ou.edu.vn, Sổ tay sinh viên khóa 2023. Sau quá trình thu thập, dữ liệu cần được xử lý trước để loại bỏ nhiễu, trùng lặp dữ liệu và chuẩn hóa định dạng. Điều này bao gồm các hoạt động như tách văn bản khỏi định dạng nguồn (PDF, HTML, v.v.), xóa các ký tự đặc biệt, mã hóa không chuẩn, chuẩn hóa biểu đồ và hình ảnh cũng như xử lý dữ liệu bị hỏng hoặc thiếu. Sau khi hoàn thành những bước trên, một “Knowledge Sources” cơ bản đã hoàn thiện. Nhưng để nâng cao chất lượng của dữ liệu, chúng em đã đề xuất một phương pháp mới. Chúng em gọi đó là “Clause is all you need” (Sẽ được trình bày ở phần Tiền xử lý dữ liệu).

3.1.3. Tiền xử lý dữ liệu

Trong quá trình triển khai mô hình Retrieval-Augmented Generation (RAG), bước tiền xử lý dữ liệu đóng một vai trò quan trọng để chuẩn bị nguồn tri thức (knowledge source) và đảm bảo chất lượng của dữ liệu đầu vào cho mô hình. Tiền xử lý dữ liệu trong RAG bao gồm các hoạt động làm sạch, chuẩn hóa và biến đổi dữ liệu thô thành định dạng phù hợp để đầu vào cho mô hình. Quá trình này giúp loại bỏ nhiễu, sửa lỗi, xử lý dữ liệu bị thiếu hoặc không nhất quán, và cải thiện chất lượng dữ liệu. Đồng thời, tiền xử lý dữ liệu cũng đóng vai trò lớn trong việc nâng cao khả năng truy vết dữ liệu bằng cách nâng cao chất lượng của “Knowledge Sources”.

Trong quá trình thực nghiệm với chatbot, chúng em nhận ra sự khác biệt lớn giữa hiệu suất trả lời theo những câu truy vấn của người dùng. Cụ thể ở đây là chính tả và viết tắt, với những câu hỏi càng rõ ràng, đúng chính tả và ít viết tắt, mô hình sẽ hiểu rõ ngữ cảnh câu hỏi hơn, từ đó đưa ra được câu trả lời mong muốn. Chúng em đề xuất một phương pháp đơn giản nhưng hiệu quả là sử dụng một từ điển viết tắt. Khi người dùng nhập câu truy vấn vào chatbot, câu truy vấn đó sẽ được so khớp với từ điển và trả về dạng đúng đắn của nó.

Xử lý chính tả và viết tắt trong tiền xử lý truy vấn là bước quan trọng vì nó giúp tăng cường khả năng hiểu ý định của người dùng và cải thiện kết quả tìm kiếm và trả lời câu hỏi. Lỗi chính tả và viết tắt có thể dẫn đến sự hiểu nhầm ý nghĩa của truy vấn, từ đó ảnh hưởng đến kết quả cuối cùng. Bằng cách sửa lỗi và giải mã, hệ thống có thể hiểu chính xác hơn nhu cầu của người dùng và cung cấp câu trả lời phù hợp. Ngoài ra, xử lý chính tả và viết tắt cũng giúp cải thiện trải nghiệm người dùng bằng cách giảm bớt nhu cầu nhập lại truy vấn và tăng cường tính linh hoạt trong cách nhập liệu. Điều này đặc biệt quan trọng trong các ứng dụng trợ lý ảo hoặc giao diện tự nhiên, nơi người dùng mong muốn một trải nghiệm liền mạch và tự nhiên.

3.1.3.1. Clause is all you need

Khi chúng em thu thập dữ liệu để tạo thành “Knowledge Sources”, các tài liệu thường được viết theo một câu đa nghĩa và sử dụng nhiều đại từ thay thế. Điều đó giúp văn phong có vẻ trôi chảy hơn nhưng vô hình chung khiến chatbot không thể hiểu được ý nghĩa thực sự của câu đó. Ví dụ như câu: “Bùi Tiến Phát là sinh viên khoa Công nghệ thông tin. Anh ấy đã học ở đó được 4 năm.”, Từ “anh ấy” và “đó” ở câu thứ 2 là 2 đại từ thay thế cho “Bùi Tiến Phát” và “khoa Công nghệ thông tin”. Trong quá trình truy xuất dữ liệu, tùy vào cách phân chia dữ liệu (sẽ được chúng em đề cập ở phần Phân chia dữ liệu), câu ví dụ có thể bị riêng ra thành 2 câu nhỏ riêng biệt là: “Bùi Tiến Phát là sinh viên khoa Công nghệ thông tin.” và “Anh ấy đã học ở đó được 4 năm”. Nếu ta cung cấp cho mô hình một “Knowledge Sources” chỉ chứa câu “Anh ấy đã học ở đó được 4 năm”, mô hình sẽ không thể đưa ra được câu trả lời do ngữ cảnh không còn được rõ ràng nữa. Để khắc phục vấn đề trên, phương pháp “Clause is all you need” đã được chúng em sáng tạo, phát triển và ứng dụng.

Clause ở đây có nghĩa là mệnh đề, trong ngữ pháp, mệnh đề là một đơn vị ngữ pháp chứa một chủ ngữ và một vị ngữ, có thể đứng một mình thành câu hoàn chỉnh hoặc kết hợp với các mệnh đề khác trong một câu phức. Clause là một khái niệm cơ bản trong ngữ pháp, đóng vai trò quan trọng trong việc cấu trúc câu và

truyền đạt ý nghĩa trong ngôn ngữ. Mệnh đề là một nhóm từ tạo thành một ý nghĩa hoàn chỉnh, có thể tồn tại một mình và diễn đạt một ý tưởng hoặc ý nghĩa. Mỗi mệnh đề thường bao gồm hai thành phần chính: chủ ngữ và động từ (vị ngữ). Chủ ngữ là người, vật hoặc vật mà mệnh đề đề cập đến, còn động từ (vị ngữ) mô tả hành động hoặc trạng thái của chủ ngữ. Ví dụ, trong mệnh đề “She reads books”, “She” là chủ ngữ và “reads” là động từ (vị ngữ).

Mệnh đề có thể được chia thành hai loại chính: mệnh đề đơn giản và mệnh đề phức tạp. Mệnh đề đơn giản là mệnh đề độc lập có thể tự tồn tại và tạo thành một câu hoàn chỉnh, trong khi mệnh đề phức tạp là một phần của câu lớn hơn và không thể tự tồn tại. Ví dụ về mệnh đề đơn giản là "Mặt trời mọc ở phía đông", trong khi mệnh đề phức tạp có thể là "Mặc dù trời mưa nhưng chúng em vẫn tập thể dục. Nó giúp bản thân hình thành sự kỉ luật."

Đặt vấn đề, khi ta cung cấp kiến thức cho một chatbot, dữ liệu thường là một đoạn văn, một biên bản hoặc một bảng biểu nào đó. Đa phần các thông tin là đưa vào sẽ là mệnh đề phức tạp, ví dụ ta có câu sau: “Ngày 22/6/2006, Thủ Tướng Chính phủ ban hành Quyết định số 146/2006/QĐ-TTg chuyển loại hình một số trường đại học và cao đẳng bán công, dân lập. Theo quyết định này, Trường Đại học Mở bán công Thành phố Hồ Chí Minh được chuyển sang trường đại học công lập và có tên gọi là Trường Đại học Mở Thành phố Hồ Chí Minh...” - Trích sổ tay sinh viên Đại học Mở TP.HCM, 2023, trang 3. Ở câu đầu tiên, “Ngày 22/6/2006, Thủ Tướng Chính phủ ban hành Quyết định số 146/2006/QĐ-TTg chuyển loại hình một số trường đại học và cao đẳng bán công, dân lập.”, dữ liệu cho ta biết hai thông tin trong một câu. Một là, ngày ban hành quyết định số 146/2006/QĐ-TTg. Hai là, trường được chuyển từ loại hình. Một câu có thể chứa nhiều thông tin, vậy nên khi tiến hành truy xuất đoạn thông tin cần thiết cho câu hỏi: “Trường được chuyển đổi loại hình khi nào?”, nó sẽ không thể đưa ra hiệu suất tìm kiếm đúng vào đó. Xét đến câu thứ hai của ví dụ trên, “Theo quyết định này, Trường Đại học Mở bán công Thành phố Hồ Chí Minh được chuyển sang trường đại học công lập và có tên gọi là Trường Đại học Mở Thành phố Hồ Chí Minh...”, câu này có thể cho ta biết các

thông tin như sau. Một là, Trường đại học Mở sẽ chuyển sang trường công lập. Hai là, một quyết định nào đó đã làm trường đại học Mở Thành phố Hồ Chí Minh chuyển thành trường công lập. Ta thấy, nếu hệ thống truy vết được câu trên và đưa cho Reader để trả lời câu hỏi “Quyết định nào chuyển trường đại học Mở Thành phố Hồ Chí Minh thành trường công lập?”, Reader chỉ có thể tìm thấy thông tin: “Quyết định này.”, đây là một câu trả lời vô nghĩa và ta sẽ không mong muốn. Tóm lại, ta có hai vấn đề làm việc truy xuất không hiệu quả như sau:

- Đa ý nghĩa: Một câu có thể dùng để trả lời nhiều câu hỏi khác nhau, mặc dù ngắn gọn về việc lưu trữ nhưng nó gây khó khăn cho hệ thống truy vết.
- Đại từ nối câu: Trong các mệnh đề phức tạp, câu sau thường dùng để bổ nghĩa cho câu trước, trong thế giới hiện thực, điều này tránh lặp từ và làm câu văn trôi chảy nhưng nó cản trở ý nghĩa trong hệ thống truy vấn.

Việc chuyển các câu trong bộ dữ liệu thu thập sang mệnh đề có hai lợi ích chính. Một là, dựa vào khái niệm của mệnh đề độc lập - là mệnh đề có thể đứng một mình thành câu hoàn chỉnh vì nó bày tỏ một ý đầy đủ. Ví dụ, ta có một câu như sau: “Bùi Tiến Phát sinh năm 2002 ở TP.HCM”. Về cơ bản, câu trên cung cấp cho chúng ta 2 thông tin chính. Đầu tiên, người mang tên “Bùi Tiến Phát” được sinh ra vào năm 2002. Cuối cùng, người tên “Bùi Tiến Phát” này được sinh ra tại TP.HCM. Mệnh đề độc lập làm cho câu trở nên đơn giản, từ đó khi ta tiến hành truy vết dữ liệu (sẽ được chúng em đề cập ở phần Truy vết dữ liệu) trở nên dễ dàng hơn. Hai là, mệnh đề giải quyết vấn đề câu không rõ ràng, câu “Anh ấy đã học ở đó được 4 năm” ở ví dụ trên sẽ được chuyển thành “Bùi Tiến Phát ấy đã học ở khoa Công nghệ thông tin được 4 năm.”. Từ đó, mô hình có thể hiểu rõ ngữ cảnh câu nói và trả lời chính xác hơn.

Chúng em sử dụng mô hình GPT-3.5, kỹ thuật prompt engineering và few-shot learning để chuyển bộ dữ liệu thu thập thành những câu mệnh đề. Quá trình này bắt đầu bằng việc cung cấp cho GPT-3.5 một đoạn văn bản cụ thể và yêu cầu mô hình tách và cải tạo đoạn văn đó thành các câu mệnh đề riêng biệt. Sau nhiều

thử nghiệm, chúng em đề xuất đoạn prompt sau:

Tách "Nội dung" thành các mệnh đề rõ ràng và đơn giản, đảm bảo chúng có thể hiểu được khi tách khỏi ngữ cảnh.

Bước 1: Phân tích cấu trúc câu

1. Chia nhỏ các câu ghép thành các câu đơn. Cố gắng giữ nguyên cách diễn đạt ban đầu của nội dung.
2. Xác định các thực thể có tên và thông tin mô tả đi kèm.
3. Loại bỏ các mệnh đề không có ý nghĩa giải thích.
4. Nhóm các mệnh đề liệt kê chi tiết nội dung của mệnh đề trước đó thành một mệnh đề duy nhất.

Bước 2: Phi ngữ cảnh hóa

1. Bổ sung các bộ ngữ cần thiết cho danh từ hoặc toàn bộ câu để đảm bảo tính rõ ràng và dễ hiểu.
2. Thay thế các đại từ (ví dụ: "nó", "anh ấy", "cô ấy", "họ", "cái này", "cái kia") bằng tên đầy đủ của các thực thể mà chúng đề cập.

Bước 3: Trình bày kết quả

1. Sắp xếp các mệnh đề đã được phân tích và phi ngữ cảnh hóa thành một danh sách các chuỗi.
2. Định dạng danh sách chuỗi này bằng JSON.

Đầu vào: Nội dung: Vòng thuần sắc và các hệ màu

RGB là hệ màu được tạo nên từ 3 màu cơ bản bao gồm: màu đỏ, xanh lục và xanh lam. Ba màu này sẽ được kết hợp theo các tỷ lệ khác nhau để tạo ra các màu sắc mà bạn nhìn thấy trên màn hình máy tính, tivi hoặc trên các thiết bị điện tử khác như

camera...

Output: ["RGB là hệ màu được tạo nên từ 3 màu cơ bản bao gồm: màu đỏ, RGB là hệ màu được tạo nên từ 3 màu cơ bản bao gồm: xanh lục, RGB là hệ màu được tạo nên từ 3 màu cơ bản bao gồm: xanh lam." , "Ba màu đỏ, xanh lục và xanh lam sẽ được kết hợp theo các tỷ lệ khác nhau để tạo ra các màu sắc mà bạn nhìn thấy trên màn hình máy tính, tivi hoặc trên các thiết bị điện tử khác như camera..."]

Đầu vào: Nội dung: {Nội dung người dùng muốn chuyển đổi}

Output:

Table 1: Prompt giúp chuyển các câu đa nghĩa thành một câu mệnh đề

Ví dụ, {Nội dung người dùng muốn chuyển đổi} = “Bùi Tiến Phát là sinh viên khoa Công nghệ thông tin. Anh ấy đã học ở đó được 4 năm.”. Ta sẽ thu được output như sau: ["Bùi Tiến Phát là sinh viên khoa Công nghệ thông tin.", "Bùi Tiến Phát đã học ở khoa Công nghệ thông tin được 4 năm."].

3.1.3.2. Remove acronyms

Trong quá trình xử lý và phân tích văn bản, một trong những kỹ thuật quan trọng là loại bỏ các từ viết tắt (acronyms) để tăng tính dễ đọc và hiểu của văn bản. Kỹ thuật "Remove Acronyms" được sử dụng để thay thế các từ viết tắt bằng đối tượng đầy đủ tương ứng, nhằm cải thiện khả năng hiểu và xử lý văn bản của các hệ thống xử lý ngôn ngữ tự nhiên. Kỹ thuật "Remove Acronyms" đóng vai trò quan trọng trong các ứng dụng xử lý ngôn ngữ tự nhiên như hệ thống trả lời câu hỏi, tóm tắt văn bản, dịch máy, và nhiều ứng dụng khác. Bằng cách loại bỏ các từ viết tắt, chúng ta giúp tăng tính dễ hiểu của văn bản, từ đó cải thiện hiệu suất và chất lượng của các hệ thống chatbot.

Có nhiều phương pháp khác nhau để loại bỏ các từ viết tắt trong xử lý ngôn ngữ tự nhiên. Ta có thể điểm qua các loại và ưu, nhược điểm như sau:

Phương	Khái niệm	Ưu điểm	Nhược điểm
--------	-----------	---------	------------

pháp			
Sử dụng Dictionary	Tạo một dictionary chứa các từ viết tắt và đối tượng đầy đủ tương ứng. Sử dụng hàm <code>replace()</code> hoặc <code>re.sub()</code> để thay thế các từ viết tắt trong văn bản bằng đối tượng đầy đủ từ dictionary.	Đơn giản và dễ triển khai. Hiệu quả với các từ viết tắt phổ biến. Dễ dàng thêm/xóa/sửa đổi các từ viết tắt trong dictionary.	Phụ thuộc vào việc xây dựng và duy trì dictionary. Khó khăn trong việc xử lý các từ viết tắt mới hoặc lạ. Không thể xử lý các trường hợp từ viết tắt có nhiều nghĩa khác nhau.
Học từ ngữ cảnh	Sử dụng các mô hình học máy để học nghĩa của từ viết tắt từ ngữ cảnh xung quanh. Áp dụng các kỹ thuật như Word Embeddings (Word2Vec, GloVe, etc.) để mô hình hóa ngữ nghĩa của từ. Dự đoán đối tượng đầy đủ của từ viết tắt dựa trên ngữ cảnh.	Có khả năng học và giải nghĩa các từ viết tắt mới hoặc lạ. Khai thác ngữ cảnh xung quanh để giải nghĩa từ viết tắt. Phù hợp với các lĩnh vực và miền dữ liệu khác nhau.	Phức tạp hơn về mặt triển khai và huấn luyện mô hình. Yêu cầu tập dữ liệu huấn luyện lớn và đa dạng. Hiệu suất phụ thuộc vào chất lượng mô hình và dữ liệu huấn luyện.

Sử dụng Ontologies và Kiến thức nền	Xây dựng các ontologies (cơ sở tri thức) chứa các từ viết tắt và đối tượng đầy đủ tương ứng. Kết hợp với các nguồn kiến thức nền (như Wikipedia, WordNet, etc.) để tăng cường khả năng nhận dạng và giải nghĩa các từ viết tắt.	Khai thác kiến thức nền và cơ sở tri thức để giải nghĩa các từ viết tắt. Có khả năng giải nghĩa các từ viết tắt trong nhiều lĩnh vực khác nhau. Tăng cường khả năng nhận dạng và giải nghĩa các từ viết tắt mới hoặc lạ.	Phức tạp trong việc xây dựng và duy trì ontologies và cơ sở tri thức. Yêu cầu nguồn lực và công sức lớn để tích hợp các nguồn kiến thức khác nhau. Hiệu quả phụ thuộc vào phạm vi và chất lượng của ontologies và kiến thức nền.
Phương pháp Heuristic	Sử dụng các quy tắc heuristic để xác định và giải quyết các từ viết tắt. Ví dụ: nếu một từ viết hoa và xuất hiện trong ngoặc đơn sau một cụm từ, thì có khả năng đó là từ viết tắt.	Đơn giản và dễ triển khai. Không yêu cầu tập dữ liệu huấn luyện lớn. Có thể kết hợp với các phương pháp khác để tăng cường hiệu quả.	Phụ thuộc vào việc xây dựng các quy tắc heuristic phù hợp. Khó khăn trong việc xử lý các trường hợp phức tạp hoặc ngoại lệ. Hiệu suất giới hạn trong các miền dữ liệu và lĩnh vực

			mới.
Mô hình Học sâu	Sử dụng các mô hình học sâu như BERT, GPT, vận dụng kỹ thuật Transfer Learning. Huấn luyện mô hình trên tập dữ liệu lớn chứa các từ viết tắt và đối tượng đầy đủ tương ứng. Sử dụng mô hình huấn luyện để dự đoán đối tượng đầy đủ của từ viết tắt trong ngữ cảnh mới.	Khả năng học và giải nghĩa các từ viết tắt mới hoặc lạ. Khai thác ngữ cảnh và đặc trưng ngôn ngữ để giải nghĩa từ viết tắt. Hiệu suất cao trong nhiều lĩnh vực và miền dữ liệu khác nhau.	Phức tạp về mặt triển khai và huấn luyện mô hình. Yêu cầu tập dữ liệu huấn luyện lớn và đa dạng. Tốn nhiều tài nguyên tính toán để huấn luyện và sử dụng mô hình.

Table 2: Bảng các phương pháp khả thi cho việc xử lý từ viết tắt

Miền tri thức (Knowledge Sources) đã được chuẩn bị sẵn và trong quá trình đó, bằng nghiệp vụ, chúng em đã xác định được các trường hợp viết tắt thông tin phổ biến. Vì vậy, chúng em đã tận dụng nó để tạo thành một từ điển biên dịch từ viết tắt và nghĩa gốc. Có ba lý do chính để chúng em lựa chọn phương pháp “Sử dụng dictionary” thay vì sử dụng các phương pháp khác đã nêu ở trên. Một là, về mặt logic các thông tin mà người dùng hỏi phải có trong miền tri thức (Knowledge Sources) thì chatbot mới có thể đưa ra đáp án trả lời. Kéo theo đó, tập hợp các từ viết tắt có trong miền tri thức chính là tập hợp đủ các trường hợp viết tắt cần thiết để chatbot có thể trả lời. Thứ hai, độ đơn giản và dễ triển khai của dictionary là một điểm cộng lớn so với các phương pháp khác như học sâu hay học ngữ cảnh - nơi yêu cầu một lượng lớn dữ liệu đào tạo và cũng không cần thiết để dự đoán các từ

viết tắt chưa được đào tạo. Ba là, việc so sánh qua dictionary sẽ tốn ít tài nguyên hơn so với các phương pháp phức tạp. Vì vậy, phương pháp Dictionary đã được chúng em lựa chọn và triển khai.

Quá trình loại bỏ các từ viết tắt bao gồm các bước sau:

1. Xác định các từ viết tắt: Đầu tiên, chúng ta cần xác định các từ viết tắt trong văn bản. Điều này được chúng em thực hiện bằng cách sử dụng các biểu thức chính quy (regular expressions) hoặc các kỹ thuật phân tích cú pháp khác.
2. So sánh từ viết tắt: Tiếp theo, chúng em tiến hành so sánh các từ ngữ viết tắt với các đối tượng tương ứng với nó. Ví dụ: {"CNTT": "Công nghệ thông tin", "KHTM": "Khoa học máy tính", "HTTQL": "Hệ thống thông tin quản lý"}.
3. Thay thế các từ viết tắt: Chúng em sử dụng các hàm hoặc thư viện xử lý chuỗi để thay thế các từ viết tắt trong văn bản bằng đối tượng đầy đủ tương ứng từ dictionary.

3.1.4. Phân chia dữ liệu

Trong quá trình triển khai mô hình Retrieval-Augmented Generation (RAG), việc chia dữ liệu nguồn tri thức thành các khối có kích thước khác nhau là một kỹ thuật quan trọng để tối ưu hóa hiệu suất và hiệu quả của ứng dụng. hệ thống. Việc chia dữ liệu thành các phần có kích thước khác nhau bao gồm việc chia tập dữ liệu nguồn kiến thức thành các phần văn bản riêng biệt có độ dài khác nhau thay vì sử dụng một kích thước tiêu chuẩn cho tất cả. Điều này cho phép hệ thống linh hoạt hơn trong việc xử lý và truy xuất thông tin từ các đoạn văn bản có độ dài khác nhau. Dựa vào quá trình tiền xử lý bằng “Clause is all you need”, các dữ liệu rời rạc và đa nghĩa đã được mệnh đề hóa. Chúng ta chỉ việc chia tập hợp các câu đó dựa trên một số lượng ký tự nhất định (các chunk). Chúng em đã tiến hành thử nghiệm trên các chunk-size khác nhau để đo mức độ truy vết dữ liệu đúng cho đề tài. Việc phân chia dữ liệu thành các chunk kích thước khác nhau đem lại một số lợi ích quan trọng sau:

- Tối ưu hóa bộ nhớ và hiệu suất: Việc chia nhỏ dữ liệu giúp hệ thống dễ dàng xử

lý và tải dữ liệu hiệu quả hơn. Điều này dẫn đến giảm tải cho bộ nhớ và tăng tốc độ xử lý, đặc biệt đối với các tập dữ liệu lớn. Đồng thời, sử dụng các chunk nhỏ hơn cũng giúp tối ưu hóa việc sử dụng bộ nhớ cache, giúp truy cập dữ liệu nhanh hơn và giảm thiểu thời gian chờ.

- Cải thiện độ chính xác: Với kích thước chunk phù hợp, mô hình học máy có thể dễ dàng hơn trong việc xác định và trích xuất thông tin liên quan để trả lời câu hỏi, từ đó nâng cao độ chính xác của kết quả. Phân chia dữ liệu thành các chunk nhỏ hơn cũng có thể giúp giảm nhiễu và cải thiện chất lượng dữ liệu đầu vào, dẫn đến kết quả tốt hơn cho các mô hình học máy.
- Tăng cường tính linh hoạt: Việc phân chia dữ liệu thành các chunk kích thước khác nhau cho phép hệ thống xử lý linh hoạt hơn với các đoạn văn bản có chiều dài khác nhau. Điều này giúp hệ thống có thể thích ứng với nhiều trường hợp sử dụng khác nhau và tránh được các vấn đề về tràn bộ nhớ hoặc hiệu suất thấp khi xử lý các đoạn văn bản dài.
- Tối ưu hóa truy xuất dữ liệu: Với các chunk nhỏ hơn, quá trình truy xuất và tìm kiếm thông tin có thể được thực hiện nhanh chóng và hiệu quả hơn. Điều này giúp cải thiện trải nghiệm người dùng và giảm thời gian chờ khi truy cập dữ liệu.

3.1.5. Lưu trữ dữ liệu

Kho lưu trữ vectơ (Vector store) hay còn gọi là lưu trữ vectơ là một kỹ thuật lưu trữ và truy xuất dữ liệu dựa trên các vectơ mật độ trong các hệ thống Retrieval-Augmented Generation (RAG) và các ứng dụng liên quan đến tìm thông tin và trả lời câu hỏi. Kho lưu trữ vectơ là cấu trúc dữ liệu cho phép lưu trữ và truy xuất hiệu quả các phân đoạn văn bản hoặc dữ liệu dưới dạng vectơ mật độ. Thay vì lưu trữ dữ liệu dưới dạng văn bản thông thường, các đoạn văn bản được biểu diễn bằng vectơ số thực trong không gian vectơ nhiều chiều, với mỗi chiều biểu thị một đặc điểm ngữ nghĩa của đoạn văn bản đó. Quá trình sử dụng vector store trong RAG bao gồm các bước sau:

Đầu tiên, chúng em sử dụng mô hình học sâu

“sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2” - là một mô hình được đào tạo trước để nhúng câu, được thiết kế đặc biệt để nhận dạng diễn giải bằng nhiều ngôn ngữ, để tiến hành word embeddings, quá trình này chuyển đổi các đoạn văn sau khi phân chia (chunking) thành các vector mật độ.

Sau đó, chúng em lưu trữ các vector biểu diễn văn bản cùng với thông tin ngữ cảnh (metadata) liên quan trong một cấu trúc dữ liệu phức tạp như cơ sở dữ liệu hoặc thư viện vector chuyên biệt (vector store library). Ở bước này, ChromaDB đã được chúng em lựa chọn. ChromaDB là một thư viện vector store nguồn mở phổ biến, được sử dụng rộng rãi trong các hệ thống Retrieval-Augmented Generation (RAG) và các ứng dụng liên quan đến tìm kiếm thông tin và trả lời câu hỏi. Với hiệu suất tìm kiếm nhanh, khả năng linh hoạt, tốc độ truy xuất cao, cũng như cộng đồng nguồn mở lớn, ChromaDB là một lựa chọn tốt để làm một vector store library.

Vector store mang lại nhiều lợi ích trong các hệ thống RAG, nó cho phép tìm kiếm và trích xuất các đoạn văn bản liên quan ngữ nghĩa với câu hỏi, vượt qua hạn chế của các phương pháp tìm kiếm từ khóa truyền thống. Ngoài ra, không gian lưu trữ nhỏ gọn hơn phương pháp lưu trữ văn bản thông thường, cũng như khả năng tìm kiếm nhanh, khiến Vectore store là lựa chọn tối ưu, mặc dù nó đòi hỏi một số bước xử lý đặc biệt.

Phân trên đã nói về cách lưu trữ dữ liệu cơ bản bằng cách nhúng các đoạn văn sau quá trình xử lý bằng phương pháp “Clause is all you need” thành các vector sau đó tiến hành lưu trữ vào một vector store. Nhưng trong quá trình thực nghiệm chúng em nhận ra rằng độ chính xác của truy vết dữ liệu có thể bị giảm thiểu tùy thuộc vào vector đại diện và câu hỏi truy vấn. Lý do ở đây phần lớn là do “ngữ cảnh”, một Clause cung cấp cho chúng ta một thông tin đầy đủ về một vấn đề cụ thể, nhưng có vô số cách hỏi khi chúng ta đề cập đến đến Clause đó. Ví dụ ta có mệnh đề: “Trường Đại học Mở Thành phố Hồ Chí Minh được thành lập vào ngày 15/6/1990.”, ta có thể đặt câu hỏi: “Trường được thành lập khi nào?”, “Trường ra đời vào thời điểm nào?”... Có vô số cách để tiếp cận về việc hỏi một vấn đề, vì vậy

với mỗi câu hỏi khác nhau có thể nhận được độ tương đồng khác nhau trên cùng một Clause. Điều tương tự cũng xảy ra với chính Clause, một clause cũng có thể được biểu diễn một cách khác nhau nhưng không làm thay đổi về mặt ngữ nghĩa. Lấy ví dụ từ câu: “Trường Đại học Mở Thành phố Hồ Chí Minh được thành lập vào ngày 15/6/1990.”, câu này có thể được biểu diễn khác đi nhưng không làm mất đi mục đích nói như: “Vào ngày 15/6/1990, Trường Đại học Mở Thành phố Hồ Chí Minh đã được thành lập.”, “Ngày 15/6/1990 là ngày Trường Đại học Mở Thành phố Hồ Chí Minh được thành lập.”, bằng cách đảo ngữ hoặc thay đổi động từ biểu diễn, ta đã nhận được một câu mệnh đề khác biệt với câu gốc mà vẫn giữ nguyên ý nghĩa. Trong quá trình thực nghiệm, chúng em nhận ra rằng, với mỗi cặp câu hỏi - mệnh đề khác nhau được sinh ra từ sự tương đồng của câu gốc, chúng sẽ có độ tương đồng cosine khác nhau và sẽ luôn có một cặp mệnh đề - câu hỏi có độ tương đồng cao hơn hẳn so với những câu khác (chúng em xin được đặt tên là retrieve couple). Ví dụ như cặp câu hỏi - mệnh đề: “Trường Đại học Mở Thành phố Hồ Chí Minh ra đời vào ngày 15/6/1990.” - “Trường đại học Mở được ra đời vào ngày nào?” sẽ có độ tương đồng khác biệt so với cặp câu hỏi - mệnh đề: “Trường Đại học Mở Thành phố Hồ Chí Minh ra đời vào ngày 15/6/1990.” - “Trường đại học mở được thành lập vào năm nào?”. Chúng em đặt giả thuyết rằng, có vô hạn cách hỏi cho một câu và vô hạn cách trình bày cho một đoạn dữ liệu để hỏi, vì vậy chúng em áp dụng “Giả thuyết trung bình” cho vấn đề tăng cường ngữ nghĩa cho câu.

Law of Large Numbers (Định lý Giả thiết Trung bình) là một trong những nguyên lý quan trọng nhất trong xác suất và thống kê. Định lý này nói rằng khi số lượng mẫu trong một mẫu ngẫu nhiên tăng lên, thì trung bình của các mẫu sẽ tiến dần về giá trị kỳ vọng của biến ngẫu nhiên đó. Cụ thể, nếu $X_1, X_2, \dots, X_n$ là các biến ngẫu nhiên độc lập và có cùng phân phối xác suất, với giá trị kỳ vọng $E(X_i) = \mu$, thì với một số n đủ lớn, trung bình của các biến ngẫu nhiên sẽ gần bằng với giá trị kỳ vọng của chúng:

$$\lim_{n \rightarrow \infty} \frac{X_1 + X_2 + \dots + X_n}{n} = \mu$$

Giả sử bạn có một đồng xu không đều, trong đó xác suất để mặt sấp (mặt 0) xuất hiện là $p = 0.6$ và xác suất để mặt ngửa (mặt 1) xuất hiện là $1 - p = 0.4$. Bạn muốn kiểm tra xem khi ném đồng xu này nhiều lần, trung bình của các lần ném có tiến dần về xác suất trung bình của mặt sấp là p hay không. Bạn quyết định ném đồng xu 10 lần và tính trung bình của các kết quả. Kết quả có thể là như sau: [Mặt sấp (mặt 0), Mặt ngửa (mặt 1), Mặt sấp, Mặt sấp, Mặt ngửa, Mặt ngửa, Mặt ngửa, Mặt sấp, Mặt sấp, Mặt ngửa]. Xác suất của mặt sấp là $p = 0.5$. Tuy nhiên, nếu bạn tiếp tục ném đồng xu 100 lần, 1000 lần, hoặc thậm chí 10000 lần, bạn sẽ nhận thấy rằng trung bình của các lần ném dần dần tiến dần về $p = 0.6$. Dựa vào nguyên mẫu của định luật Giả thiết Trung bình, chúng em đưa ra giả thuyết: “Tồn tại một vector đại diện được chuyển hóa từ một mệnh đề gốc thỏa mãn trả lời toàn bộ câu hỏi có thể hỏi từ mệnh đề đó”. Phương pháp của chúng em sáng tạo ra tên là “Information Augmentation”, sẽ lấy trung bình các vector đại diện của Clause, bằng cách tạo ra các phiên bản tương đồng phù hợp, sau đó biểu diễn các câu đó thành các vector ngữ nghĩa, cuối cùng, chúng em lấy trung bình cộng của các vector làm vector đại diện để truy vết.

Mô tả phương pháp “Information Augmentation” như sau:

1. Chuyển đổi các mệnh đề gốc (Clause) sau quá trình “Clause is all you need” thành các vector ngữ nghĩa bằng cách sử dụng các mô hình ngôn ngữ tiên tiến như BERT, PhoBERT để encode mệnh đề gốc và các câu hỏi thành các vector ngữ nghĩa. Vector của mệnh đề gốc được ký hiệu là V_g .
2. Tạo ra các phiên bản tương đồng của mệnh đề gốc khác nhau nhưng vẫn giữ nguyên ý nghĩa. Các phiên bản này được encode thành các vector ngữ nghĩa $V_{g1}, V_{g2}, \dots, V_{gm}$.
3. Lấy trung bình các vector ngữ nghĩa của các mệnh đề tương đồng để tạo ra một vector đại diện duy nhất cho mệnh đề gốc tương ứng V_{avg} .

$$V_{avg} = \frac{1}{m} \sum_{i=1}^m V_{gi}$$

Với số lượng mệnh đề và câu hỏi đủ lớn, vector đại diện V_{avg} sẽ tiến dần đến giá trị kỳ vọng và thỏa mãn giả thuyết chúng em đưa ra, từ đó đảm bảo độ chính xác và tính tổng quát của việc truy vấn thông tin.

3.1.6. Truy vết dữ liệu

Trong mô hình Retrieval-Augmented Generation (RAG), truy vết dữ liệu (data tracing) là một quy trình quan trọng nhằm đảm bảo tính minh bạch, trách nhiệm giải trình và độ tin cậy của hệ thống. Nó gắn liền với việc theo dõi và ghi lại nguồn gốc của thông tin được sử dụng để đi đến câu trả lời cuối cùng. Truy vết dữ liệu trong RAG bao gồm việc ghi lại các đoạn văn bản liên quan được trích xuất từ nguồn tri thức, cũng như bất kỳ thông tin bổ sung nào được sử dụng để tạo ra câu trả lời. Quá trình này giúp xác định rõ ràng rằng câu trả lời được sinh ra dựa trên nguồn thông tin nào và có thể truy ngược lại các bằng chứng hỗ trợ.

Để truy vết dữ liệu trong RAG, phương pháp Vector store-backed retriever được sử dụng một cách phổ biến. Phương pháp này hiểu đơn giản là ta thực hiện so khớp các vector được lưu trữ trong vector store với câu hỏi (query) của người dùng nhập vào. So khớp các vector ở đây có thể sử dụng một số phương pháp như Khoảng cách Euclid (Euclidean Distance), tích vô hướng (Dot Product) hay độ tương tự cosine (Cosine Similarity - Phương pháp chúng em sử dụng). Về mặt lý thuyết thông thường, Vector store-backed retriever là đủ để ta tiến hành lấy ra được lượng thông tin cần thiết để trả lời câu hỏi của người dùng. Tuy nhiên, lượng thông tin này thường sẽ bị thiếu và có độ nhiễu cao. Có nhiều nguyên nhân gây ra vấn đề nhiễu thông tin này, có thể kể đến dưới đây.

Một câu chứa quá nhiều thông tin và cách biểu diễn phức tạp, dẫn đến việc câu này có khả năng cao được trả về bất kể câu hỏi của người dùng có là gì đi nữa. Vấn đề này đã được chúng em giải quyết triệt để bằng phương pháp “Clause is all

you need” đã trình ở trên.

Tôi xin được trích dẫn một câu nói mà hầu như mọi người đều đã nghe qua, đó là “Phong ba bão táp, không bằng ngữ pháp Việt Nam.”. Đúng vậy, ở Tiếng Việt, một từ có thể mang nhiều nghĩa, ví dụ như từ “ấy” chẳng hạn. Tùy thuộc vào tình huống cụ thể, đại từ “ấy” đóng vai trò là đại từ thay thế cho sự vật được đề cập trước đó. Trong câu “Nhìn vào bức tranh kia kìa. Tôi thích phong cách vẽ ấy.”, “ấy” trong ví dụ vừa nêu là đại từ thay thế cho “bức tranh”, cách nói từ “ấy” giúp cho câu văn của chúng ta trở nên linh hoạt, thân thiện giữa người với người nhưng không may, nó không thân thiện cho mô hình truy vết. Tiếng Việt là một ngôn ngữ đặc biệt, không chỉ một từ có thể có nhiều nghĩa, mà một nghĩa còn có thể có nhiều từ để nói về nó. Một ví dụ đơn giản cho chúng ta hình dung, vật dụng có hình dáng là một đĩa tròn hoặc dạng bán cầu bị chia đôi tiết diện, được sử dụng để đựng thức ăn, gọi là gì?. Ở miền Nam, nó được gọi là cái “chén”, người Miền Bắc thì gọi là cái “bát”, trong khi miền Trung nó là cái “đọi”. Sự đa dạng về ngôn ngữ đó là một thách thức lớn trong quá trình truy vết, chủ đề ta muốn hỏi có thể có dữ liệu nhưng mô hình không thể đưa ra được câu trả lời do cách trình bày dữ liệu khác nhau. Để giải quyết vấn đề trên, chúng em đề xuất một kỹ thuật mới gọi là “Information Augmentation” hay “IA” và “MultiQueryRetriever”.

Yêu cầu của người dùng đối với chatbot còn cao hơn một RAG thông thường, RAG thường sẽ nhận vào một câu hỏi rõ ràng từ người dùng, trong khi đó chatbot yêu cầu trả lời và ghi nhớ lịch sử cuộc trò chuyện như cuộc đối thoại bình thường giữa hai con người. Vấn đề ở đây là: Trong những cuộc trò chuyện thông thường, người ta hay sử dụng cái đại từ thay thế như “ấy”, “đó”, “nó”, “anh ấy”, “ông kia”, v.v., mục đích là để cuộc trò chuyện tự nhiên và đỡ mất thời gian nói hơn. Thói quen này được chuyển từ những cuộc trò chuyện trực tiếp giữa người với người sang nhắn tin giữa người với người và giờ là sang người và các bot chat. Não người chứa khoảng 100 tỉ tế bào thần kinh (nơ-ron), các nơ-ron này đóng vai trò tạo ra nhận thức và tư duy của con người, vì vậy, việc liên kết các sự kiện đã nói trước đó với các đại từ thay thế để biết các đại từ thay thế đang chỉ đến việc gì đối với chúng ta

là tương đối dễ dàng. Thế nhưng, đây là điều khó khăn với chatbot. Với số lượng bộ nhớ và khả năng xử lý nhỏ, ta không thể nào nhắc lại toàn bộ lịch sử chat cho chatbot được, mà dù có thể đi chăng nữa, chatbot sẽ mất rất nhiều thời gian để xử lý hết toàn bộ lịch sử chat đó để nhận ra đại từ thay thế của câu hỏi người dùng đang nói về điều gì. Giả sử ta có cuộc trò chuyện giữa người và máy như sau:

Người	Máy
Bùi Tiến Phát là ai?	Bùi Tiến Phát là sinh viên đại học Mở TP.HCM
Vậy anh ấy sinh năm bao nhiêu?	Tôi không có câu trả lời.

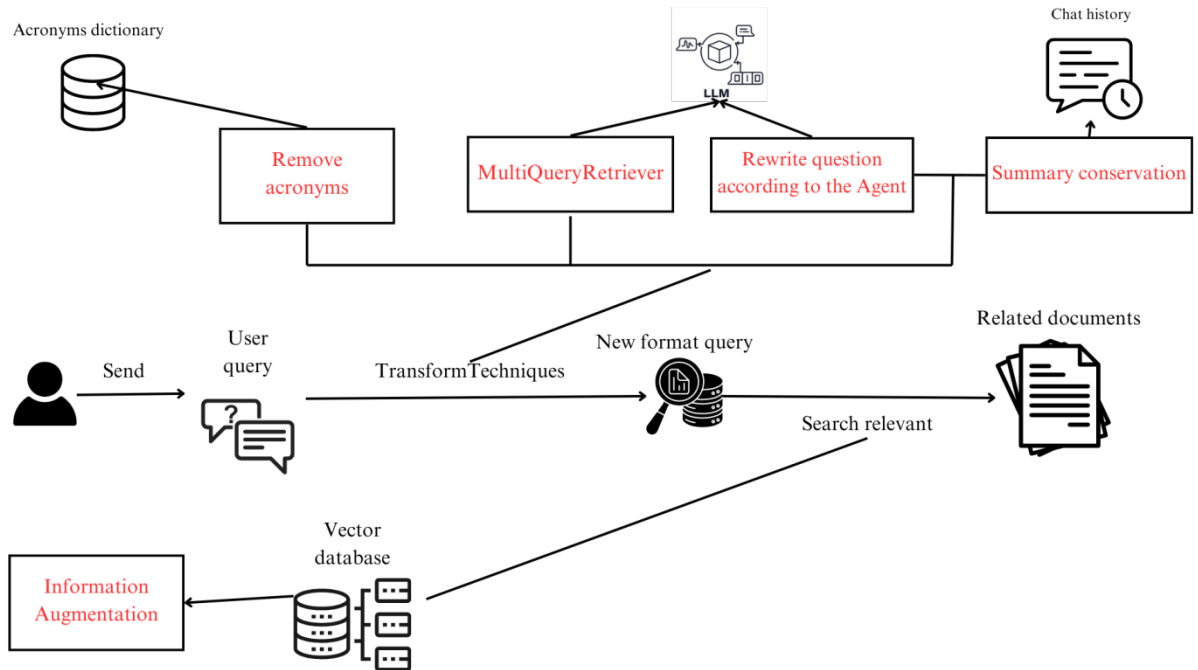
Table 3: Bảng ví dụ cho cuộc trò chuyện giữa người và máy khi dùng đại từ quan hệ

Ở câu hỏi “Vậy anh ấy sinh năm bao nhiêu?”, RAG đã sử dụng phương pháp Vector store-backed retriever để tìm kiếm trong Vector store ChromaDB, mặc dù ChromaDB chứa vector đại diện cho câu “Bùi Tiến Phát sinh năm 2002.”, nhưng chatbot không thể đưa ra câu trả lời vì nó chỉ tìm kiếm thực thể tên là “anh ấy”. Chúng ta có thể giải quyết vấn đề này cơ bản bằng cách cho toàn bộ lịch sử những cuộc trò chuyện trước đó vào một mô hình ngôn ngữ lớn (LLM) để mô hình có thể đọc toàn bộ quá trình trò chuyện cũ và hiểu được đại từ thay thế. Nhưng phương pháp đó sẽ tốn rất nhiều tài nguyên và hiệu suất thấp. Để giải quyết được vấn đề trên, chúng em đã ứng dụng phương pháp tóm tắt cuộc trò chuyện (summary) và từ nội dung tóm đó tìm ra thực thể và đại từ quan hệ đại diện. Bản tóm tắt là phần tóm gọn của những cuộc trò chuyện trước đó, vậy nên phương pháp này cải thiện hiệu suất hơn nhiều so với việc đưa toàn bộ lịch sử trò chuyện cũ.

Vấn đề cuối mà chúng em nhắc đến đó là viết tắt, chúng ta có thói quen ghi chép tắt một số từ với mục đích làm ngắn gọn câu nói, ví dụ như “khoa CNTT”, “CLB LT trên TBĐĐ”, “môn CTDL>”, v.v. Như đã phân tích trước đó, viết tắt là một dạng khác của đại từ thay thế, mặc dù tiện cho giao tiếp giữa người với người nhưng là thách thức lớn cho máy móc. Vì vậy, chúng em đã tạo ra một từ điển để biên dịch các từ viết tắt để cải thiện hiệu suất chatbot. Sau khi giải quyết hết những vấn đề về câu hỏi người dùng, chúng ta tiến hành so sánh câu hỏi sau biến đổi với vector database chúng ta đã chuẩn bị.

Phần trên đã nêu ra một số vấn đề gặp phải trong quá trình truy vấn khiến mô hình khó khăn trong việc tìm kiếm câu trả lời. Sau đây, chúng em xin phép được

tóm tắt lại sơ đồ xử lý truy vết của chatbot khi nhận được câu hỏi của người dùng như sau:



Hình 13: Quy trình biến đổi và truy vết câu hỏi người dùng

Quy trình truy vết bắt đầu khi nhận câu hỏi từ người dùng, sau đó hệ thống sẽ chuyển đổi câu hỏi đó trở nên tối ưu để truy vấn. Đầu tiên, “Remove acronyms” sẽ chuyển đổi các từ viết tắt trở về dạng đầy đủ của nó thông qua cơ chế dò từ điển. Sau đó, hệ thống dựa vào “Summary conservation” để tóm tắt lại lịch sử trò chuyện. Từ lịch sử cuộc trò chuyện đó, mô hình ngôn ngữ lớn sẽ kiểm tra và thay đổi các đại từ thay thế trở về từ nguyên mẫu. Cuối cùng, dựa vào câu truy vấn đã được tối ưu bằng những bước trước đó, MultiQueryRetriever sẽ đảm nhận việc tạo ra thêm những câu truy vấn tương đồng với câu hỏi của người dùng, mục đích là tăng khả năng tìm ra câu trả lời cho vấn đề của người hỏi. Cuối cùng, hệ thống tiến hành truy vết các câu truy vấn đó với trong vector database đã được tăng cường dữ liệu bằng phương pháp “Information Augmentation” và trả về các dữ liệu liên quan.

3.1.6.1. Rewrite question according the Agent và Summary Conversation

Đại từ thay thế là một hiện tượng ngôn ngữ phổ biến trong đó một từ hoặc cụm từ được sử dụng để thay thế một biểu thức ngôn ngữ khác đã đề cập trước đó. Trong bối cảnh một hệ thống RAG, việc thay thế đại từ có thể ảnh hưởng đáng kể đến hiệu suất hệ thống.

Một trong những thách thức chính là khả năng xác định chính xác mối tương quan giữa đại từ biến tố và biểu thức ngôn ngữ mà nó đại diện. Nếu hệ thống không thể giải quyết vấn đề này có thể dẫn đến hiểu lầm và trả lời sai lệch. Ví dụ: trong câu hỏi “Bùi Tiến Phát đã làm gì để giải quyết vấn đề đó?”, đại từ “đó” đề cập đến một vấn đề cụ thể đã đề cập trước đó. Nếu hệ thống không thể xác định chính xác chính sách nào đang được đề cập đến thì nó sẽ không thể trả lời chính xác câu hỏi. Hơn nữa, trong nhiều trường hợp, các đại từ thay thế có thể đề cập đến thông tin phức tạp hoặc dài dòng đã đề cập trước đó, làm tăng khó khăn cho hệ thống trong việc xử lý và phản hồi chính xác. Điều này đòi hỏi hệ thống phải có khả năng hiểu ngữ cảnh và liên kết thông tin một cách hiệu quả. Vì những lý do trên, chúng em đề xuất phương pháp kết hợp bao gồm việc tóm tắt hội thoại trước đó (Summary Conversation) với tái tạo lại câu hỏi của người dùng (Rewrite question according the Agent).

Trong quá trình hỏi đáp với chatbot, các vấn đề (problem) và các thực thể (Agent) được nhắc đến liên tục, vấn đề cần giải quyết ở đây là: Làm sao để mô hình có thể hiểu nhớ được các thực thể cũ và hiểu được bối cảnh và đại từ nhân xưng thay thế mới để có thể đưa ra được câu trả lời chính xác. Hiện tại có hai cách phổ biến để giải quyết vấn đề hiểu ngữ cảnh cuộc trò chuyện cho chatbot đó là “Cung cấp toàn bộ lịch sử cuộc trò chuyện” và “Tóm tắt lịch sử trò chuyện”. Việc cung cấp toàn bộ hoặc một phần cuộc trò chuyện gần nhất giúp đảm bảo mô hình có đủ ngữ cảnh và thông tin để hiểu câu hỏi và nội dung cuộc trò chuyện. Tuy nhiên, khối lượng dữ liệu lớn có thể gây khó khăn cho mô hình trong việc xử lý và tập trung vào thông tin quan trọng, dẫn đến việc mô hình bị phân tâm bởi thông tin không

liên quan đến câu hỏi. Hiệu suất mô hình còn bị ảnh hưởng nặng nề do lịch sử đàm thoại quá dài, cũng như ta không biết một ngữ cảnh mới được bắt đầu từ đâu để tối ưu. Ngoài phương pháp “Cung cấp toàn bộ lịch sử cuộc trò chuyện”, thì việc tóm tắt cuộc trò chuyện sẽ giúp mô hình giảm khối lượng dữ liệu đầu vào, cải thiện hiệu suất xử lý, tập trung vào bối cảnh và ngữ cảnh chính, tránh sự phân tâm. Ưu điểm lớn nhất của việc tóm tắt cuộc trò chuyện là việc giảm khối lượng từ ngữ đưa vào, nhưng vô hình chung lại là điểm yếu của nó. Câu hỏi ở đây là: “Mô hình phải chọn tóm tắt cuộc hội thoại bắt đầu từ đâu?”. Ta xét đoạn hội thoại sau:

STT	Người dùng	Chatbot
1	Ai là Hiệu trưởng của trường Đại học Mở TP.HCM?	Hiệu trưởng của trường Đại học Mở Thành phố Hồ Chí Minh là GS.TS. Nguyễn Minh Hà.
2	Trường Đại học Mở TP.HCM được thành lập vào năm nào?	Trường Đại học Mở Thành phố Hồ Chí Minh được thành lập vào ngày 15/6/1990.
3	Thầy Dương Hữu Thành là ai trong khoa Công nghệ thông tin?	Thầy Dương Hữu Thành là Phó Trưởng Bộ môn Cơ sở ngành trong Khoa Công nghệ thông tin (CNTT) của Trường Đại học Mở Thành phố Hồ Chí Minh.
4	Thầy ấy có học vị là gì?	Học vị của Thầy Dương Hữu Thành là thạc sĩ.

Table 4: Bảng ví dụ về đoạn hội thoại dài có dùng đại từ thay thế

Đoạn hội thoại trên, ta hãy giả sử rằng chatbot đã có thể trả lời phù hợp. Giả sử ta chọn số 3 cuộc hội thoại gần nhất để tóm tắt ngữ cảnh trả lời cho tôi hỏi cuối cùng. Vậy mô hình sẽ có thể tóm tắt thành như sau: “Người dùng hỏi về thông tin liên quan đến trường Đại học Mở TP.HCM, bao gồm tên của Hiệu trưởng hiện tại, GS.TS. Nguyễn Minh Hà, năm thành lập của trường là 1990, và vai trò của Thầy

Dương Hữu Thành, là Phó Trưởng Bộ môn Cơ sở ngành trong Khoa Công nghệ thông tin.”. Ta thấy có quá quá nhiều thông tin được tóm tắt và tổng hợp lại, bao gồm một câu hỏi về hiệu trưởng, một câu hỏi về thời gian thành lập trường và một câu hỏi về một người cụ thể mang tên Dương Hữu Thành. Khi ta cho mô hình thông qua đoạn tóm tắt trên, có nguy cơ cao mô hình sẽ hiểu rằng câu hỏi “Thầy ấy có học vị là gì?” là hỏi về hiệu trưởng và câu trả lời sẽ là “Học vị của Thầy Nguyễn Minh Hà là Giáo sư Tiến Sĩ.” thay vì “Học vị của Thầy Dương Hữu Thành là thạc sĩ.”. Vậy nếu chúng ta thử tóm tắt 2 hay 1 đoạn hội thoại gần nhất thì sao? Trong quá trình thực nghiệm, chúng em nhận thấy nó vẫn có thể xảy ra như trường hợp trên và ta phải có một phương pháp cụ thể nào đó để quyết định xem số lượng đoạn hội thoại tóm tắt để mô hình có thể hiểu được ngữ cảnh những câu trước đó là bao nhiêu.

Câu trả lời của chúng em là 1. Nhưng không đơn giản là chúng em đưa đoạn hội thoại cuối cùng giữa người và botchat để tóm tắt. Thay vào đó chúng em có một cơ chế biến đổi câu hỏi theo ngữ cảnh trước đó và nếu ngữ cảnh đã chuyển sang một chủ đề khác phần tóm tắt của chúng em sẽ thay đổi. Chatbot của chúng em sẽ lưu trữ lịch sử cuộc trò chuyện của người dùng, song song với đó nó còn lưu trữ lịch sử “ngữ cảnh” đã được xử lý. Từ đó, mô hình luôn biết được các đại từ thay thế đang đại diện cho thực thể nào.

Chúng em sẽ mô tả quy trình dựa trên ví dụ trước đó (Bảng 4). Ở ví dụ hội thoại (Bảng 4), ta thấy chatbot đã nắm bắt rất tốt được thực thể và trả lời được những câu hỏi chính xác (ta giả sử các quy trình như dữ liệu đã được làm tốt). Để làm được như vậy, chúng em đã sử dụng mô hình ngôn ngữ lớn GPT 3.5 để tiến hành tóm tắt lại đoạn hội thoại cũ gần nhất. Chúng em đã nghiên cứu, và đề xuất đoạn prompt sau:

You are a professional summarizer in summarizing the conversation between the user and the AI. Your task is to summarize the content of the conversation from "History", the goal is for the language model to understand the context and hidden

entities of the question so that it can give the most accurate answer.

1. Read the context carefully and memorize concepts, terms or ideas.
2. Check which entity the conversation is talking about, and what issues are related to the entity.
3. Bring up the last mentioned entity in the conversation, so the AI can continue the conversation with the user.
4. Present the results in Vietnamese.

Table 5: Prompt giúp tóm tắt hội thoại

Đoạn prompt giúp ta tạo ra được đoạn tóm tắt dựa trên các cuộc trò chuyện trước. Ví dụ, khi sử dụng prompt trên với đoạn hội thoại:

Người dùng	Chatbot
Ai là Hiệu trưởng của trường Đại học Mở TP.HCM?	Hiệu trưởng của trường Đại học Mở Thành phố Hồ Chí Minh là GS.TS. Nguyễn Minh Hà.

Table 6: Bảng hội thoại ví dụ để thử nghiệm “Prompt giúp tóm tắt hội thoại”

Mô hình sẽ trả về tóm tắt ngữ cảnh và thực thể cũ như sau: “Cuộc trò chuyện trước đó nói về GS.TS. Nguyễn Minh Hà là hiệu trưởng của trường Đại học Mở TP.HCM.”. Sau khi có được thông tin về các thực thể và các tóm tắt nội dung, chúng em tiến hành tạo lại câu hỏi mới. Việc tạo ra câu hỏi mới nhằm hai mục đích, một là nó sẽ thay đổi phong cách hỏi từ đó giúp tăng khả năng tìm ra đáp án (phương pháp MultiQueryRetriever), hai là nó sẽ giúp ta thay thế những đại từ thay thế bằng thực thể thực sự của nó. Để tạo ra câu hỏi mới, chúng em đã sáng tạo và đề xuất đoạn prompt sau:

You are an effective assistant specializing in answering questions related to academics and admissions at Ho Chi Minh City Open University. Answer all questions to the best of your ability. Based on "Summary" and "Questions" below.

1. Read "Summary" and "Question" carefully, check whether the question is related

to the previous summary paragraph by examining the entities.

2. Decontextualize the "Question" by adding necessary modifiers to nouns or entire sentences and replacing pronouns (e.g. "it", "he", "she", "they" , "this", "that") by the full names of the entities they refer to.

3. Format the question so that the tracking system can find the answer from the question. Don't try to ask the opposite question.

4. Present results in Vietnamese. Format into a string that includes only the recreated question.

Table 7: Prompt giúp tạo lại câu hỏi mới từ lịch sử cuộc hội thoại trước đó
Ví dụ khi sử dụng đoạn prompt trên ta có output như sau:

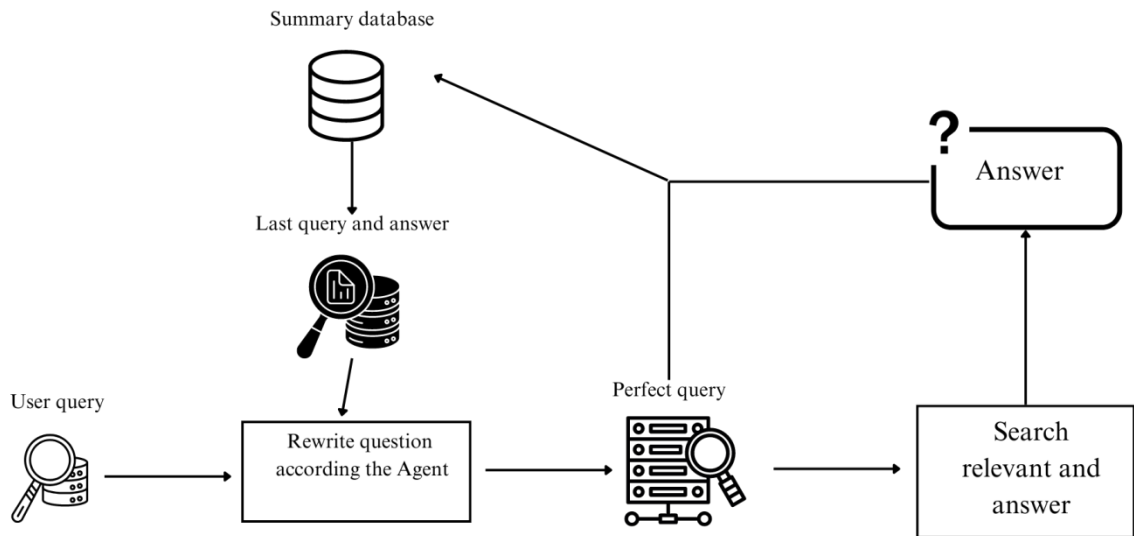
Summary: Cuộc trò chuyện trước đó nói về GS.TS. Nguyễn Minh Hà là hiệu trưởng của trường Đại học Mở TP.HCM.

Question: Ông ấy có trách nhiệm gì

Output: Hiệu trưởng của trường Đại học Mở TP.HCM có trách nhiệm gì?

Table 8: Ví dụ và kết quả khi sử dụng “Prompt giúp tạo lại câu hỏi mới từ lịch sử cuộc hội thoại trước đó”

Hệ thống nhận vào tóm tắt từ quá trình trước, sau đó kiểm tra xem câu hỏi của người dùng có chứa các đại từ thay thế không. Ở ví dụ trên, đại từ thay thế là “Ông ấy”, thực thể bị thay thế là “Hiệu trưởng Nguyễn Minh Hà”. Sau hệ thống nắm bắt được thực thể, hệ thống sẽ tiến hành viết lại câu hỏi, thay thế thực thể và hỏi theo một phong cách khác. Mục đích là chuẩn hóa và tăng khả năng tìm ra đáp án cần thiết. Câu hỏi sau khi được viết lại bởi hệ thống sẽ được lưu lại cho quá trình tóm tắt sau đó. Để tóm tắt lại, chúng ta có quy trình như sau:



Hình 14: Quy trình sử dụng phương pháp Rewrite question according to the Agent và Summary Conversation

Đầu tiên, phương pháp “Rewrite question according to the Agent” được sử dụng, dựa vào câu hỏi của người dùng và lịch sử tóm tắt nội dung trò chuyện gần nhất, LLM tạo ra câu hỏi mới phù hợp với ngữ cảnh, loại bỏ các đại từ thay thế dựa trên lịch sử cho trước. Từ câu hỏi đó, hệ thống tiến hành truy vết các tài liệu liên quan bằng phương pháp Vector store-backed retriever, kết quả trả ra sau khi được diễn giải qua quá trình “Tổng hợp dữ liệu” (sẽ được nói đến ở phần dưới) sẽ được gửi đến người dùng, đồng thời sẽ được lưu trữ lại để thực hiện một quy trình mới (nếu có).

3.1.6.2. Vector retriever

Phương pháp truyền thống Vector Store-backed Retriever đóng vai trò quan trọng trong hệ thống RAG. Phương pháp này sử dụng kỹ thuật biểu diễn văn bản dưới dạng vector và lưu trữ chúng trong một vector store. Khi có câu hỏi đầu vào, hệ thống sẽ biểu diễn câu hỏi thành một vector và truy vấn vector store để tìm kiếm các đoạn văn bản có vector gần nhất với vector của câu hỏi. Việc sử dụng vector store giúp tăng tốc quá trình tìm kiếm và truy xuất thông tin liên quan, đặc biệt

trong trường hợp có lượng dữ liệu lớn. Thay vì phải quét qua toàn bộ tập dữ liệu văn bản, hệ thống chỉ cần tìm kiếm trong vector store, điều này làm giảm đáng kể thời gian xử lý và tăng hiệu suất.

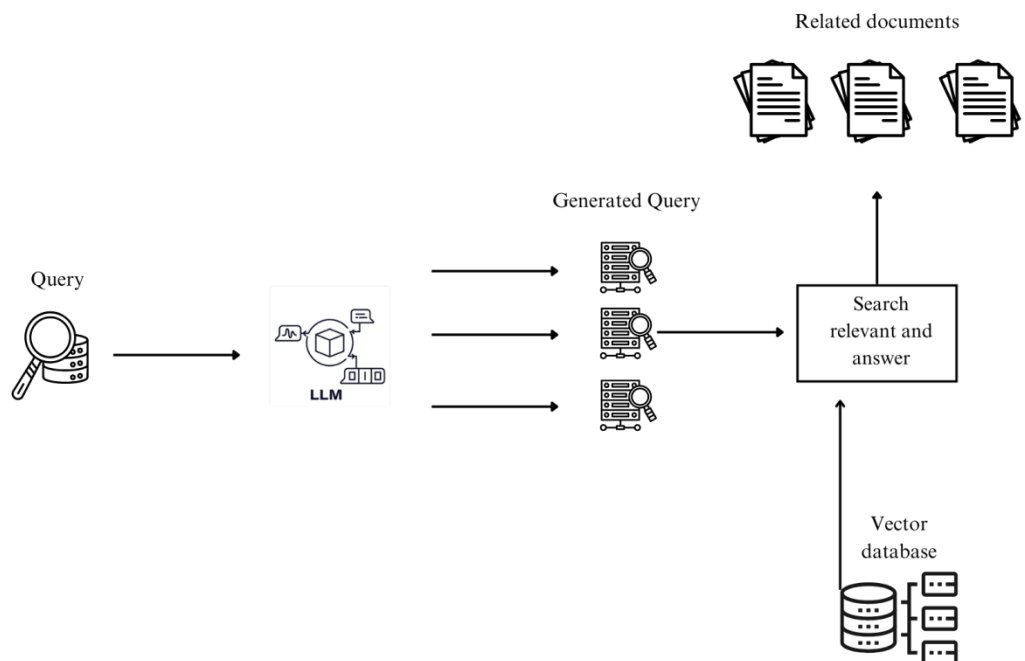
Tuy nhiên, phương pháp Vector Store-backed Retriever cũng có một số hạn chế. Đầu tiên, hiệu quả của phương pháp này phụ thuộc rất nhiều vào chất lượng của việc biểu diễn vector. Nếu vector không thể bao gồm đầy đủ ngữ nghĩa của câu hỏi hoặc đoạn văn bản, kết quả truy xuất có thể không chính xác. Thứ hai, phương pháp này có thể gặp khó khăn trong việc xử lý các câu hỏi phức tạp hoặc đa dạng, khi mà các đoạn văn bản liên quan có thể không được biểu diễn bằng các vector gần nhau trong vector store. Thứ ba, khả năng lấy lại ngữ cảnh liên quan có thể bị giới hạn bởi kích thước của vector store. Nếu vector store quá nhỏ, nó có thể không chứa đủ thông tin để trả lời câu hỏi một cách đầy đủ. Mặc dù chúng em đã cải thiện nó bằng phương pháp “Clause is all you need” và “Information Augmentation” nhưng chúng em vẫn tiếp tục cải tiến hiệu quả truy xuất bằng cách kết hợp những phương pháp khác.

Để giải quyết vấn đề không tìm thấy dữ liệu, ngoài vấn đề cải thiện nguồn dữ liệu gốc và cải thiện vector đại diện trong vector st (Knowledge Sources) bằng “Clause is all you need” và “Information Augmentation”. Chúng em đề xuất giải pháp các kỹ thuật là MultiQueryRetriever và Parent Document Retriever. MultiQueryRetriever tạo ra nhiều truy vấn khác nhau dựa trên câu hỏi ban đầu, giúp cải thiện khả năng thu thập ngữ cảnh liên quan. Trong khi đó, Parent Document Retriever kết hợp việc truy xuất từ vector store và tập hợp tài liệu gốc, đảm bảo không bỏ sót bất kỳ ngữ cảnh quan trọng nào.

Phương pháp MultiQueryRetriever là một kỹ thuật cải tiến quan trọng để nâng cao hiệu suất truy xuất thông tin trong các hệ thống Hỏi đáp Tăng cường Truy xuất và Sinh (RAG). Phương pháp này mở rộng khả năng của Vector Store-backed Retriever truyền thống bằng cách tạo ra nhiều truy vấn khác nhau dựa trên câu hỏi ban đầu. Trong Vector Store-backed Retriever, chỉ có một truy vấn duy nhất được

tạo ra từ câu hỏi và sử dụng để tìm kiếm trong vector store. Tuy nhiên, do sự đa nghĩa và tính phức tạp của ngôn ngữ tự nhiên, một câu hỏi có thể được diễn đạt bằng nhiều cách khác nhau. Việc chỉ sử dụng một truy vấn duy nhất có thể dẫn đến việc bỏ sót các đoạn văn bản liên quan quan trọng.

MultiQueryRetriever giải quyết vấn đề này bằng cách tạo ra nhiều truy vấn khác nhau dựa trên câu hỏi ban đầu. Các truy vấn này có thể được tạo ra bằng cách sử dụng các kỹ thuật như cắt ngữ, đồng nghĩa từ, biến đổi câu hỏi, hoặc áp dụng các mẫu ngữ pháp khác nhau. Sau đó, tất cả các truy vấn này sẽ được sử dụng để tìm kiếm trong vector store, và các đoạn văn bản liên quan thu được từ tất cả các truy vấn sẽ được tổng hợp lại. Bằng cách sử dụng nhiều truy vấn khác nhau, MultiQueryRetriever có thể tăng khả năng thu thập các đoạn văn bản liên quan bằng cách khai thác các biểu diễn khác nhau của câu hỏi. Điều này giúp giảm thiểu rủi ro bỏ sót thông tin quan trọng và cải thiện chất lượng của ngữ cảnh được truy xuất để trả lời câu hỏi.



Hình 15: Quy trình tạo ra và tìm đoạn văn bản liên quan
(MultiQueryRetriever)

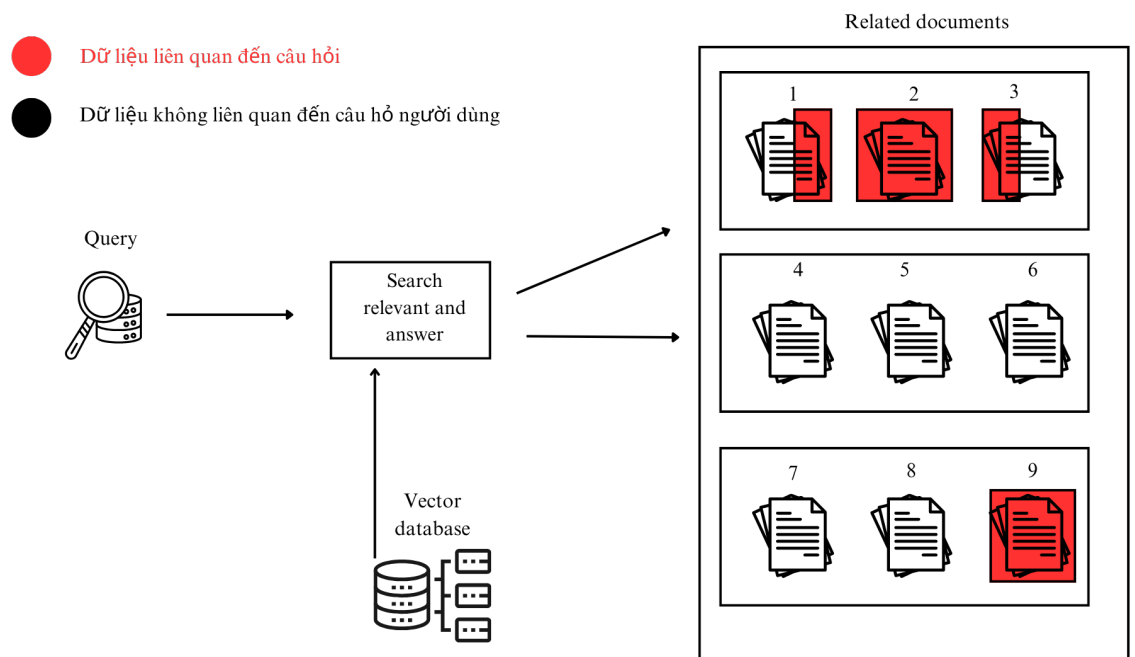
Từ câu hỏi của người dùng, mô hình sẽ tạo ra thêm các câu hỏi tương tự, mục tiêu là làm đa dạng phong cách hỏi, ngữ cảnh câu văn, từ đó tăng khả năng tìm được các đoạn văn bản chứa thông tin cần thiết trong Vector database. Việc sử dụng MultiQueryRetriever đã giúp hệ thống tăng cơ hội tìm kiếm được thông tin cần thiết nhưng cũng làm hệ thống giảm thiểu về mặt hiệu suất (đánh đổi giữa việc tăng tỉ lệ trả lời chính xác và thời gian phản hồi). Vì vậy, ta cần phải có những đánh giá thêm về vấn đề này.

Một vấn đề phát sinh trong quá trình truy vấn bằng Vector Store-backed Retriever hoặc MultiQueryRetriever là Related documents trả về có độ dài quá ngắn. Lý do, vì ở bước Phân chia dữ liệu (ở phần trên), chúng ta đã định nghĩa các chunk có độ dài cố định. Nếu ta chọn lưu trữ một chunk dài, giả sử khoảng 800 ký tự, các thông tin trong chunk đó sẽ có xu hướng bị nhiễu. Do một đoạn văn bản dài thường chứa nhiều thông tin khác nhau, khi ta tiến hành truy vết theo vector đại diện, hệ thống thường sẽ không tìm đúng được dữ liệu ta cần. Ta sẽ phải lưu trữ các chunk có chunk size nhỏ hơn. Phương pháp này cho thấy hiệu quả tốt hơn so với việc lấy chunk size dài, do các câu ngắn sẽ tập trung vào một thông tin nhất định, vì vậy các vector đại diện sẽ không bị nhiễu. Vấn đề ở đây là khi ta tìm được các văn bản liên quan từ câu hỏi của người dùng (Related documents), các Related documents có độ dài nhỏ sẽ không đảm bảo được lượng thông tin cần thiết, các ngữ cảnh không đầy đủ làm thông tin trả về bị thiếu, gây ra hiện tượng trả lời không đầy đủ của chatbot. Vì vậy, chúng em đề xuất một phương pháp vừa giải quyết vấn đề truy vấn hiệu quả từ chunk size nhỏ, vừa trả về đầy đủ ngữ cảnh chứa các câu liên quan (Related documents) là Parent Document Retriever.

Phương pháp Parent Document Retriever là một kỹ thuật quan trọng trong hệ thống RAG. Phương pháp này được sử dụng để giải quyết một trong những hạn chế của Vector Store-backed Retriever, đó là khả năng lấy lại ngữ cảnh liên quan có thể bị giới hạn bởi kích thước của vector store. Trong Vector Store-backed Retriever truyền thống, các đoạn văn bản được biểu diễn dưới dạng vector và lưu trữ trong vector store. Tuy nhiên, do giới hạn về kích thước, vector store có thể không chứa

đủ thông tin để trả lời đầy đủ câu hỏi, đặc biệt là trong trường hợp câu hỏi đòi hỏi nhiều ngữ cảnh liên quan.

Parent Document Retriever giải quyết vấn đề này bằng cách kết hợp việc truy xuất từ vector store và từ tập hợp các tài liệu gốc. Thay vì chỉ tìm kiếm trong vector store, phương pháp này cũng truy xuất các tài liệu gốc liên quan đến câu hỏi. Sau đó, các đoạn văn bản từ các tài liệu gốc này sẽ được xử lý và truy vấn trong vector store để tìm ra các đoạn văn bản liên quan nhất. Bằng cách kết hợp việc truy xuất từ vector store và tài liệu gốc, Parent Document Retriever đảm bảo rằng không có ngữ cảnh quan trọng nào bị bỏ sót do giới hạn của vector store. Điều này giúp cải thiện chất lượng của ngữ cảnh được truy xuất, đồng thời đảm bảo rằng hệ thống có đủ thông tin để trả lời câu hỏi một cách đầy đủ và chính xác hơn.



Hình 16: Quy trình của Parent Document Retriever

Các chunk size của một tài liệu liên quan có thể nhỏ, nhưng chúng ta có thể chỉ định nó là tập con của một chunk size lớn hơn. Ở hình trên, giả sử vector store của chúng ta có 9 chunks, mỗi chunk được ký hiệu bằng icon 📄, phân màu đỏ tượng trưng cho thông tin liên quan cần thiết để đáp ứng câu hỏi của người dùng.

Thông thường, các thông tin sẽ nằm ở nhiều chunk khác nhau, và mỗi chunk có thể có toàn bộ hoặc một phần nhỏ thông tin. Parent Document Retriever có vai trò gộp một số lượng chunk nhất định (parent chunk) - ở ví dụ trên là 3 chunks nhỏ (child chunk) hợp thành một parent chunk. Khi mô hình truy vết tìm thấy thông tin liên quan đến câu hỏi (phần màu đỏ). Hệ thống sẽ trả về parent chunk chứ không phải child chunk, điều này giúp hạn chế sự mất mát về thông tin cần thiết để trả lời câu hỏi do thiếu ngữ cảnh hoặc do cơ chế chọn số lượng tài liệu trả về. Các mô hình truy vết thường có số lượng trả về nhất định, nếu ta chọn số lượng trả về là 3, thì các tài liệu trả về có thể sẽ là [1, 2, 9], rõ ràng tổ hợp [1, 2, 9] không cung cấp đầy đủ thông tin cho câu trả lời. Còn nếu ta áp dụng Parent Document Retriever, với số lượng trả về là 3, thì toàn bộ các thông tin liên quan đến câu hỏi của người dùng sẽ được trả về đầy đủ, đó là 2 parent chunk chứa tập hợp các child chunk [1, 2, 3, 9].

3.1.7. Tổng hợp dữ liệu phản hồi

Việc tổng hợp dữ liệu để phản hồi cho người dùng là một nhiệm vụ quan trọng trong các hệ thống hỏi đáp dựa trên retrieval và generation (RAG). Sau khi thu thập được các đoạn văn bản liên quan từ các bước truy xuất, hệ thống cần phải tổng hợp và xử lý thông tin từ các đoạn này để tạo ra một câu trả lời cuối cùng nhằm đáp ứng yêu cầu của người dùng. Trong quá trình này, mô hình PhoBERT (Bidirectional Encoder Representations from Transformers dựa trên tiếng Việt) đóng một vai trò quan trọng.

PhoBERT là mô hình ngôn ngữ đã được tiền huấn luyện trên một lượng lớn dữ liệu tiếng Việt, cho phép nó hiểu và biểu diễn ý nghĩa của văn bản tiếng Việt một cách hiệu quả. Trong bước tổng hợp dữ liệu của RAG, PhoBERT có thể được sử dụng để biểu diễn cả câu hỏi của người dùng và các đoạn văn bản liên quan dưới dạng vector, giúp hệ thống có thể so sánh và đánh giá mức độ liên quan của chúng. Sau khi xác định được các đoạn văn bản quan trọng nhất, PhoBERT có thể được sử dụng để tổng hợp và sinh ra câu trả lời cuối cùng dựa trên thông tin từ các đoạn này. Quá trình sinh câu trả lời thường được thực hiện bằng cách sử dụng mô hình

decoder (như trong kiến trúc Transformer), với đầu vào là biểu diễn của câu hỏi và các đoạn văn bản liên quan từ PhoBERT. Sau khi xác định được các đoạn văn bản quan trọng nhất, PhoBERT có thể được sử dụng để tổng hợp và sinh ra câu trả lời cuối cùng dựa trên thông tin từ các đoạn này. Quá trình sinh câu trả lời thường được thực hiện bằng cách sử dụng mô hình decoder (như trong kiến trúc Transformer), với đầu vào là biểu diễn của câu hỏi và các đoạn văn bản liên quan từ PhoBERT. Việc sử dụng PhoBERT giúp hệ thống RAG có khả năng hiểu và xử lý ngôn ngữ tự nhiên tiếng Việt một cách hiệu quả hơn, đồng thời tận dụng được tri thức ngôn ngữ tiền huấn luyện sẵn trong mô hình. Điều này giúp cải thiện chất lượng của câu trả lời sinh ra, đảm bảo tính nhất quán, mạch lạc và chính xác của thông tin.

PhoBERT, với khả năng hiểu và biểu diễn ý nghĩa của văn bản tiếng Việt, đóng vai trò quan trọng trong việc trích xuất và diễn giải thông tin từ các đoạn văn bản liên quan. Mô hình này có thể biểu diễn câu hỏi của người dùng và các đoạn văn bản dưới dạng vector, cho phép hệ thống đánh giá mức độ liên quan và tầm quan trọng của chúng. Tuy nhiên, PhoBERT không được huấn luyện để sinh văn bản, nên việc tổng hợp và tạo ra câu trả lời cuối cùng có thể gặp khó khăn. Đây là nơi GPT, một mô hình sinh văn bản mạnh mẽ được huấn luyện trên một lượng lớn dữ liệu ngôn ngữ tự nhiên, đóng vai trò quan trọng. Bằng cách kết hợp với GPT, hệ thống RAG có thể tận dụng khả năng sinh văn bản của mô hình này để tạo ra câu trả lời cuối cùng dựa trên thông tin đầu vào từ PhoBERT.

Câu hỏi của người dùng sẽ được PhoBERT, trả lời dựa vào đoạn văn bản liên quan (Related documents) được tìm ra ở bước truy vết. Sau đó, kỹ thuật prompt engineering được thực hiện để mô hình GPT có thể tổng hợp văn bản và diễn giải một cách trơn tru hơn. Chúng em đề xuất đoạn prompt sau cho GPT để tổng hợp câu trả lời từ mô hình PhoBERT:

Role: You are a specialized assistant, responsible for answering questions about academics and admissions at Ho Chi Minh City Open University. Please use the information in the "Context" section, combined with the keywords given by the

system to give the appropriate answer.

Mission:

Carefully review the "Context" section:

- Identify key concepts, terms, or ideas relevant to the topic.
- Understand the overall context and background information.

Analyze the "Keyword" and its relevance to the "Question":

- Determine if the "Keyword" provides additional information or guidance.
- Assess whether the "Keyword" is essential for answering the question.

Formulate a comprehensive and informative response:

- Utilize the information from "Context" and "Keyword" to address the question directly.
- Avoid introducing irrelevant or unnecessary details.
- Ensure the response is clear, concise, and easy to understand.

Format the response in Vietnamese:

- Provide the response in Vietnamese using proper grammar and syntax.
- Maintain a professional and objective tone throughout the response.

Address potential limitations:

- If the "Context" and "Keyword" lack sufficient information, acknowledge the lack of clarity.
- If the question cannot be answered definitively, write: "I don't have enough information about this issue."

Example: Context: A vector store-based retrieval engine is a system designed to retrieve documents based on their similarity to a query. This system is based on two main components: Vector store-backed retriever: A database specifically designed

to store and manage high-dimensional vectors. These vectors represent the “meaning” or content of the document in numerical form. The vector store allows efficient retrieval of documents based on their vector similarity to a given query vector.

Question: What is vector store-backed retriever?

Keyword: database

Output: Vector store-backed retriever is a database specifically designed to store and manage high-dimensional vectors. These vectors represent the “meaning” or content of the document in numerical form. The vector store allows efficient retrieval of documents based on their vector similarity to a given query vector.

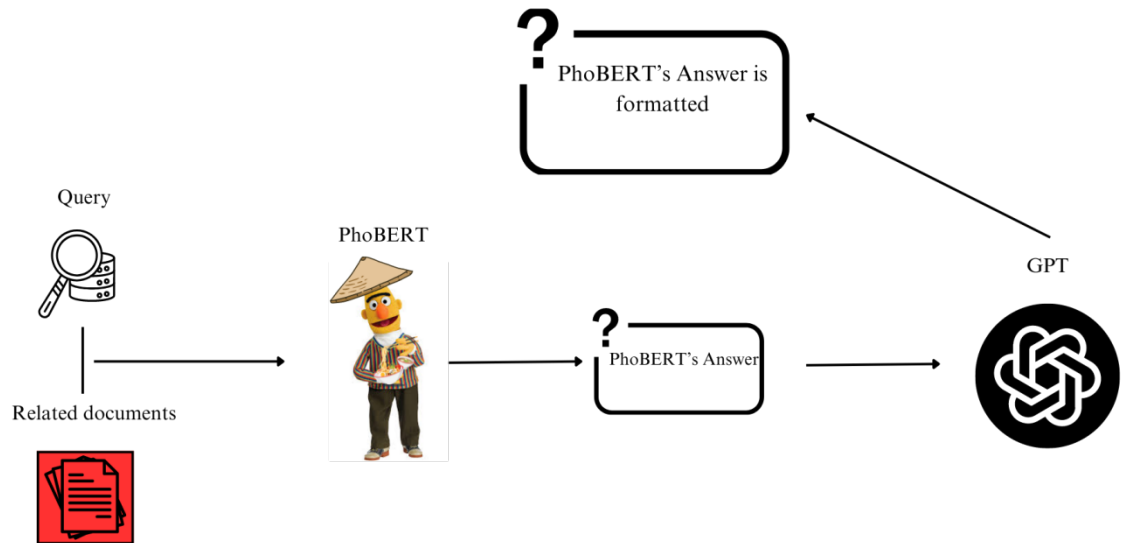
Context: {Đầu ra của mô hình truy vết}

Question: {Câu hỏi của người dùng}

Keyword: {Đầu ra của mô hình PhoBERT}

Output:

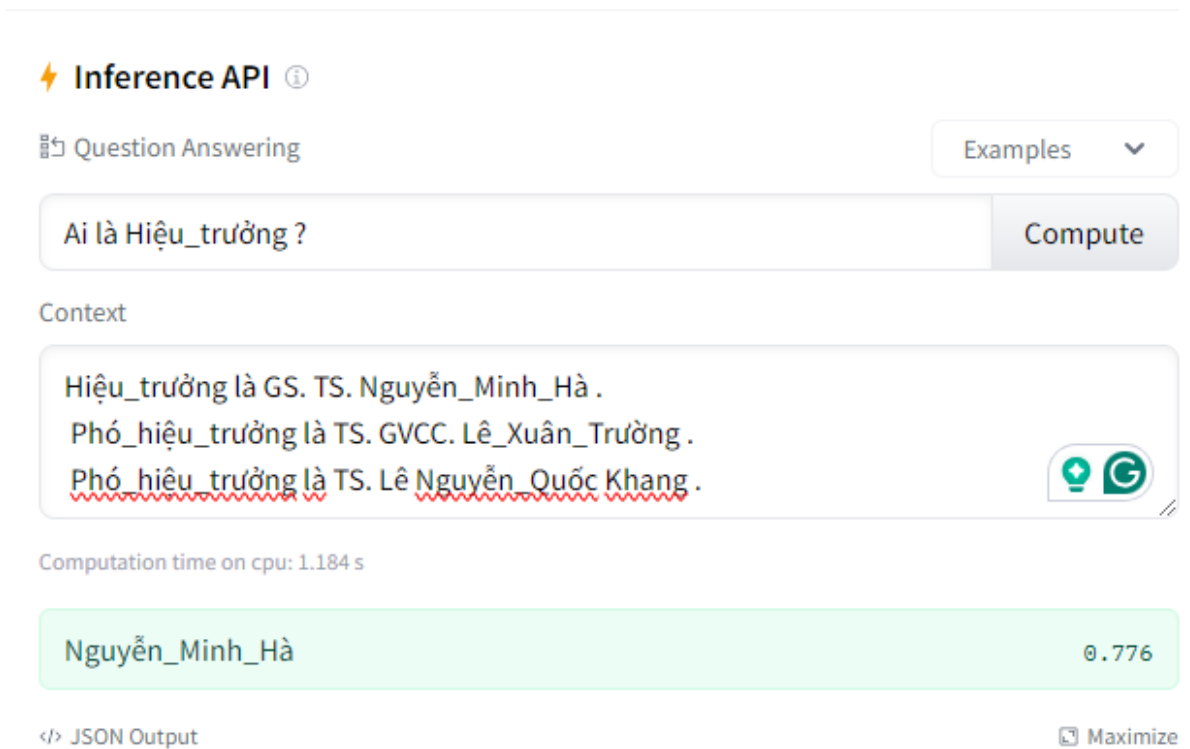
Table 9: Prompt tổng hợp câu trả lời từ PhoBERT và đoạn văn đã truy vết
Quy trình tổng hợp dữ liệu có thể được hiểu như sau:



Hình 17: Quy trình tổng hợp dữ liệu đưa ra cho người dùng

Câu hỏi và tài liệu liên quan đến câu hỏi được trả về từ quy trình truy vết dữ liệu sẽ được mô hình PhoBERT xử lý. PhoBERT nhận đầu vào gồm câu hỏi của người dùng và đoạn văn bản liên quan có thể trả lời câu hỏi đó. Sau đó, PhoBERT đưa ra đáp án cho câu trả lời, đáp án này gồm điểm bắt đầu và điểm kết thúc của câu trả lời nằm trong văn bản. Từ câu trả lời mà PhoBERT đưa ra, GPT sẽ tổng hợp câu hỏi và câu trả lời, mục đích là biến đổi câu trả lời của PhoBERT thành một câu trả lời tự nhiên nhất.

Giả sử người dùng nhập vào câu hỏi: “Ai là Hiệu trưởng?”, mô hình truy vết sẽ tìm ra đoạn văn bản liên quan từ vector database là: “Hiệu_trưởng là GS. TS. Nguyễn_Minh_Hà . Phó_hiệu_trưởng là TS. GVCC. Lê_Xuân_Trường . Phó_hiệu_trưởng là TS. Lê_Nguyễn_Quốc_Khang .”. Khi ta có được đoạn văn bản liên quan và câu hỏi, ta đưa chúng vào mô hình PhoBERT đã được fine-tune để trả lời. Đầu ra của mô hình PhoBERT sẽ là đáp án của câu hỏi.

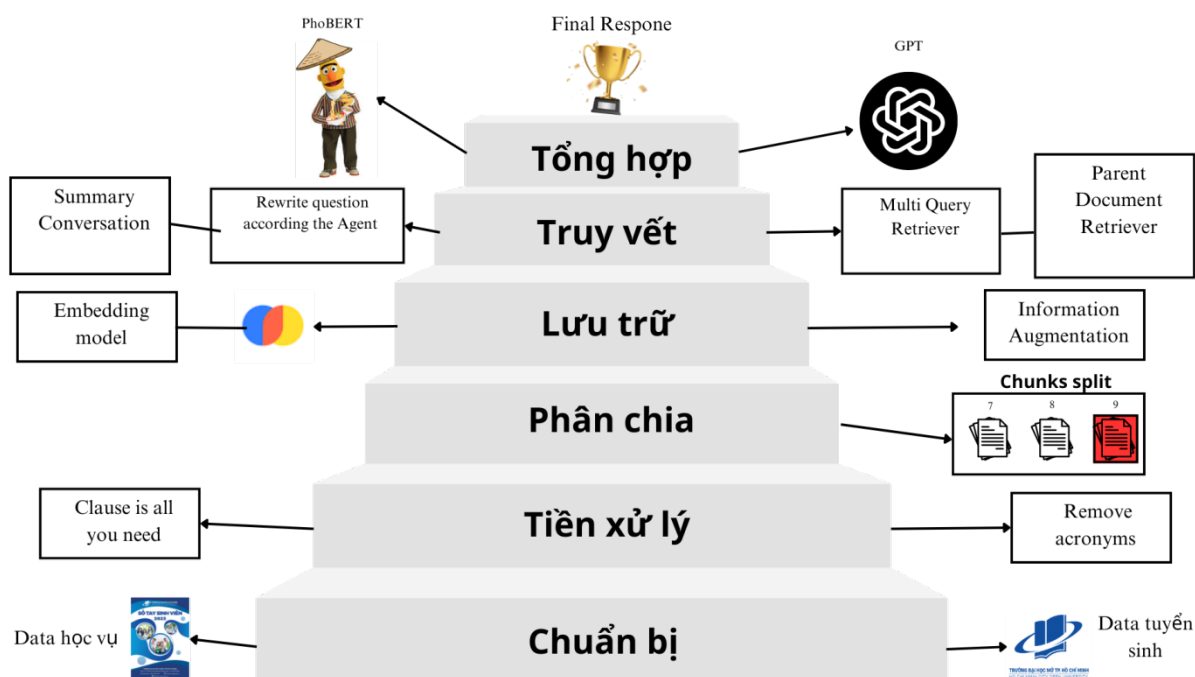


Hình 18: Ví dụ về câu trả lời được trả về từ PhoBERT

Đáp án của mô hình PhoBERT sẽ được mô hình tổng hợp văn bản GPT diễn giải một cách tự nhiên hơn, sau đó sẽ được trình bày cho người dùng. Câu trả lời sẽ được viết lại một cách trang trọng hơn là: “Hiệu trưởng của Trường Đại học Mở Thành phố Hồ Chí Minh là GS. TS. Nguyễn Minh Hà.”.

3.2. Tổng kết

Ở chương này chúng em tóm tắt cái nhìn tổng quát về mô hình RAG cải tiến phía trên. Quy trình này được chia làm 6 phần bao gồm: Chuẩn bị dữ liệu, tiền xử lý dữ liệu, phân chia dữ liệu, lưu trữ dữ liệu, truy vết dữ liệu và tổng hợp dữ liệu đã truy vết. Trong đó 4 thành phần đầu tiên gồm: Chuẩn bị dữ liệu, tiền xử lý dữ liệu, phân chia dữ liệu và lưu trữ dữ liệu, giúp hệ thống tạo nên một bộ não hay knowledge source. Khi đã có knowledge source, các kỹ thuật truy vết và tổng hợp sẽ lần lượt tìm kiếm thông tin liên quan và trả về câu trả lời cho người dùng.



Hình 19: Tổng hợp quy trình từ chuẩn bị dữ liệu đến khi đưa ra kết quả cho người dùng

Quy trình chuẩn bị dữ liệu bao gồm tập hợp các tài liệu có nguồn gốc uy tín, rõ ràng và chính xác.

	Nội dung
Vai trò	- Góp phần xây dựng kho lưu trữ tri thức bên ngoài cho mô hình RAG truy cập và khai thác thông tin. - Ảnh hưởng trực tiếp đến hiệu suất và hiệu quả của mô hình RAG. - Giúp trả lời các câu hỏi phức tạp đòi hỏi kiến thức sâu rộng trong một lĩnh vực cụ thể.
Thách thức	- Thu thập, tích hợp, tổ chức lượng lớn dữ liệu văn bản từ nhiều nguồn. - Phân loại, tổ chức cẩn thận để đảm bảo thông tin dễ dàng truy

	cập. – Lọc, chuẩn hóa, loại bỏ dữ liệu trùng lặp và sai lệch. – Đảm bảo bảo mật và tính nhạy cảm của dữ liệu. – Lưu trữ và tổ chức bằng cấu trúc dữ liệu phù hợp cho truy cập nhanh.
Giải pháp	– Lựa chọn nguồn tin cậy từ website trường đại học. – Thu thập, xử lý dữ liệu: loại bỏ nhiễu, trùng lặp, chuẩn hóa định dạng. – Đề xuất và phát triển phương pháp "Clause is all you need" để nâng cao chất lượng dữ liệu.

Table 10: Bảng tổng kết quy trình chuẩn bị dữ liệu

Quy trình tiền xử lý dữ liệu sẽ chuyển đổi dữ liệu thành một dạng mà RAG có thể sử dụng để truy xuất thông tin và tạo ra các câu trả lời.

	Nội dung
Vai trò	– Chuẩn bị nguồn tri thức (knowledge source) – Đảm bảo chất lượng dữ liệu đầu vào cho mô hình. – Loại bỏ từ viết tắt để tăng tính dễ hiểu và xử lý văn bản – Cải thiện hiệu suất hệ thống chatbot
Thách thức	– Xác định từ viết tắt – Xác định nghĩa chính xác của từ viết tắt – Thay thế từ viết tắt phù hợp trong ngữ cảnh – Đa nghĩa của câu – Đại từ nối câu

Giải pháp	– Sử dụng từ điển viết tắt, áp dụng phương pháp "Remove Acronyms" – Phương pháp "Clause is all you need" – Loại bỏ từ viết tắt
-----------	--

Table 11: Bảng tổng kết quy trình tiền xử lý dữ liệu

Quá trình phân chia dữ liệu trong RAG (Retrieval Augmented Generation) là một bước quan trọng để đảm bảo chất lượng và hiệu quả của chatbot.

	Nội dung
Vai trò	– Chia nhỏ dữ liệu nguồn tri thức thành các khối có kích thước khác nhau – Tối ưu hóa hiệu suất và hiệu quả của hệ thống – Cải thiện độ chính xác – Tăng cường tính linh hoạt – Tối ưu hóa truy xuất dữ liệu
Thách thức	– Phân chia nhỏ dữ liệu có kích thước là bao nhiêu
Giải pháp	– Chia tập dữ liệu thành các phần văn bản riêng biệt có độ dài khác nhau. – Dựa trên số lượng ký tự nhất định (các chunk)

Table 12: Bảng tổng kết quy trình phân chia dữ liệu

Quy trình lưu trữ dữ liệu trong RAG (Retrieval Augmented Generation) đóng vai trò quan trọng trong việc cung cấp thông tin chính xác và phù hợp cho chatbot. Dữ liệu được lưu trữ và tổ chức một cách hiệu quả để hệ thống có thể truy xuất và sử dụng chúng một cách nhanh chóng và chính xác.

	Nội dung
Vai trò	– Lưu trữ vector (Vector store) là kỹ thuật lưu trữ và truy xuất dữ liệu dựa trên vector mật độ trong hệ thống RAG. – Giúp tìm kiếm và trích xuất các đoạn văn bản liên quan ngữ nghĩa với câu hỏi, vượt qua hạn chế của phương pháp tìm kiếm từ khóa truyền thống. – Tiết kiệm không gian lưu trữ hơn so với phương pháp lưu trữ văn bản thông thường.
Thách thức	– Độ chính xác của truy vết dữ liệu có thể bị giảm do ngữ cảnh và cách diễn đạt câu hỏi/mệnh đề. – Cùng một mệnh đề có thể được diễn đạt theo nhiều cách khác nhau, dẫn đến độ tương đồng cosine khác nhau khi so sánh với câu hỏi.
Giải pháp	– Sử dụng phương pháp tự sáng tạo "Information Augmentation". – Tạo các phiên bản tương đồng phù hợp của mệnh đề gốc. – Biểu diễn các câu đó thành vector ngữ nghĩa. – Lấy trung bình cộng các vector làm vector đại diện để truy vết.

Table 13: Bảng tổng kết quy trình lưu trữ dữ liệu

Trong hệ thống RAG, quá trình truy vết dữ liệu đóng một vai trò quan trọng trong việc đảm bảo tính minh bạch và giải trình của các câu trả lời được tạo ra. Quá trình này trả về các đoạn văn bản trong Knowledge Source có liên quan đến câu hỏi người dùng. Các vai trò trong việc truy vết, thách thức và hướng giải quyết của chúng em trong quá trình thực hiện truy vết dữ liệu như sau.

	Nội dung
--	----------

Vai trò	– Đảm bảo tính minh bạch, trách nhiệm giải trình và độ tin cậy của hệ thống. – Giúp xác định rõ ràng nguồn gốc thông tin được sử dụng để tạo ra câu trả lời. – Cho phép truy ngược lại bằng chứng hỗ trợ cho câu trả lời.
Thách thức	– Lượng thông tin thu được từ Vector store-backed retriever có thể bị thiếu và nhiều cao. – Tiếng Việt có nhiều từ đồng nghĩa và đại từ thay thế, gây khó khăn cho truy vết. – Yêu cầu chatbot cần ghi nhớ lịch sử cuộc trò chuyện và hiểu đại từ thay thế. – Viết tắt gây khó khăn cho máy móc trong việc hiểu và xử lý thông tin.
Giải pháp	– Information Augmentation (IA) và MultiQueryRetriever: Tăng cường dữ liệu truy vấn để tìm kiếm thông tin chính xác hơn. – Tóm tắt cuộc trò chuyện: Giúp chatbot hiểu đại từ thay thế trong lịch sử trò chuyện. – Từ điển viết tắt: Biên dịch các từ viết tắt để cải thiện hiệu suất chatbot. – Quy trình biến đổi và truy vết câu hỏi (Rewrite question according the Agent và Summary Conversation): Biến đổi câu hỏi người dùng để tối ưu hóa truy vấn và truy vết dữ liệu trong vector database. – Tăng hiệu suất tìm ra câu trả lời và tránh nhiễu bằng phương pháp MultiQueryRetriever và Parent Document Retriever

	– MultiQueryRetriever tạo ra thêm những truy vấn tương đồng giúp tăng khả năng tìm thấy câu trả lời. – Parent Document Retriever trả về đoạn văn bản dài hơn, từ đó không làm thông tin bị thiếu.
--	--

Table 14: Bảng tổng kết quá trình truy vết dữ liệu

Quy trình tổng hợp dữ liệu và phản hồi là quy trình cuối cùng của một chatbot. Ở quá trình này, các mô hình ngôn ngữ sẽ được sử dụng để lấy thông tin từ quá trình truy vết trước đó, từ đó tìm ra được câu trả lời cho câu hỏi của người dùng.

	Nội dung
Vai trò	– Dùng để đưa ra câu trả lời cuối cùng cho người dùng bằng PhoBERT và GPT – PhoBERT: Hiểu và biểu diễn ý nghĩa văn bản tiếng Việt, đánh giá mức độ liên quan giữa câu hỏi và các đoạn văn bản. – GPT: Tổng hợp và tạo câu trả lời cuối cùng dựa trên thông tin từ PhoBERT, đảm bảo câu trả lời tự nhiên và mạch lạc. – Kỹ thuật Prompt Engineering: Hướng dẫn GPT tổng hợp văn bản một cách hiệu quả. – Kỹ thuật Few-shot learning: Đưa ra một số mẫu để GPT tổng hợp văn bản đúng với định dạng yêu cầu.
Thách thức	– PhoBERT không được huấn luyện để sinh văn bản, cần GPT để tạo câu trả lời cuối cùng. – Cần đào tạo PhoBERT trước trên tác vụ Question Answering. – Cần có các nghiên cứu về prompt engineering và few-shot learning để có thể tinh chỉnh đầu ra cho GPT

Giải pháp	– Kết hợp PhoBERT và GPT: – PhoBERT xử lý câu hỏi và đoạn văn bản liên quan, đưa ra đáp án. – GPT tổng hợp câu hỏi và đáp án từ PhoBERT thành câu trả lời tự nhiên. – Sử dụng kỹ thuật Prompt Engineering để hướng dẫn GPT đưa ra đáp án chính xác.
-----------	--

Table 15: Bảng tổng kết quá trình tổng hợp dữ liệu và phản hồi

Chương 4. CÁC TIẾP CẬN HỆ THỐNG VÀ KẾT QUẢ THỰC NGHIỆM

Chúng em đã tiến hành thực nghiệm phương pháp "Clause is all you need" nhằm cải thiện hiệu suất của chatbot trong việc trả lời câu hỏi từ nguồn kiến thức. Kỹ thuật này chuyển đổi văn bản thành các mệnh đề đơn giản, rõ ràng hơn để tăng khả năng truy vết và hiểu ngữ nghĩa. Để đánh giá hiệu quả, chúng em đã tạo ra hai bộ "Knowledge Sources", một không sử dụng và một có sử dụng kỹ thuật này, và thực hiện thí nghiệm với 600 cặp câu hỏi và câu trả lời.

Chúng em cũng nghiên cứu hiệu quả của việc loại bỏ từ viết tắt (Remove Acronyms) bằng cách thay thế từ viết tắt bằng ngôn ngữ thông thường và đánh giá tỷ lệ trả lời chính xác.

Phương pháp "Information Augmentation" (IA) được áp dụng để tăng cường khả năng truy vết dữ liệu bằng cách tạo ra các câu hỏi và mệnh đề tương đồng. Chúng em kiểm tra hiệu quả của IA bằng cách so sánh độ tương đồng cosine và tỷ lệ truy vết đúng top 1, 2, 3 của các phương pháp khác nhau.

Chúng em áp dụng kỹ thuật "Rewrite question according the Agent và Summary Conversation" để cải thiện khả năng chatbot hiểu và trả lời các câu hỏi có chứa đại từ thay thế. Kỹ thuật này được đánh giá bằng cách so sánh tỷ lệ trả lời đúng giữa việc sử dụng và không sử dụng kỹ thuật.

Ngoài ra, chúng em kết hợp sử dụng MultiQueryRetriever và Parent Document Retriever để cải thiện hiệu suất truy vấn và chất lượng thông tin trả về. MultiQueryRetriever tạo ra nhiều câu hỏi truy vấn tương đồng, trong khi Parent Document Retriever trả về những tài liệu dài hơn để cung cấp đầy đủ ngữ nghĩa hơn.

Cuối cùng, chúng em đã thực mô hình PhoBERT cho bài toán Trả lời Câu hỏi (QA) bằng tiếng Việt, sử dụng một bộ dữ liệu tự tổng hợp gồm hơn 120,000 câu hỏi từ báo, truyện, sách giáo khoa tiếng Việt và SQuAD dịch sang tiếng Việt. Tiền

xử lý dữ liệu được thực hiện bằng thư viện *VnCoreNLP* để phân loại từ và tạo token, và mô hình được *fine-tuning* bằng *API Trainer* của *Hugging Face* với các thông số tối ưu hóa. Sau đó, mô hình được đánh giá thông qua chỉ số *Exact Match* và *ROUGE-metric*.

4.1. Tiền xử lý dữ liệu

4.1.1. Thử nghiệm “Clause is all you need”

“Clause is all you need” là một kỹ thuật giúp tăng cường dữ liệu trước khi tạo một “Knowledge Sources” bằng cách chuyển các văn bản thuần thành các mệnh đề. Để đánh giá độ hiệu quả của phương pháp, chúng em đã tạo ra bộ câu hỏi gồm 600 cặp câu hỏi và câu trả lời được tạo ra từ miền tri thức gốc. Đồng thời, chúng em cũng tạo ra 2 “Knowledge Sources”, một không sử dụng phương pháp “Clause is all you need”, một áp dụng tiền xử lý “Clause is all you need”. Các câu hỏi là những câu có thể trả lời được bằng “Knowledge Sources”, câu trả lời sẽ được con người đọc hiểu tài liệu và trả lời sẵn. Mục tiêu của phương pháp đánh giá này là khi áp dụng và không áp dụng “Clause is all you need”, chatbot có thể tìm thấy được nguồn thông tin để trả lời hay không và độ diễn giải có gần giống với cách con người diễn giải hay không. Ví dụ cặp câu hỏi và câu trả lời như sau:

Trường được thành lập khi nào?	Trường Đại học Mở Thành phố Hồ Chí Minh được thành lập vào ngày 15/6/1990.
--------------------------------	--

Table 16: Bảng ví dụ về data test cho thử nghiệm “Clause is all you need”

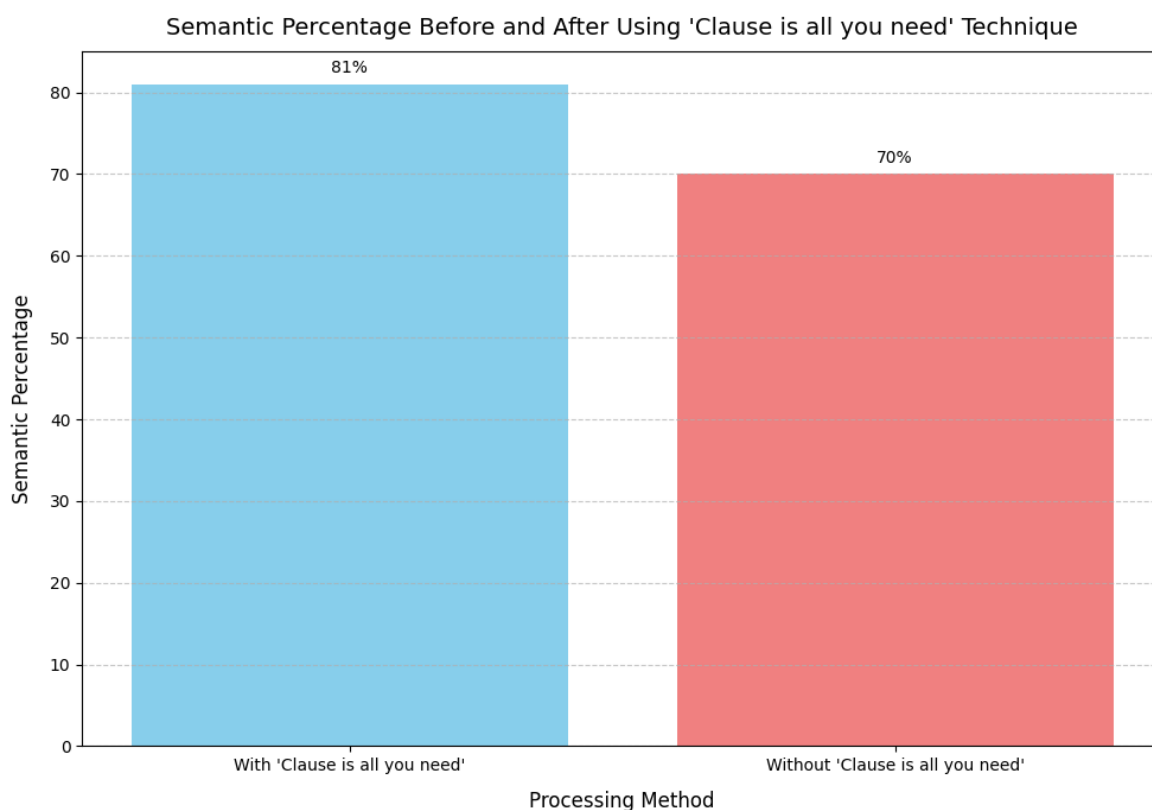
Chúng em tiến hành thử nghiệm bằng cách kiểm tra sự tương đồng ngữ nghĩa, tỷ lệ câu hỏi không thể trả lời khi dùng và không dùng kỹ thuật “Clause is all you need”. Chúng em có bảng kết quả sau:

Kỹ thuật	Độ tương đồng ngữ nghĩa (%)	Số câu không thể trả lời	Tỷ lệ câu không thể trả lời (%)
----------	-----------------------------	--------------------------	---------------------------------

Không sử dụng kỹ thuật	70	55/600	9.17
Sử dụng “Clause is all you need”	81	8/600	1.33

Table 17: Bảng kết quả thực nghiệm cho kỹ thuật “Clause is all you need”
(1)

Đầu tiên chúng em sẽ kiểm tra sự tương đồng ngữ nghĩa của câu trả lời của chatbot khi sử dụng 2 “Knowledge Sources” khác nhau. Nếu câu trả lời càng khác so với câu trả lời mẫu, chứng tỏ chatbot không thể tìm được dữ liệu để trả lời hoặc tìm sai, từ đó dẫn đến phần trăm tương đồng ngữ nghĩa sẽ giảm xuống.

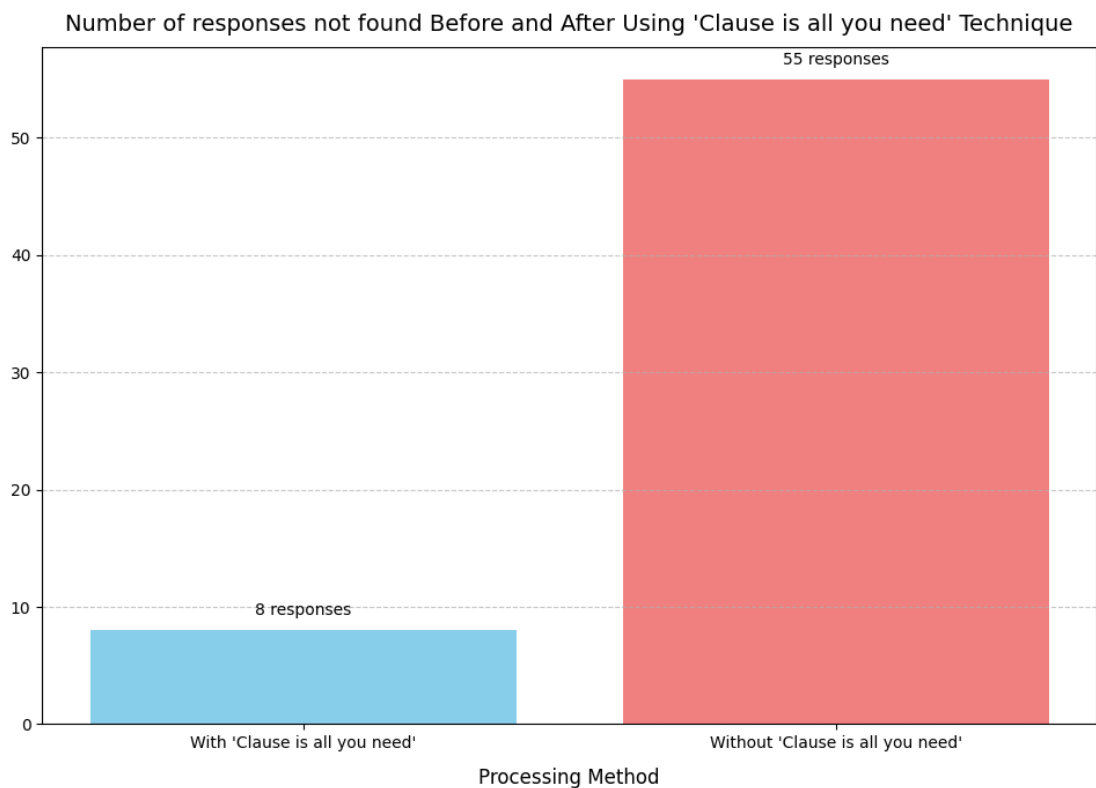


Hình 20: Biểu đồ so sánh tỉ lệ ngữ nghĩa khi dùng kỹ thuật “Clause is all you need”

Kết quả thực nghiệm cho thấy, chatbot sử dụng miền kiến thức đã được xử lý qua kỹ thuật “Clause is all you need” có thể đưa ra câu trả lời có độ chính xác cao hơn và độ chính xác tốt hơn. Với “Clause is all you need”, chatbot đưa ra câu trả lời

gần giống với câu trả lời tự nhiên và đầy đủ lên đến 81%, trong khi con số này chỉ là 70% nếu không sử dụng kỹ thuật.

Ngoài ra chúng em cũng tiến hành kiểm tra khả năng có thể tìm được câu hỏi của chatbot khi sử dụng kỹ thuật “Clause is all you need”. Chúng em tiến hành truy vết bằng các kỹ thuật Vector store-backed retriever trên bộ câu hỏi 600 câu đã giới thiệu ở phía trên, sau đó kiểm tra xem dữ liệu trả về có thể trả lời được câu hỏi hay không.



Hình 21: Biểu đồ so sánh số lượng câu hỏi không thể trả lời khi dùng kỹ thuật “Clause is all you need”

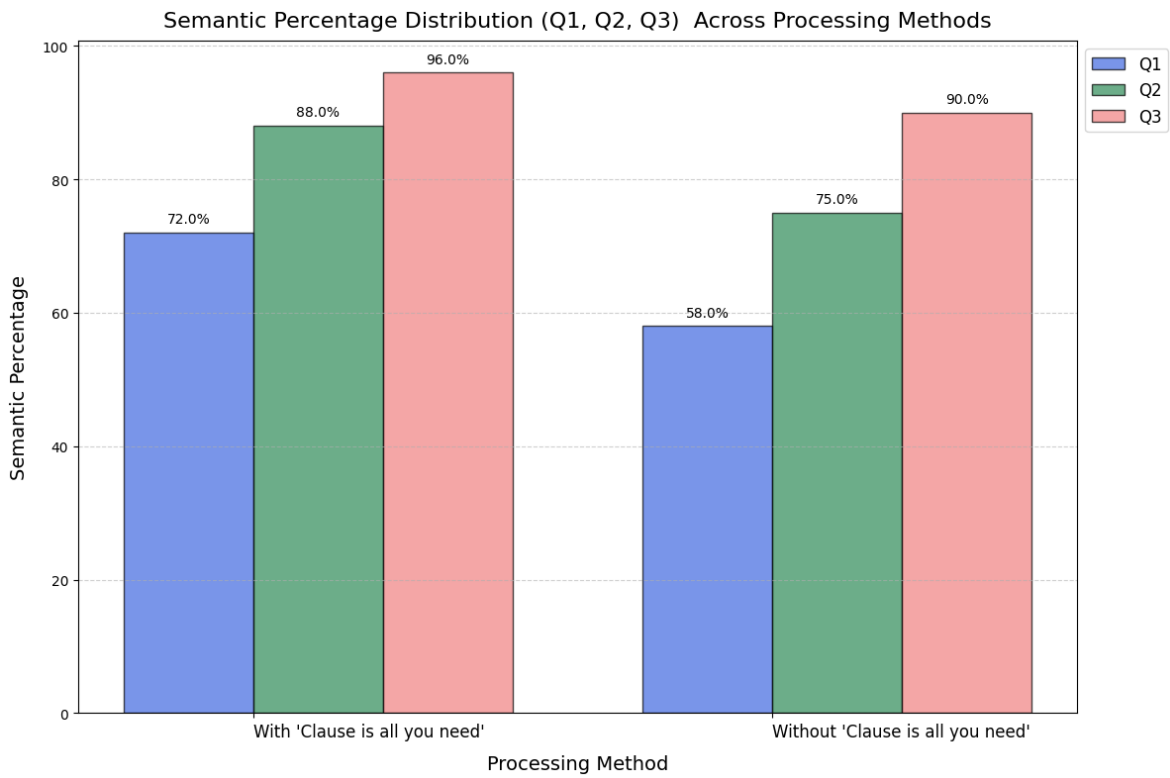
Với việc chuyển miền tri thức thành các mệnh đề, chatbot có thể dàng tìm ra được dữ liệu để trả lời cho câu hỏi hơn. Kết quả cho thấy, khi ứng dụng kỹ thuật “Clause is all you need”, chỉ có 8/600 câu chatbot không thể tìm thấy thông tin cần thiết để trả lời. Trong khi con số đó là 55/600 nếu không sử dụng “Clause is all you need”. Số câu hỏi không tìm thấy của mô hình giảm đến 85.45% khi chuyển các văn bản thô thành mệnh đề.

Cuối cùng để có cái nhìn tổng quát, chúng em thể hiện mức độ tương đồng của những câu trả lời của chatbot khi sử dụng và không sử dụng “Clause is all you need” thông qua tứ phân vị. Chúng em thu được kết quả sau:

Kỹ thuật	Độ tương đồng ngữ nghĩa tại Q1(%)	Độ tương đồng ngữ nghĩa tại Q2(%)	Độ tương đồng ngữ nghĩa tại Q3(%)
Không sử dụng kỹ thuật	72	88	96
Sử dụng “Clause is all you need”	58	75	90

Table 18: Bảng kết quả thực nghiệm cho kỹ thuật “Clause is all you need”
(2)

Để có cái nhìn trực quan, chúng em đã thể hiện kết quả dựa trên biểu đồ.



Hình 22: Biểu đồ so sánh tứ phân vị phần trăm ngữ nghĩa khi dùng kỹ thuật “Clause is all you need”

Khi sử dụng tứ phân vị, ta có thể có góc nhìn tổng quan về hiệu suất của chatbot khi sử dụng kỹ thuật “Clause is all you need”, Khi sử dụng “Clause is all you need”, các khoảng Q1 (25% dữ liệu thấp nhất), có độ tương đồng cao hơn 14%. Tương tự với Q2 (trung vị), có độ tương đồng cao hơn đến 13%. Và cuối cùng là Q3 (khoảng 75% dữ liệu), có độ tương đồng cao hơn 6% so với việc không sử dụng “Clause is all you need”.

Kết luận, “Clause is all you need” là một phương pháp mới giúp cải thiện đáng kể hiệu suất của chatbot trong việc tìm kiếm và cung cấp câu trả lời chính xác từ các nguồn tri thức. “Clause is all you need” không chỉ cải thiện chất lượng các câu trả lời mà còn cải thiện đáng kể khả năng của chatbot trong việc xác định và sử dụng các nguồn thông tin có liên quan để trả lời các câu hỏi một cách chính xác. Điều này mở ra hướng đi mới trong việc phát triển và tối ưu hóa các hệ thống trả lời tự động, đặc biệt trong lĩnh vực trí tuệ nhân tạo và khai thác dữ liệu.

4.1.2. Remove acronyms

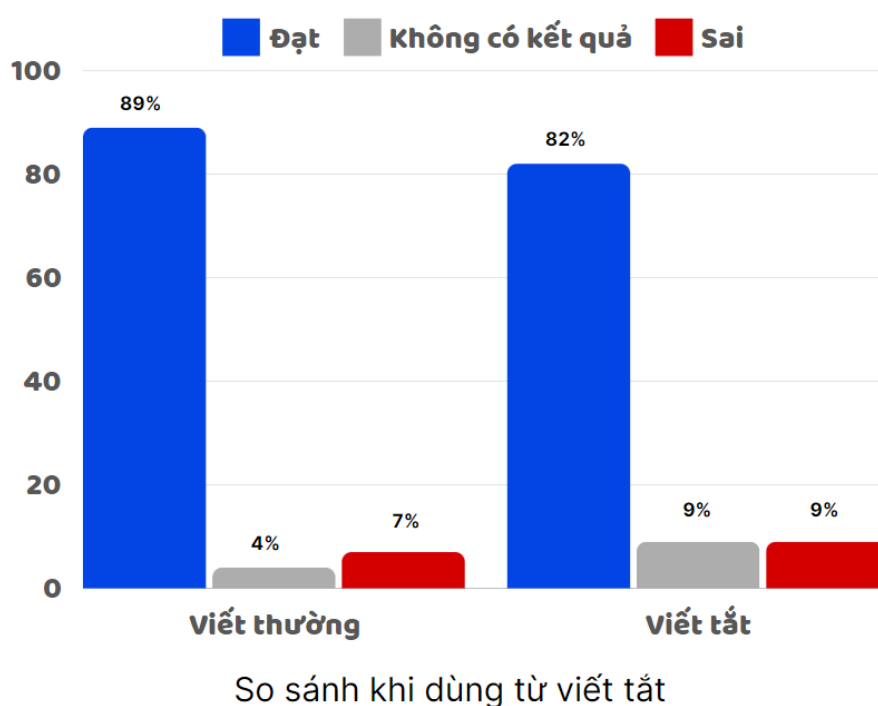
Mục tiêu của nghiên cứu này là đánh giá hiệu suất của chatbot khi xử lý các câu hỏi được thể hiện bằng ngôn ngữ thông thường so với các câu hỏi sử dụng chữ viết tắt. Dữ liệu được thu thập bằng cách đánh giá chatbot với 100 câu hỏi khác nhau, mỗi câu hỏi 2 lần: một lần sử dụng ngôn ngữ thông thường và một lần sử dụng từ viết tắt. Ví dụ: câu hỏi thông thường có thể là "Thời gian tiếp sinh viên hàng ngày là khi nào?" trong khi phiên bản viết tắt sẽ là "TG tiếp SV hàng ngày là khi nào?"

Kết quả chỉ ra rằng chatbot có xu hướng phản hồi chính xác hơn với các câu hỏi được đặt ra bằng ngôn ngữ thông thường so với những câu hỏi sử dụng từ viết tắt. Tỷ lệ chính xác của câu hỏi không có chữ viết tắt cao hơn 7% so với câu hỏi có chữ viết tắt. Điều này chứng tỏ việc loại bỏ các từ viết tắt giúp cải thiện hiệu suất của chatbot và giảm tỷ lệ câu hỏi chưa được trả lời.

Phương pháp	Độ trả lời chính xác
Thay thế từ viết tắt bằng Dictionary	~89%
IA cho mệnh đề và câu hỏi	~82%

Table 19: Bảng tỷ lệ trả lời đúng khi thay thế từ viết tắt so với không thay thế từ viết tắt

Chúng em cũng tiến hành trực quan hóa dữ liệu như sau.



Hình 23: Biểu đồ thể hiện độ chính xác khi thay thế và không thay thế từ viết tắt

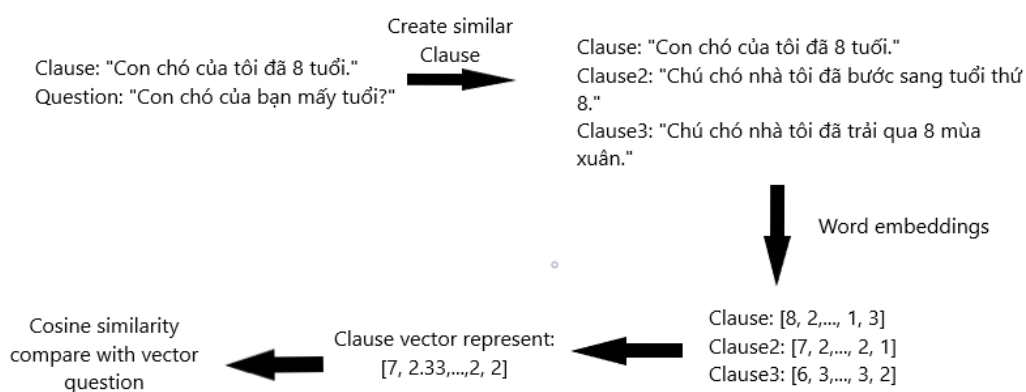
Có thể thấy rằng, chatbot có xu hướng trả lời chính xác hơn khi được đặt câu hỏi bằng ngôn ngữ bình thường so với từ viết tắt. Tỷ lệ trả lời chính xác khi người dùng không dùng từ viết tắt cao hơn 7% so với câu hỏi có chứa từ viết tắt, đồng thời loại bỏ từ viết tắt giúp chatbot tìm thấy trả lời chính xác cao hơn.

4.2. Lưu trữ dữ liệu bằng phương pháp Information Augmentation

Mục đích của việc đánh giá phương pháp Information Augmentation, chúng

em muốn kiểm tra xem IA có giúp tăng khả năng truy vết của chatbot hay không. Khả năng truy vết của chatbot bị ảnh hưởng bởi các vector tương đồng. Vì vậy, chúng em có những phương pháp so sánh sau.

Đầu tiên, chúng em tiến hành chuyển các đoạn văn ngẫu nhiên thành tập hợp các mệnh đề đầy đủ bằng kỹ thuật “Clause is all you need”. Để đảm bảo hiệu suất, chúng em đã sử dụng GPT 3.5 và các kỹ thuật prompt engineering để thực hiện tạo những câu tương đồng. Sau khi đã tạo được các câu mệnh đề từ bộ data raw, chúng em tiến hành tạo ra những câu tương đồng trên bộ dữ liệu mệnh đề, sau đó biểu diễn dưới dạng vector trung bình đại diện. Chúng em đã tạo ra và tiến hành thực nghiệm trên bộ dataset hơn 500 samples, bao gồm các Clause và các câu hỏi liên quan.



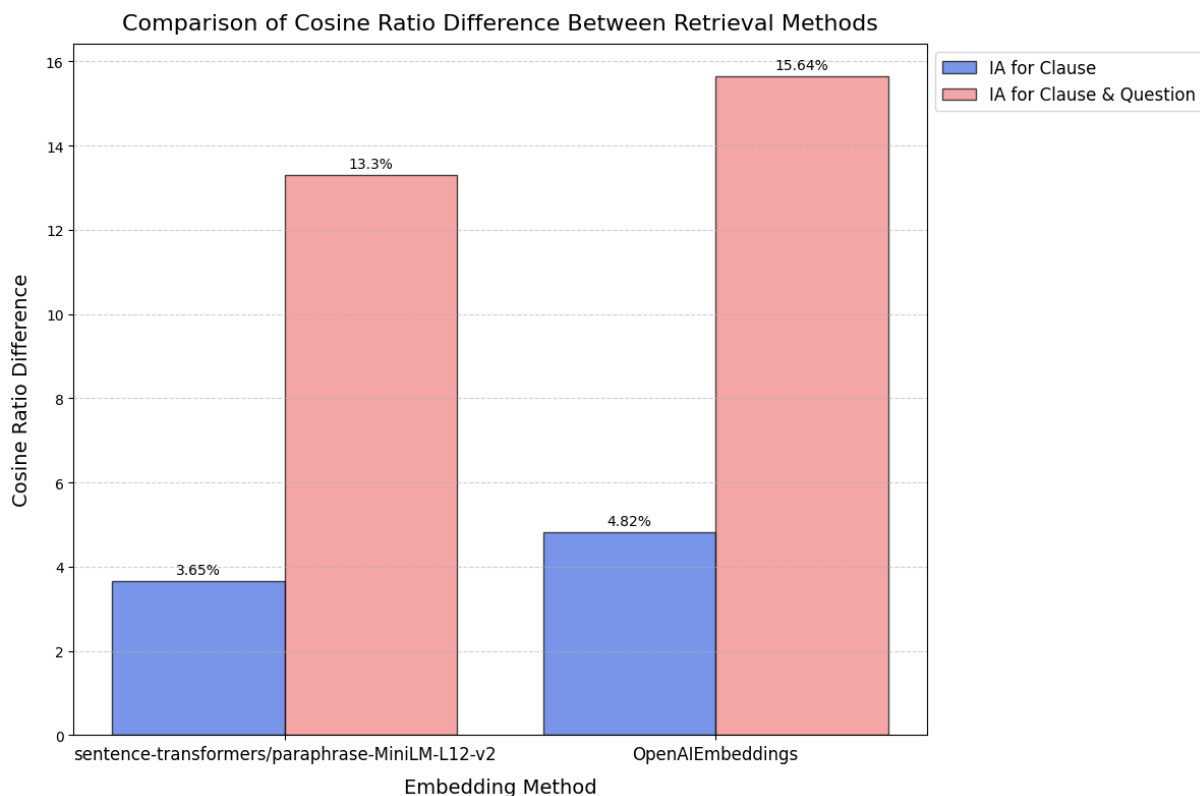
Hình 24: Quy trình cho phương pháp Information Augmentation

Word Embedding cũng là một yếu tố ảnh hưởng đến hiệu suất truy vấn, vậy nên chúng em đã sử dụng nhiều mô hình nhúng khác nhau để chuyển các câu thành vector biểu diễn. Chúng em đã thu được kết quả đáng mong đợi khi dùng sentence-transformers/paraphrase-MiniLM-L12-v2 và OpenAIEmbeddings để nhúng vector, kết hợp với phương pháp IA như sau.

Phương pháp	Độ cải thiện trung bình
-------------	-------------------------

IA cho mệnh đề	~5%
IA cho mệnh đề và câu hỏi	~16%

Table 20: Bảng kết quả khi sử dụng phương pháp Information Augmentation
Để thuận tiện cho việc so sánh phương pháp, chúng em có biểu đồ trực quan hóa như sau:



Hình 25: Tỷ lệ giữa phương pháp truy xuất truyền thống và IA cho Mệnh đề / IA cho Mệnh đề và Câu hỏi

Biểu đồ trên cho ta thấy phần trăm cải thiện của độ tương đồng cosine khi áp dụng Information Augmentation trong việc so sánh độ tương đồng cosine. Khi chúng ta áp dụng IA cho các mệnh đề trước khi tiến hành truy vết, độ tương đồng cosine giữa câu hỏi và đáp án đúng đã tăng trung bình ~4.5%. Khi chúng ta ứng dụng IA cho cả Clause và Question, độ tương đồng cosine của hai câu đã được cải thiện ~16%. Đây là một sự cải tiến đáng kể trong việc truy vết dữ liệu.

Kế tiếp, chúng em sẽ tiến hành tính toán tỷ lệ truy vết chính xác dựa trên top 1, 2, 3 độ tương đồng cosine lớn nhất, chúng em sẽ so sánh lần lượt giữa ba loại truy vết: truy vết truyền thống (embedding clause và question), IA for Clause (sử dụng kỹ thuật embedding clause + Information Augmentation và embedding question) và IA for Clause and Question (sử dụng kỹ thuật embedding clause + Information Augmentation và embedding question + Information Augmentation question). Chúng em sẽ tiến hành thực nghiệm trên bộ data trên 500 sample bằng cách áp dụng ba loại truy vết trên, mỗi row của data bao gồm Clause, năm câu hỏi có thể hỏi từ Clause đó nhằm đánh giá tốt nhất trong tất cả các trường hợp hỏi cụ thể.

Một sample dataset mẫu như sau:

Clause	Trường Đại học Mở Thành phố Hồ Chí Minh được thành lập vào ngày 15/6/1990.
Question1	Trường được thành lập khi nào?
Question2	Trường ra đời vào thời điểm nào?
Question3	Lịch sử thành lập trường là khi nào?
Question4	Trường bắt đầu hoạt động từ khi nào?
Question5	Thời điểm thành lập của trường là khi nào?

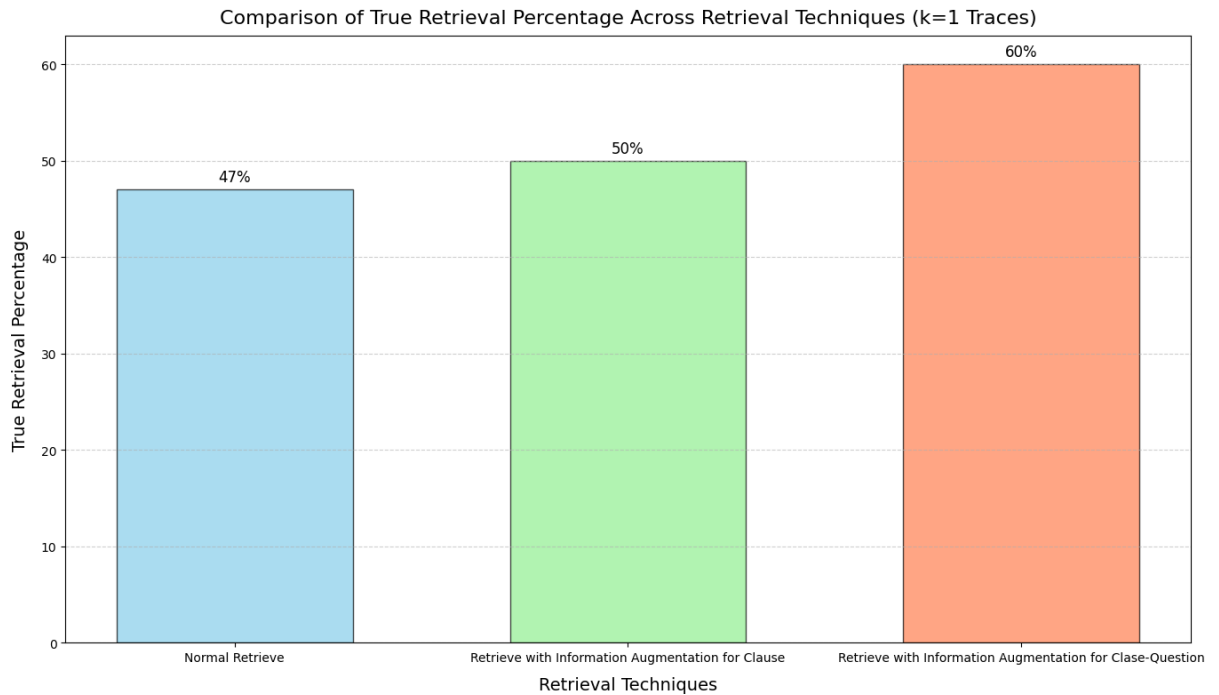
Table 21: Bảng ví dụ cho dataset test của phương pháp IA

Với việc lấy top 1 Clause có độ tương đồng lớn nhất trong tập dữ liệu 500 câu. Chúng em thu được kết quả như sau:

Phương pháp	Tỷ lệ truy vết đúng
Traditional Embedding	47%
IA for clause	50%

IA for clause-question	60%
------------------------	-----

Table 22: Bảng kết quả truy vết top 1 khi dùng Information Augmentation
 Từ kết quả trên, chúng em tiến hành trực quan hóa kết quả như sau:



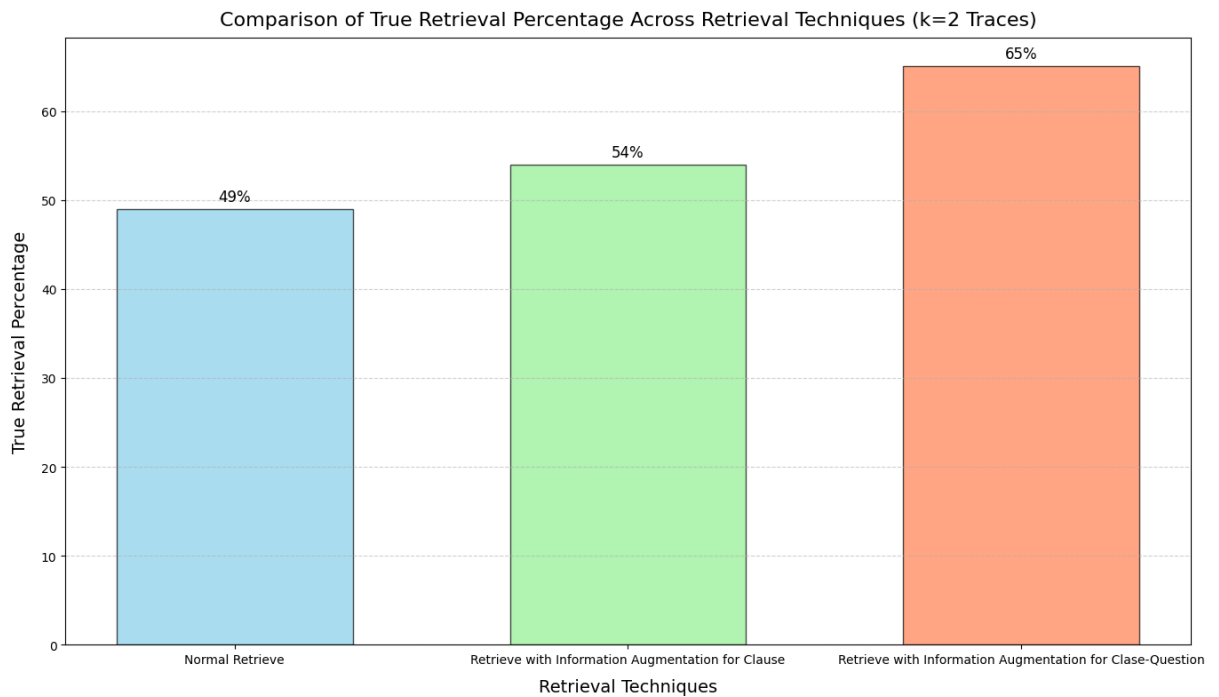
Hình 26: Tỷ lệ truy vết dữ liệu đúng mệnh đề với top $k = 1$

Với phương pháp Traditional Embedding tỷ lệ trung bình truy vết đúng Clause cần thiết để trả lời câu hỏi là khoảng 47%, với phương pháp IA for clause tỷ lệ trung bình truy vết đúng Clause cần thiết để trả lời câu hỏi là khoảng 50%, với phương pháp IA for clause-question tỷ lệ trung bình truy vết đúng Clause cần thiết để trả lời câu hỏi lên đến 60%. So sánh phương pháp tốt nhất (IA for clause-question) so với phương pháp truyền thống, ta có thể thấy tỷ lệ tìm được đoạn Clause cần thiết để trả lời câu hỏi tăng đến 13%.

Với việc lấy top 2 Clause có độ tương đồng lớn nhất trong tập dữ liệu 500 câu hỏi. Chúng em thu được kết quả như sau:

Phương pháp	Tỷ lệ truy vết đúng
Traditional Embedding	49%
IA for clause	54%
IA for clause-question	65%

Table 23: Bảng kết quả truy vết top 2 khi dùng Information Augmentation
 Từ kết quả trên, chúng em tiến hành trực quan hóa kết quả như sau:



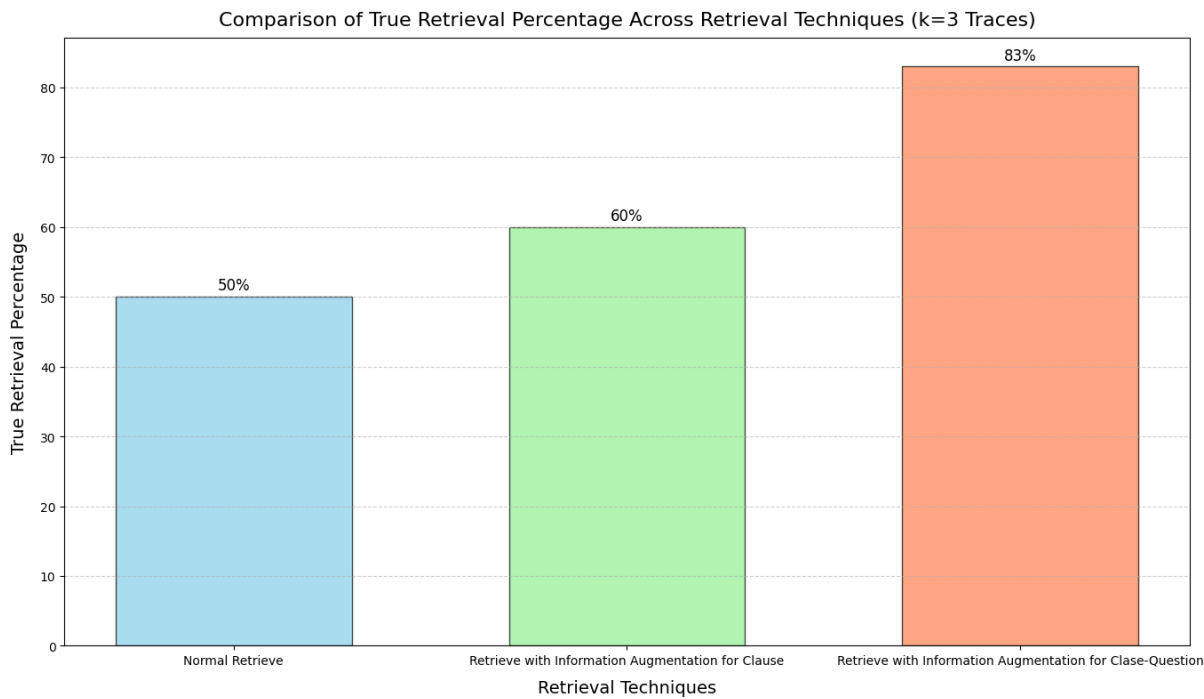
Hình 27: Tỷ lệ truy vết dữ liệu đúng mệnh đề với top $k = 2$

Với phương pháp Traditional Embedding tỉ lệ trung bình truy vết đúng Clause cần thiết để trả lời câu hỏi là khoảng 49%, với phương pháp IA for clause tỉ lệ trung bình truy vết đúng Clause cần thiết để trả lời câu hỏi là khoảng 54%, với phương pháp IA for clause-question tỉ lệ trung bình truy vết đúng Clause cần thiết để trả lời câu hỏi lên đến 65%. So sánh phương pháp tốt nhất (IA for clause-question) so với phương pháp truyền thống, ta có thể thấy tỉ lệ tìm được đoạn Clause cần thiết để trả lời câu hỏi tăng đến 16%.

Với việc lấy top 3 Clause có độ tương đồng lớn nhất trong tập dữ liệu 500 Clause. Chúng em thu được kết quả như sau:

Phương pháp	Tỷ lệ truy vết đúng
Traditional Embedding	50%
IA for clause	60%
IA for clause-question	83%

Table 24: Tỷ lệ truy vết dữ liệu đúng mệnh đề với top $k = 3$
Từ kết quả trên, chúng em tiến hành trực quan hóa kết quả như sau:



Hình 28: Tỷ lệ truy vết dữ liệu đúng mệnh đề với top $k = 3$

Với phương pháp Traditional Embedding tỉ lệ trung bình truy vết đúng Clause cần thiết để trả lời câu hỏi là khoảng 50%, với phương pháp IA for clause tỉ lệ trung bình truy vết đúng Clause cần thiết để trả lời câu hỏi là khoảng 60%, với phương pháp IA for clause-question tỉ lệ trung bình truy vết đúng Clause cần thiết để trả lời câu hỏi lên đến 83%. So sánh phương pháp tốt nhất (IA for clause-

question) so với phương pháp truyền thống, ta có thể thấy tỉ lệ tìm được đoạn Clause cần thiết để trả lời câu hỏi tăng đến 33%.

Kết quả thực nghiệm đã minh chứng rằng phương pháp Information Augmentation không chỉ tăng cường độ tương đồng giữa câu hỏi và đáp án mà còn cải thiện đáng kể khả năng truy vết dữ liệu, từ đó nâng cao hiệu suất và chất lượng của hệ thống trả lời tự động. Điều này đem lại triển vọng lớn cho việc ứng dụng IA trong các ứng dụng thực tế của trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên.

4.3. Truy vết dữ liệu

4.3.1. Rewrite question according the Agent và Summary Conversation

Rewrite question according the Agent và Summary Conversation là phương pháp kết hợp giữa lịch sử trò chuyện và thay đổi cấu trúc câu hỏi. Mục tiêu là giúp chatbot có thể hiểu được câu hỏi có chứa các mệnh đề thay thế, từ đó có thể trả lời được các câu hỏi của người dùng.

Chúng em đã tạo bộ dataset gồm 100 đoạn hội thoại, trong đó người dùng mở đầu cuộc trò chuyện bằng cách hỏi về một thực thể, sau đó người dùng tiếp tục hỏi về các thực thể đó nhưng dùng đại từ thay thế, đảm bảo “Knowledge Sources” có thông tin có thể trả lời câu hỏi. Chúng em, so sánh khả năng trả lời thành công của chatbot khi sử dụng kỹ thuật “Rewrite question according the Agent và Summary Conversation” và khi không sử dụng. Một đoạn hội thoại mẫu như sau.

User	Chatbot
Quyết định số 451/TCCB được ban hành khi nào?	(Câu trả lời của chat bot)
Ai là người ban hành nó?	(Câu trả lời của chat bot)

Table 25: Ví dụ cho bộ dataset test kỹ thuật “Rewrite question according the Agent và Summary Conversation”

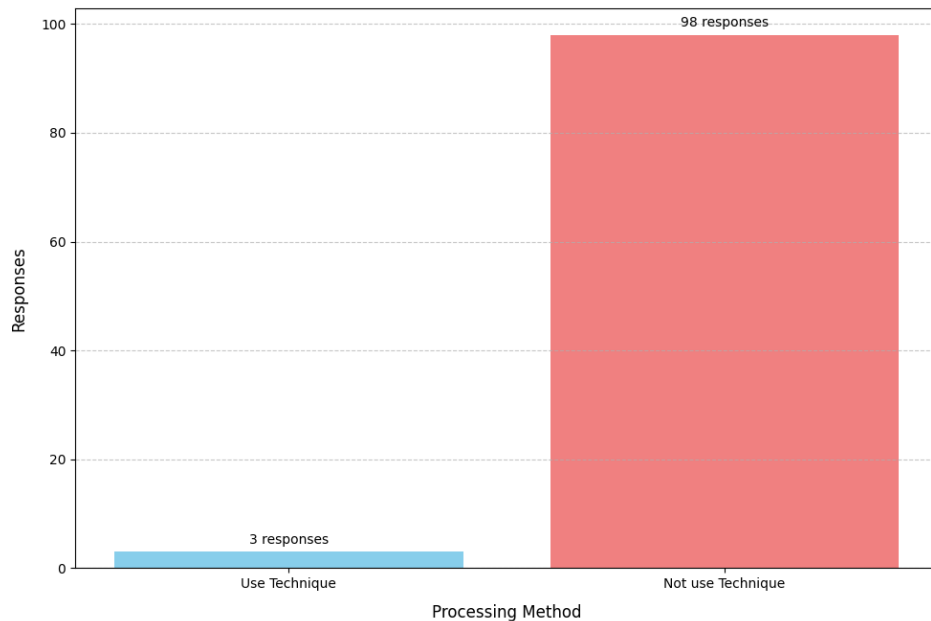
Sau khi tiến hành thử nghiệm chúng em thu được kết quả sau.

Phương pháp	Tỷ lệ trả lời đúng (%)	Tỷ lệ trả lời sai (%)
Không sử dụng kỹ thuật	2	98
Sử dụng “Rewrite question according the Agent và Summary Conversation”	97	3

Table 26: Bảng kết quả sử dụng kỹ thuật "Rewrite question according the Agent và Summary Conversation"

Chúng em tiến hành trực quan hóa kết quả như sau:

Number of responses not found Before and After Using 'Rewrite question according the Agent và Summary Conversation' Technique



Hình 29: Kết quả sử dụng kỹ thuật "Rewrite question according the Agent và Summary Conversation"

Khi sử dụng kỹ thuật “Rewrite question according the Agent và Summary Conversation”, có 97% câu hỏi đã được biên dịch lại và giúp chatbot trả lời thành công, và 3 % trả lời không thành công. Trong khi chỉ có 2% câu hỏi được chatbot trả lời được một cách may mắn nếu không được thay thế đại từ thay thế bằng thực thể và 98% câu hỏi chatbot không thể trả lời được.

Sử dụng kỹ thuật "Rewrite question according the Agent và Summary

Conversation" rất hiệu quả trong việc cải thiện khả năng trả lời câu hỏi của chatbot, đặc biệt là các câu hỏi sử dụng đại từ thay thế.

4.3.2. Vector retriever

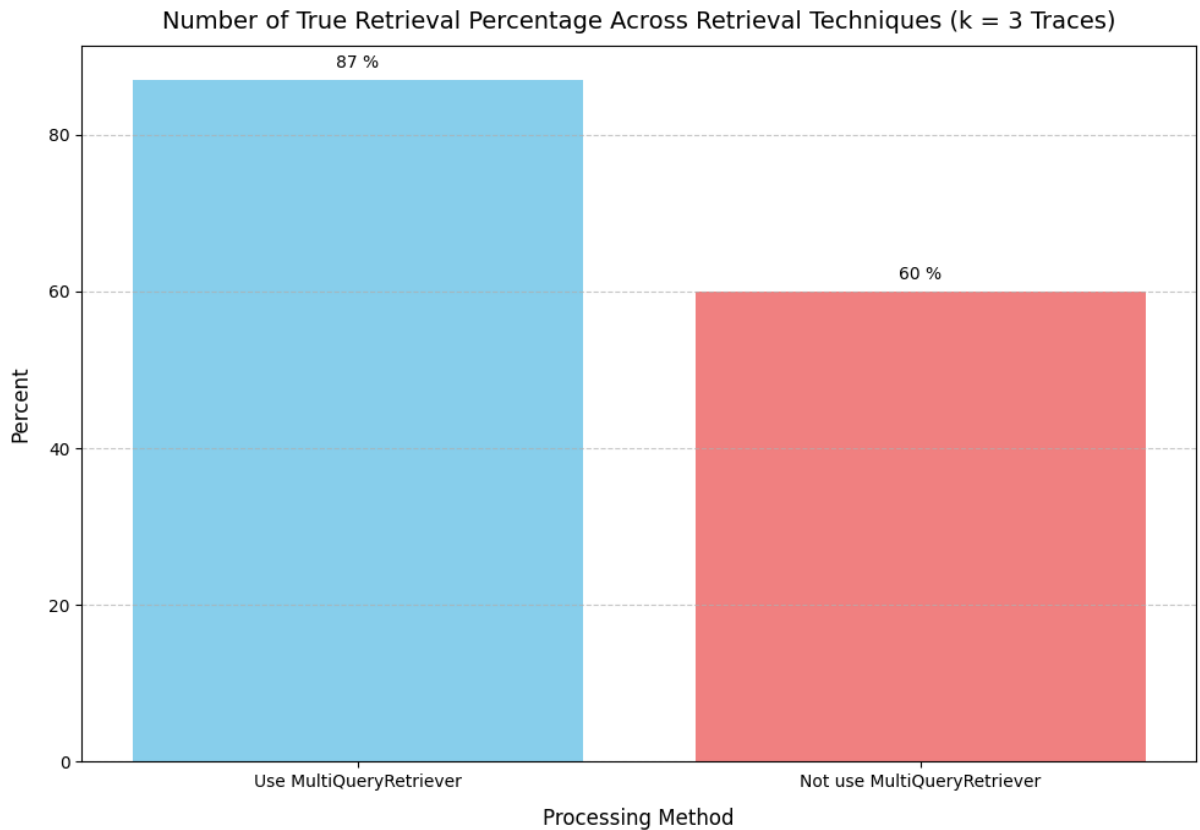
Thay vì chỉ sử dụng Vector store backed retriever truyền thống chúng em tiến hành kết hợp sử dụng MultiQueryRetriever và Parent Document Retriever, Trong đó MultiQueryRetriever tăng hiệu suất truy vấn thành công bằng cách tạo thêm nhiều câu hỏi truy vấn tương đồng, trong khi Parent Document Retriever là một phương pháp đóng gói giúp trả về những tài liệu dài hơn, từ đó đầy đủ ngữ nghĩa hơn.

Ở phương pháp MultiQueryRetriever chúng đã tạo bộ dữ liệu gồm hơn 1000 cặp câu hỏi - trả lời. Mỗi câu trả lời ứng với một câu hỏi và mục tiêu đề ra là so sánh hiệu quả tìm kiếm giữa truy vết truyền thống và MultiQueryRetriever. Đồng thời với mỗi câu hỏi chúng em cũng tạo thêm các câu hỏi tương đồng để tiến hành so sánh.

Chúng em tiến hành truy vết dữ liệu từ câu hỏi và lấy top 3 câu có độ tương đồng lớn nhất giữa một bên chỉ sử dụng 1 câu hỏi truy vấn và một bên dùng nhiều câu hỏi truy vấn. Chúng em đã thu được kết quả khả quan như sau.

Phương pháp	Tỷ lệ truy vết đúng (%)
Sử dụng MultiQueryRetriever	87
Không sử dụng MultiQueryRetriever	60

Table 27: Bảng kết quả khi sử dụng phương pháp MultiQueryRetriever
Chúng em tiến hành trực quan hóa kết quả như sau.



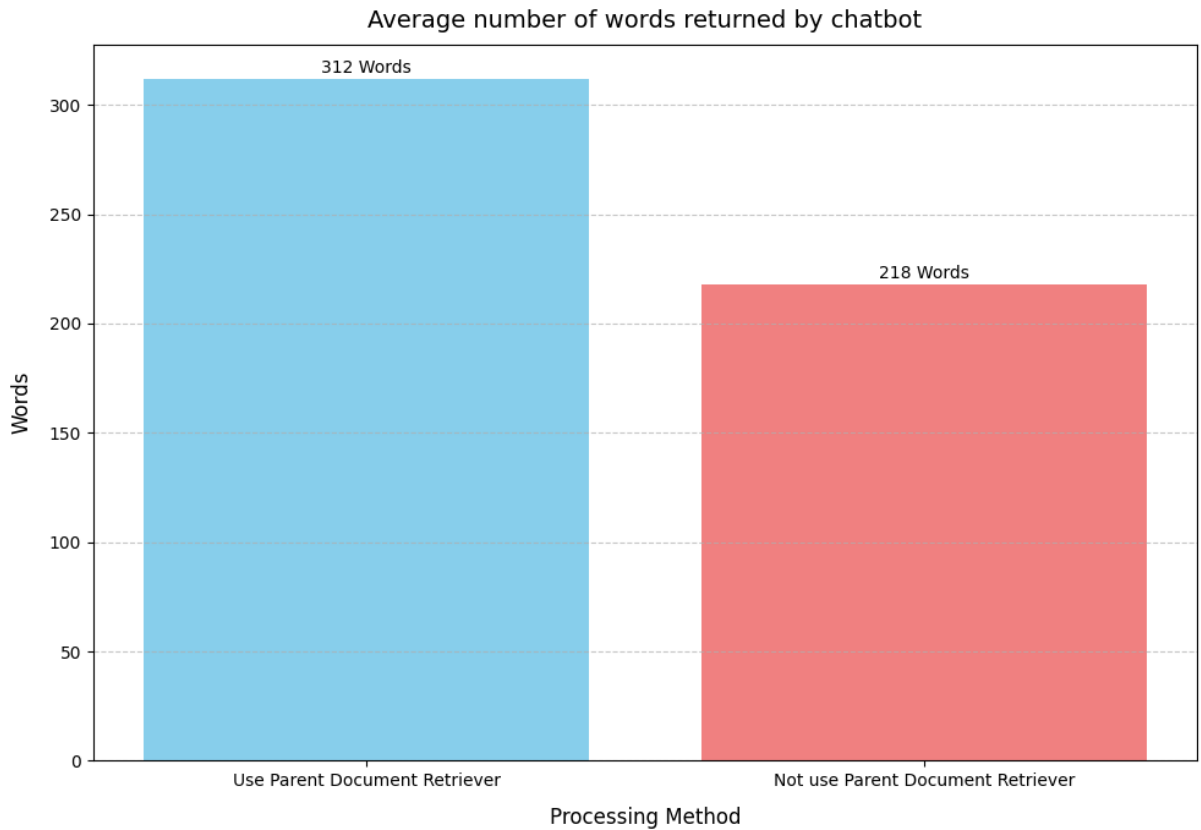
Hình 30: Biểu đồ thể hiện kết quả khi dùng và không dùng phương pháp MultiQueryRetriever

Khi sử dụng MultiQueryRetriever tỉ lệ truy vết thành công là 87% với việc lấy top 3 truy vết có độ tương đồng cao nhất. Tỉ lệ truy vết đúng cao hơn so với phương pháp truyền thống ~27%.

Bước tiếp theo, chúng em so sánh chất lượng văn bản trả về của chatbot khi sử dụng phương pháp Parent Document Retriever. Bộ dataset sử dụng là bộ dataset chúng em sử dụng ở phương pháp MultiQueryRetriever. Chúng em thu được kết quả trung bình số lượng từ trả về như sau.

Phương pháp	Số từ trả về (từ)
Sử dụng Parent Document Retriever	312
Không sử dụng Parent Document Retriever	218

Table 28: Bảng kết quả khi sử dụng phương pháp Parent Document Retriever
Để có các nhìn rõ ràng hơn, chúng em tiến hành trực quan hóa như sau.



Hình 31: Biểu đồ thể hiện kết quả khi dùng và không dùng phương pháp Parent Document Retriever

Số lượng từ phản hồi sau khi dùng Parent Document Retriever nhiều hơn ~1.5 lần so với khi không sử dụng. Chatbot có thể cung cấp được nhiều thông tin cần thiết hơn và mở rộng cuộc trò chuyện hơn.

Tổng kết lại, việc kết hợp sử dụng MultiQueryRetriever và Parent Document Retriever đã mang lại những kết quả đáng chú ý trong việc cải thiện hiệu suất và chất lượng của chatbot trong việc truy vấn và cung cấp thông tin.

4.4. Thực nghiệm trên PhoBERT

4.4.1. Huấn luyện mô hình

Nghiên cứu được triển khai mô hình PhoBERT cho bài toán Trả lời Câu hỏi

(QA) bằng tiếng Việt. Mặc dù các mô hình QA phổ biến trên các nền tảng mã nguồn mở chủ yếu là tiếng Anh, nhưng do đặc thù ngôn ngữ và hạn chế về tài nguyên, việc phát triển mô hình QA cho tiếng Việt gặp nhiều thách thức.

Vì vậy một bộ dataset được chúng em tự tổng hợp gồm hơn 120,000 câu hỏi đã được xây dựng, bao gồm các cột: context (ngữ cảnh), question (câu hỏi), answer (câu trả lời), answer_start (điểm bắt đầu câu trả lời). Bộ dữ liệu này được lấy từ các nguồn như báo, truyện, sách giáo khoa tiếng Việt và một phần từ bộ dữ liệu SQuAD dịch sang tiếng Việt.

Thông số	Mô tả
Số lượng câu hỏi	120,000
Nguồn dữ liệu	Báo, truyện, SGK tiếng Việt, SQuAD được dịch sang tiếng Việt
Các cột dữ liệu	Context, Question, Answer, Answer_start

Table 29: Cấu trúc tập dữ liệu huấn luyện

Tiếp theo, chúng em tiến hành tiền xử lý dữ liệu. Tiền xử lý dữ liệu là quá trình chuẩn bị và biến đổi dữ liệu ban đầu để nó có thể được sử dụng hiệu quả bởi mô hình PhoBERT. Thư viện VnCoreNLP được sử dụng để phân loại từ và tạo token. Các dấu câu và từ ghép được xử lý đặc biệt để giữ nguyên ngữ cảnh.

Cuối cùng, chúng em tiến hành fine-tuning PhoBERT sử dụng API Trainer của Hugging Face, giúp đơn giản hóa quy trình và đạt hiệu suất cao nhất. Mô hình sau đó được lưu trữ trên Hugging Face Hub. Các thông số trong quá trình đào tạo như sau:

Thông số	Mô tả	Giá trị
GPU	Tên GPU đào tạo	Tesla T4
learning_rate	Tốc độ học của trình tối ưu hóa	2e-5

num_train_epochs	Số lượng epoch để huấn luyện mô hình	3
weight_decay	Mức giảm trọng số cho trình tối ưu hóa	0.01
fp16	Đào tạo có độ chính xác hỗn hợp	True
hub_model_id	Tên của mô hình trên Hugging Face Hub	ShynBui/vie_qa

Table 30: Giá trị dữ liệu huấn luyện

4.4.2. Kết quả đánh giá mô hình

Sử dụng bộ dữ liệu test với hơn 770 câu hỏi và ngữ cảnh khác nhau: Ví dụ về một câu test:

Question: “Số điện_thoại Khoa đào_tạo đặc_biệt?”

Context: “Khoa_Đào tạo đặc_biệt gồm các ngành : Quản_trị kinh_doanh , Tài_chính - Ngân_hàng , Kế_toán , Công_nghệ_sinh_học , CNKT Công_trình xây_dựng , Khoa_học máy_tính , Luật kinh_tế , Ngôn_ngữ Anh , Ngôn_ngữ Trung_Quốc , Ngôn_ngữ Nhật , Email : www.ou.edu.vn/dacbiet sas@ou.edu.vn. Điện_thoại Khoa_Đào tạo đặc_biệt : (028) 3930.9918 .”

Answer: “(028) 3930.9918 ”

Exact Match (EM) là một phương pháp đánh giá trong các tác vụ như truy vấn thông tin và phân loại văn bản. Phương pháp này xác định sự khớp chính xác giữa đầu vào và đầu ra dự đoán mà không xét đến sự tương đồng hay độ chính xác của các phần tử bên trong. Trong nghiên cứu này, chúng em sử dụng mô hình đã được fine-tuned để tạo ra các câu trả lời dựa trên đoạn ngữ cảnh. Kết quả đạt được cho chỉ số Exact Match là 36.72%.

Chỉ số	Giá trị
Exact match	36.72%

Table 31: Bảng kết quả Exact match

Ngoài Exact Match chúng em cũng tiến hành thực nghiệm thông qua các ROUGE-1 và ROUGE-2. Trong ROUGE-1, độ phủ (recall) là 0.62, tức là khoảng 62% các từ/cụm từ trong tóm tắt tham chiếu được tìm thấy trong tóm tắt thử nghiệm. Độ chính xác (precision) là 0.58, nghĩa là khoảng 58% các từ/cụm từ trong tóm tắt thử nghiệm khớp với tóm tắt tham chiếu. Điểm F1-score của ROUGE-1 là 0.57, cho thấy sự cân bằng tốt giữa độ chính xác và độ phủ, phản ánh hiệu suất khá của hệ thống trong việc trả lời văn bản so với tóm tắt tham chiếu.

Metric	ROUGE-1	ROUGE-2
Recall (r)	0.62	0.26
Precision (p)	0.58	0.22
F1 - score (f)	0.57	0.13

Table 32: Bảng kết quả đánh giá bằng ROUGE metric

Trong ROUGE-2, độ phủ là 0.26, nghĩa là chỉ 26% các cặp từ/cụm từ liên tiếp trong tóm tắt tham chiếu được tìm thấy trong tóm tắt thử nghiệm. Độ chính xác là 0.22, tức là chỉ 22% các cặp từ/cụm từ liên tiếp trong tóm tắt thử nghiệm là chính xác. Điểm F1-score của ROUGE-2 là 0.13, cho thấy hiệu suất thấp hơn, có thể do dữ liệu huấn luyện chưa đủ hoặc kỹ thuật tinh chỉnh mô hình chưa phù hợp. Chúng em sẽ cải thiện mô hình trong tương lai.

4.4.3. Phân tích kết quả và thử nghiệm PhoBERT

Nhìn chung việc fine-tune mô hình PhoBERT trên tác vụ Reading Comprehension tiếng Việt đã đạt được kết quả đáng nói. Nhưng bản thân chúng em vẫn mong muốn đào tạo một mô hình có hiệu suất tốt hơn và sẽ tiếp tục cố gắng hết sức. Mô hình sẽ cần được cải thiện thêm để tăng cường khả năng nắm bắt ngữ cảnh và chính xác trong các câu trả lời. Đồng thời, các kỹ thuật cải tiến như tăng cường dữ liệu huấn luyện, sử dụng các mô hình tiên tiến hơn, và tối ưu hóa các siêu tham số có thể giúp nâng cao hiệu suất của mô hình. Cuối cùng, chúng em sẽ tạo ra bộ dữ

liệu lớn hơn và đa dạng hơn, việc này cũng có thể giúp cải thiện kết quả của các chỉ số ROUGE và Exact Match.

Chương 5. PHÂN TÍCH KIẾN TRÚC HỆ THỐNG

Trong chương này, chúng em sẽ khám phá chi tiết quá trình phân tích và thiết kế kiến trúc hệ thống cho ứng dụng web. Đầu tiên, chúng ta sẽ tiến hành phân tích yêu cầu để xác định rõ ràng các nhu cầu và mong đợi của người dùng cuối cũng như các yêu cầu kỹ thuật cần thiết để đáp ứng chúng. Tiếp theo, chương sẽ giới thiệu mô hình kiến trúc tổng quan của hệ thống, mô tả các thành phần chính và cách chúng tương tác với nhau. Sau đó, chúng ta sẽ đi sâu vào thiết kế cơ sở dữ liệu, từ việc xây dựng mô hình dữ liệu đến mô tả chi tiết cấu trúc bảng. Cuối cùng, chương sẽ trình bày thiết kế giao diện người dùng chính, đảm bảo tính thân thiện và dễ sử dụng. Toàn bộ nội dung của chương nhằm mục đích cung cấp cho người đọc một cái nhìn toàn diện về quá trình xây dựng và tổ chức một hệ thống web hiệu quả và đáp ứng tốt các yêu cầu đặt ra.

5.1. Phân tích yêu cầu

5.1.1. Yêu cầu người dùng

Người dùng mong muốn hệ thống chatbot học vụ được phát triển với mục tiêu là tạo ra một kênh giao tiếp liền mạch giữa sinh viên và bộ phận quản lý học vụ. Chatbot này không chỉ giải quyết các thắc mắc thông thường về điểm số và quy định học vụ mà còn có khả năng nhận diện và duy trì ngữ cảnh của cuộc đối thoại. Điều này đặc biệt hữu ích khi xử lý các yêu cầu thông tin liên tục mà không cần nhập lại dữ liệu đã được cung cấp.

Ví dụ, sau khi được thông báo rằng TS.GS. Nguyễn Minh Hà giữ chức Hiệu trưởng, chatbot sẽ lưu giữ thông tin này và sử dụng nó để trả lời các câu hỏi liên quan mà không yêu cầu thông tin được lặp lại. Điều này giúp tạo ra một trải nghiệm người dùng liền mạch và thông minh, giảm thiểu sự trùng lặp và tăng cường hiệu quả giao tiếp.

Thêm vào đó, chatbot cũng cần tích hợp chức năng xem và tính toán điểm tuyển sinh. Sinh viên có thể nhập điểm của mình và chatbot sẽ tự động tính toán để

đưa ra đánh giá về khả năng đậu các môn học, dựa trên điểm chuẩn của từng năm học. Tính năng này giúp sinh viên có cái nhìn rõ ràng về tiến trình học tập của mình và hỗ trợ họ trong việc lập kế hoạch học tập tương lai.

Hệ thống cũng nên bao gồm một tính năng báo lỗi, cho phép sinh viên thông báo ngay lập tức về bất kỳ sự cố nào họ gặp phải khi sử dụng chatbot. Điều này đảm bảo rằng mọi vấn đề đều được ghi nhận và xử lý kịp thời, cải thiện chất lượng dịch vụ và trải nghiệm người dùng.

Cuối cùng, giao diện người dùng của chatbot phải được thiết kế để đảm bảo tính thân thiện và trực quan. Mục tiêu là cung cấp một trải nghiệm người dùng mượt mà và dễ dàng, ngay cả với những người không quen thuộc với công nghệ. Điều này giúp tăng cường khả năng tiếp cận và sử dụng chatbot cho tất cả sinh viên, bất kể trình độ kỹ năng công nghệ của họ.

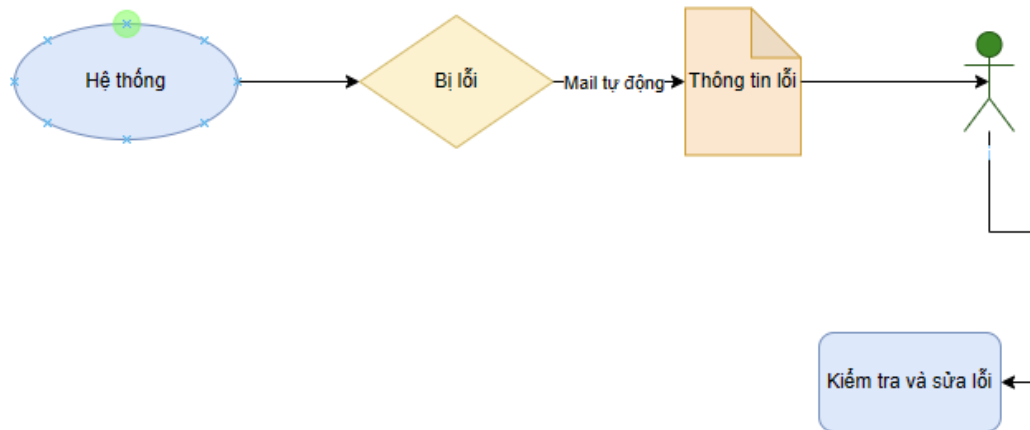
5.1.2. Yêu cầu hệ thống

Hệ thống phải đảm bảo hiệu suất cao ngay cả khi có lượng truy cập lớn, đồng thời phải có khả năng mở rộng để đáp ứng nhu cầu tăng trưởng trong tương lai. Điều này đòi hỏi cơ sở hạ tầng mạnh mẽ và khả năng tự động điều chỉnh tài nguyên dựa trên lưu lượng sử dụng.

Bên cạnh đó bảo mật là ưu tiên hàng đầu, với các biện pháp như mã hóa dữ liệu, xác thực người dùng đa yếu tố, và các cơ chế phòng vệ chống lại các cuộc tấn công mạng. Điều này bảo vệ thông tin cá nhân của sinh viên và đảm bảo tính toàn vẹn của dữ liệu học vụ.

Hệ thống được xây dựng dựa trên mô hình Client – Server, nơi mà chỉ máy chủ (Server) mới có quyền truy cập trực tiếp vào cơ sở dữ liệu. Điều này giúp tối ưu hóa việc phân quyền và quản lý dữ liệu, đồng thời giảm thiểu rủi ro bảo mật. Hệ thống phải tương thích với nhiều loại thiết bị và trình duyệt, từ máy tính đến điện thoại di động và máy tính bảng, để đảm bảo rằng tất cả sinh viên đều có thể truy cập một cách dễ dàng.

Một cơ chế thông báo lỗi tự động qua email cho quản trị viên khi có lỗi phát sinh, giúp nhanh chóng phát hiện và khắc phục sự cố, đảm bảo hệ thống hoạt động mượt mà.



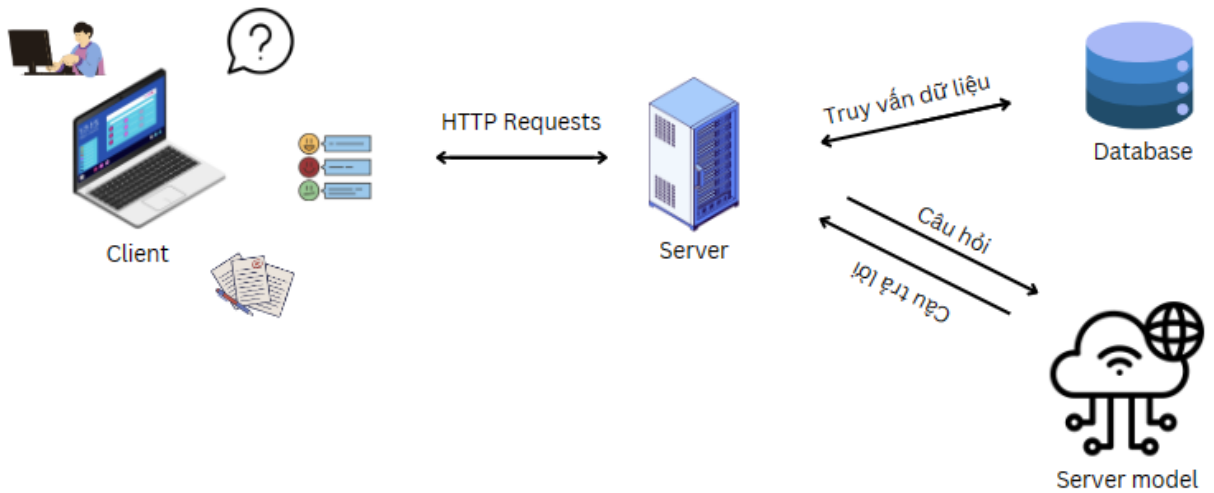
Hình 32: Hệ thống thông báo lỗi

Hệ thống cũng cần một bộ tài liệu hướng dẫn sử dụng và mô tả các chức năng cũng như các tài liệu về hệ thống như người phát triển công nghệ sử dụng....

5.2. Kiến trúc hệ thống

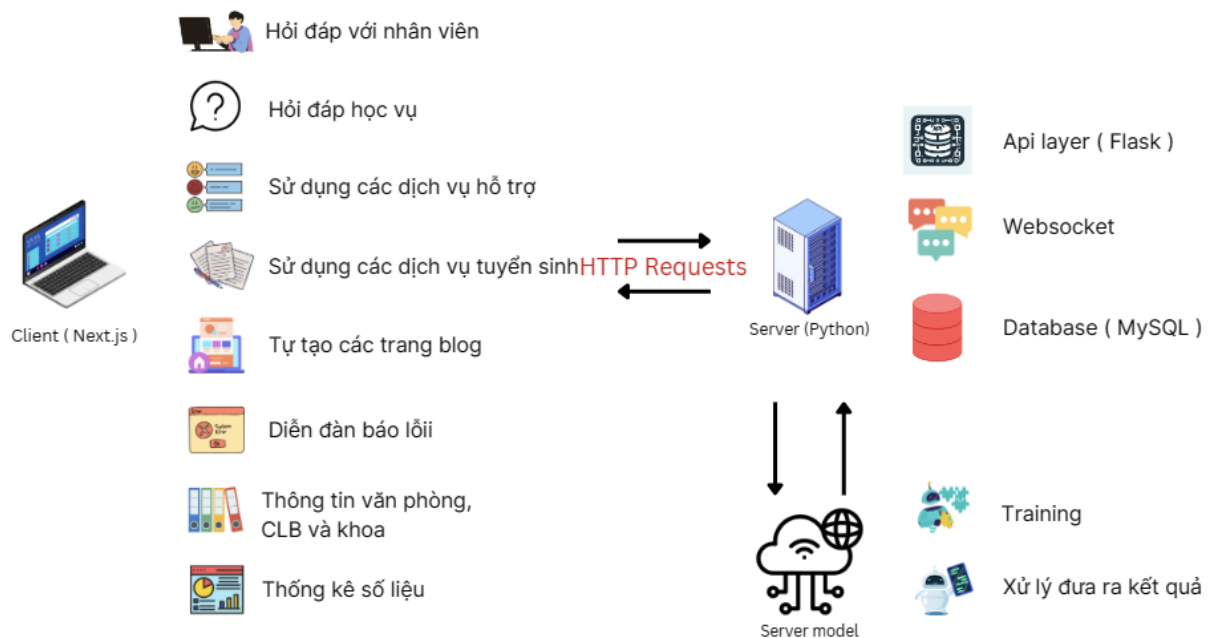
5.2.1. Mô hình kiến trúc tổng quan

Hệ thống được xây dựng theo mô hình Client – Server với FrontEnd sử dụng Next.js, BackEnd sử dụng Python để kết nối Api và Database là MySQL.



Hình 33: Kiến trúc tổng quan hệ thống

Mỗi thành phần chịu trách nhiệm cho nhiều vai trò khác nhau: Client nhận và hiển thị thông tin từ người dùng và các api, server đảm nhiệm việc kết nối model, gửi và truy vết dữ liệu, database lưu những thông tin cần thiết. Tất cả tạo nên mô hình với các chức năng hoàn chỉnh.

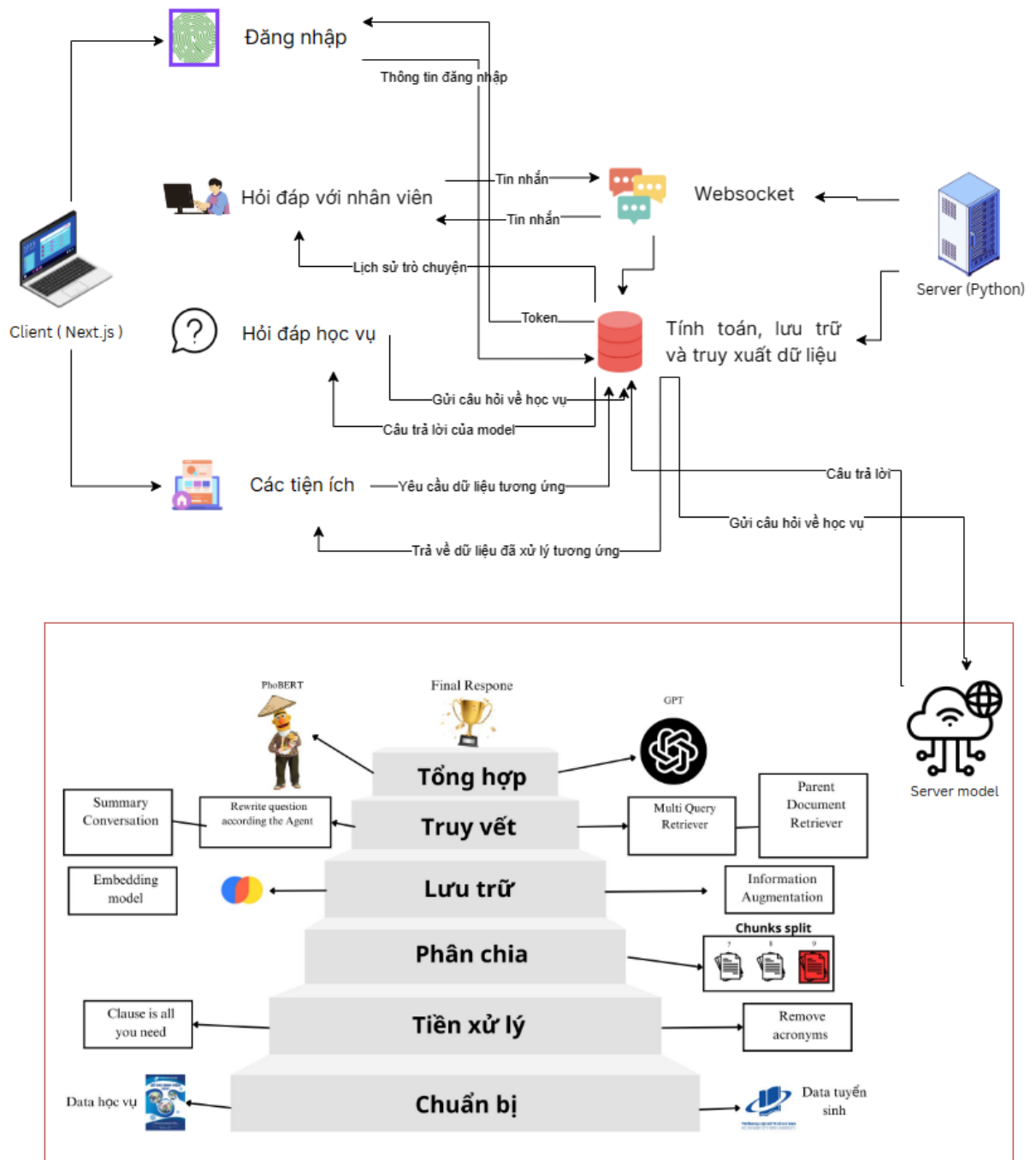


Hình 34: Cấu tạo chức năng hệ thống

Người dùng tương tác qua giao diện Client để gửi yêu cầu đến Server thông qua API. Server sau đó xử lý yêu cầu này, truy xuất và lưu trữ dữ liệu trong

Database, đồng thời sử dụng các mô hình để phân tích và trả lời. Quá trình này đảm bảo thông tin được xử lý một cách chính xác và hiệu quả, phục vụ nhu cầu thông tin của người dùng.

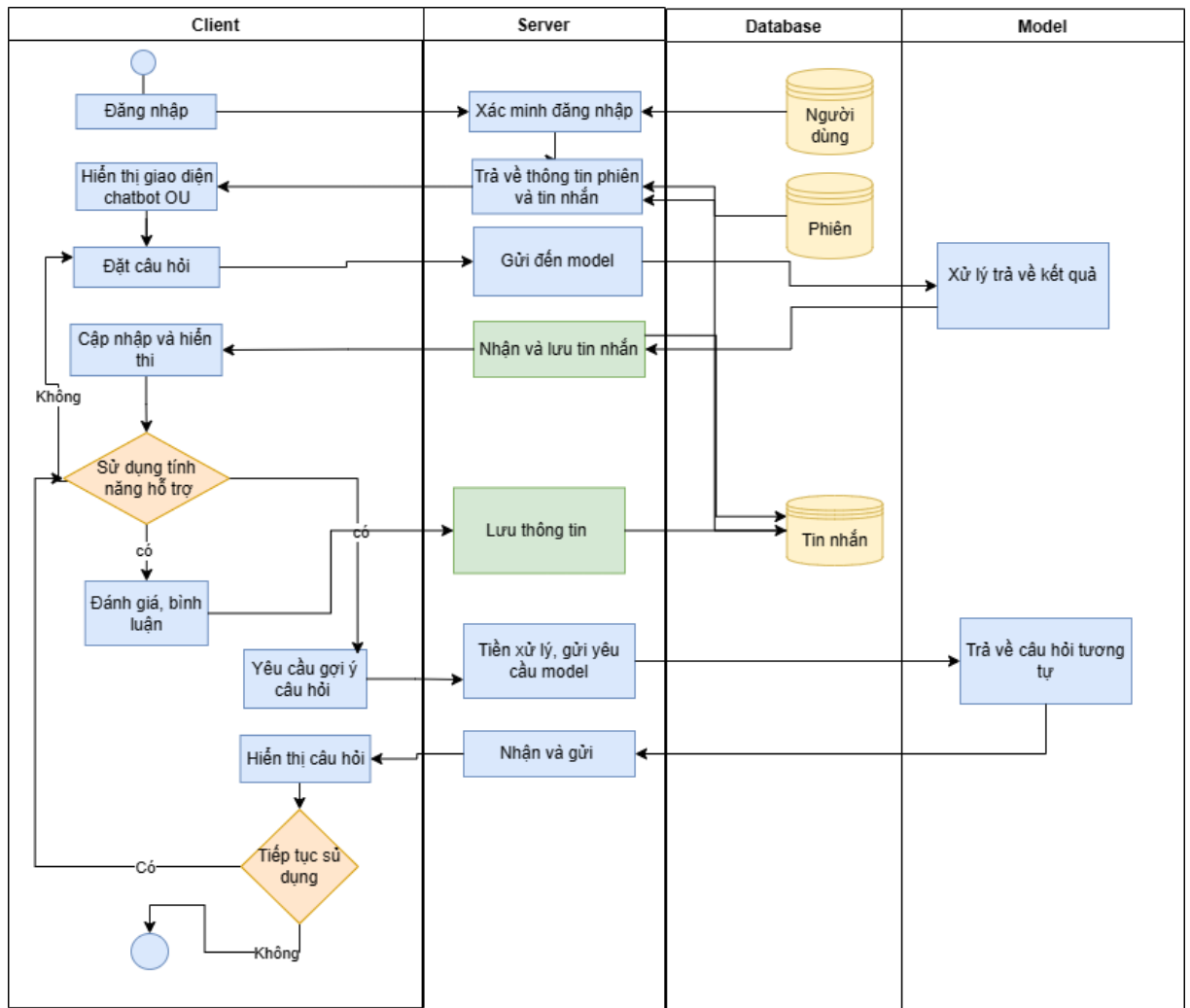
Dưới đây là một cái nhìn chi tiết về kiến trúc đằng sau hệ thống thông minh này, từ giao diện người dùng đến quá trình xử lý phía máy chủ, tất cả đều được thiết kế để mang lại trải nghiệm tương tác tự nhiên và hiệu quả nhất.



Hình 35: Kiến trúc hệ thống chi tiết

Người dùng tương tác với hệ thống ở phía Client thực hiện các hành động như đăng nhập, truy cập thông tin cá nhân, sử dụng các tiện ích. Tất cả thông tin được gửi đến Server, server nhận yêu cầu và xử lý chúng với nguồn dữ liệu từ database, và websocket được dùng cho các yêu cầu liên quan đến tin nhắn trò chuyện. Đối với yêu cầu trả lời câu hỏi Server sẽ gửi chúng đến Server model nơi sẽ xử lý các câu hỏi liên quan đến học vụ, các thuật toán xử lý như PhoBert, Embedding model ... phân tích và đưa ra câu trả lời và câu trả lời được gửi về lại Server ngay khi xử lý xong.

Cấu trúc chức năng hỏi đáp học vụ: mô tả quá trình xử lý và truyền dữ liệu dữ liệu qua các api.



Kiến trúc chức năng chatbot

Phía Client sử dụng các chức năng đã được thiết lập, các yêu cầu này sẽ gửi đến Server. Khi Server nhận được yêu cầu từ Client, nó sẽ kết nối với model đã được xây dựng sẵn để xử lý dữ liệu. Model này có nhiệm vụ lưu trữ thông tin và trả lại kết quả phù hợp cho Client. Điều này bao gồm việc cung cấp các câu hỏi liên quan cho các câu hỏi về học vụ.

5.2.2. Mô tả các thành phần hệ thống

Hệ thống được cấu trúc để cung cấp một dịch vụ web hiệu quả và an toàn. Ở phía frontend, công nghệ Single Page Application của Next.js được áp dụng để nâng cao trải nghiệm người dùng, giảm thời gian tải trang và tạo sự liền mạch khi

tương tác. Server-side rendering và pre-rendering được sử dụng để tăng tốc độ tải trang, cải thiện đáng kể hiệu suất và trải nghiệm người dùng.

Các thư viện và framework như Redux, React, Bootstrap, Chart.js, CKEditor 5, SWR, và Websockets đóng vai trò quan trọng trong việc xây dựng và duy trì hệ thống. Redux quản lý trạng thái toàn cục, React tạo giao diện người dùng linh hoạt, Bootstrap tối ưu hóa thiết kế CSS, Chart.js vẽ biểu đồ thống kê, CKEditor 5 cung cấp trình soạn thảo văn bản, SWR quản lý dữ liệu từ server, và Websockets hỗ trợ giao tiếp thời gian thực.

Ở phía backend, Python và Flask được chọn để xây dựng server vì khả năng tích hợp và linh hoạt cao. Cấu hình server Flask hỗ trợ HTTPS và tăng cường bảo mật. Hiệu suất server được tối ưu hóa thông qua caching và load balancing, đặc biệt quan trọng trong mùa tuyển sinh.

API được thiết kế dựa trên kiến trúc RESTful với Flask, quản lý các yêu cầu HTTP và sử dụng Flask-JWT-Extended cho xác thực và ủy quyền người dùng. API hỗ trợ các chức năng như truy vấn thông tin học vụ, tính toán điểm tuyển sinh, và cung cấp thông tin về các khoa và câu lạc bộ, đảm bảo thông tin được truyền đạt một cách an toàn và chính xác. Đây là những yếu tố cốt lõi giúp hệ thống hoạt động mượt mà và đáng tin cậy. Logic xử lý nghiệp vụ: Xây dựng logic nghiệp vụ trong Flask để xử lý các yêu cầu từ người dùng, như trả lời câu hỏi, tính toán điểm, và cung cấp thông tin cần thiết.

Hệ thống tích hợp AI và mô hình học máy được thực hiện thông qua việc sử dụng các thư viện như gradio. Cơ sở dữ liệu MySQL được chọn lựa do khả năng xử lý và truy xuất dữ liệu hiệu quả, đặc biệt là với các cấu trúc phức tạp. Sơ đồ quan hệ được thiết kế để đảm bảo tính toàn vẹn và nhất quán của dữ liệu, là nền tảng vững chắc cho việc lưu trữ và quản lý thông tin trong hệ thống.

5.3. Thiết kế CSDL

Một phần quan trọng của dự án "Hệ thống hỗ trợ học vụ tự động cho sinh

- user : người dùng
- role : vai trò của người dùng
- session : các phiên trò chuyện khác nhau
- message : các câu hỏi và câu trả lời của mỗi phiên
- information : các thông tin ngoài lề có thể tùy chỉnh (các thông tin riêng biệt)
- subject_combination : Tổ hợp môn xét tuyển vd: A00, A01 ..
- data_score_vs_subject_combination : lưu trữ 1 ngành cần các tổ hợp môn nào vd: cntt cần A000, A001, B00.
- admission_subject: các môn học xét tuyển như toán, văn , anh
- subject_combination_vs_admission_subject: giúp nhận định tổ hợp môn gồm những môn nào, một môn có thể thuộc nhiều tổ hợp.

5.3.2. Mô tả cấu trúc bảng

Tên bảng	Mục đích
Bug_question	Lưu trữ các câu hỏi của người dùng khi phát hiện bug.
bug_comment	Lưu trữ các bình luận cho mỗi câu hỏi bug.
subject_combination_vs_admission_subject	Xác định các môn học thuộc tổ hợp môn xét tuyển.
admission_subject	Lưu trữ thông tin về các môn học xét tuyển
data_score_vs_subject_combination	Để nhận dạng các tổ hợp môn thuộc về ngành nào và một tổ hợp môn có

	thể thuộc về nhiều ngành.
subject_combination	Lưu trữ thông tin về tổ hợp môn.
chat_with_employee	Lưu trữ thông tin trao đổi giữa người dùng và nhân viên.
data_score	Lưu trữ điểm số của các ngành qua các năm.
faculty	Lưu trữ thông tin về các khoa.
role	Lưu trữ các vai trò trong hệ thống.
message	Ghi lại câu hỏi và câu trả lời trong các phiên làm việc.
session	Lưu trữ thông tin về các phiên làm việc.
user	Lưu trữ thông tin về người dùng của hệ thống.
information	Lưu trữ thông tin về các nguồn tài liệu hoặc thông tin hữu ích khác.

Table 33: Thông tin các bảng database

Mối liên hệ giữa các bảng như sau:

Bảng *data_score_vs_subject_combination* và *data_score*: Mỗi bản ghi trong *data_score_vs_subject_combination* liên kết với một bản ghi trong *data_score* thông qua trường *id_score*. Điều này cho phép xác định điểm số liên quan đến mỗi tổ hợp môn.

Bảng chat_with_employee và user: Mỗi tin nhắn trong chat_with_employee được gửi bởi một người dùng, được xác định bởi user_id liên kết với bảng user.

Bảng data_score và faculty: Mỗi bản ghi trong data_score liên kết với một khoa cụ thể trong faculty thông qua trường id_faculty.

Bảng subject_combination và data_score_vs_subject_combination: Mỗi tổ hợp môn trong subject_combination được xác định bởi id_combination liên kết với bảng data_score_vs_subject_combination.

Bảng bug_question và user: Mỗi câu hỏi bug trong bug_question được tạo bởi một người dùng, được xác định bởi user_id liên kết với bảng user.

Bảng bug_comment, user, và bug_question: Mỗi bình luận trong bug_comment được tạo bởi một người dùng và liên kết với một câu hỏi bug cụ thể. Trường user_id và user_id_last_comment liên kết với user, trong khi bug_question_id liên kết với bug_question.

Bảng user và role: Mỗi người dùng trong user có một vai trò cụ thể được xác định bởi role_id liên kết với bảng role.

Bảng session và user: Mỗi phiên làm việc trong session được tham gia bởi một người dùng, được xác định bởi user_id liên kết với bảng user.

Bảng subject_combination và subject_combination_vs_admission_subject: Mỗi tổ hợp môn trong subject_combination liên kết với một hoặc nhiều môn học trong subject_combination_vs_admission_subject thông qua id_combination.

Bảng message và session: Mỗi cặp câu hỏi và câu trả lời trong message thuộc về một phiên làm việc cụ thể, được xác định bởi session_id liên kết với bảng session.

Bảng new_page và user: Mỗi trang mới trong new_page có thể được tạo bởi một người dùng, được xác định bởi user_id liên kết với bảng user.

Bảng *subject_combination_vs_admission_subject* và *admission_subject*: Mỗi bản ghi trong *subject_combination_vs_admission_subject* liên kết với một môn học cụ thể trong *admission_subject* thông qua *id_subject*.

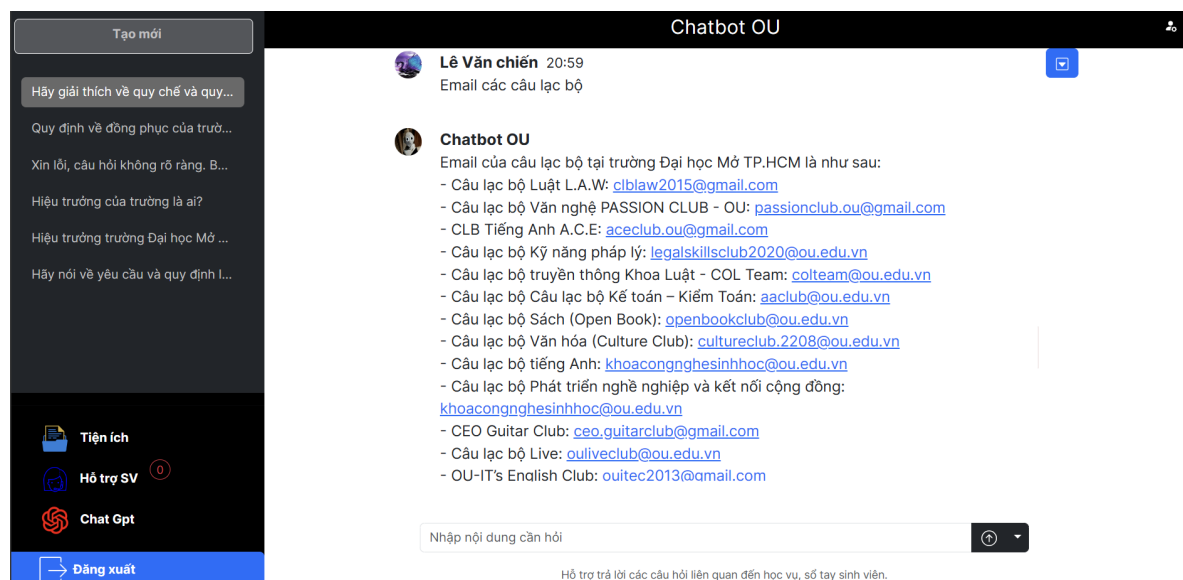
Bảng *bug_comment* và *bug_question*: Mỗi bình luận trong *bug_comment* liên kết với một câu hỏi bug cụ thể trong *bug_question* thông qua *bug_question_id*.

5.4. Thiết kế giao diện và nghiệp vụ

5.4.1. Giao diện chính

Trong phần này sẽ phân tích những giao diện quan trọng của hệ thống: Trang chủ Chatbot, trang tiện ích, và trang quản lý. Mỗi giao diện đều đóng một vai trò quan trọng trong việc cung cấp trải nghiệm người dùng mượt mà và hiệu quả.

Trang chủ Chatbot: Là trang đầu tiên người dùng tiếp xúc sau khi đăng nhập, trang chủ có vai trò là một Chatbot trả lời các câu hỏi về học vụ của người dùng, với thiết kế giản dị và tổng quan giúp người dùng dễ dàng sử dụng ngay lần đầu.



Hình 36: Giao diện trang chủ Chatbot

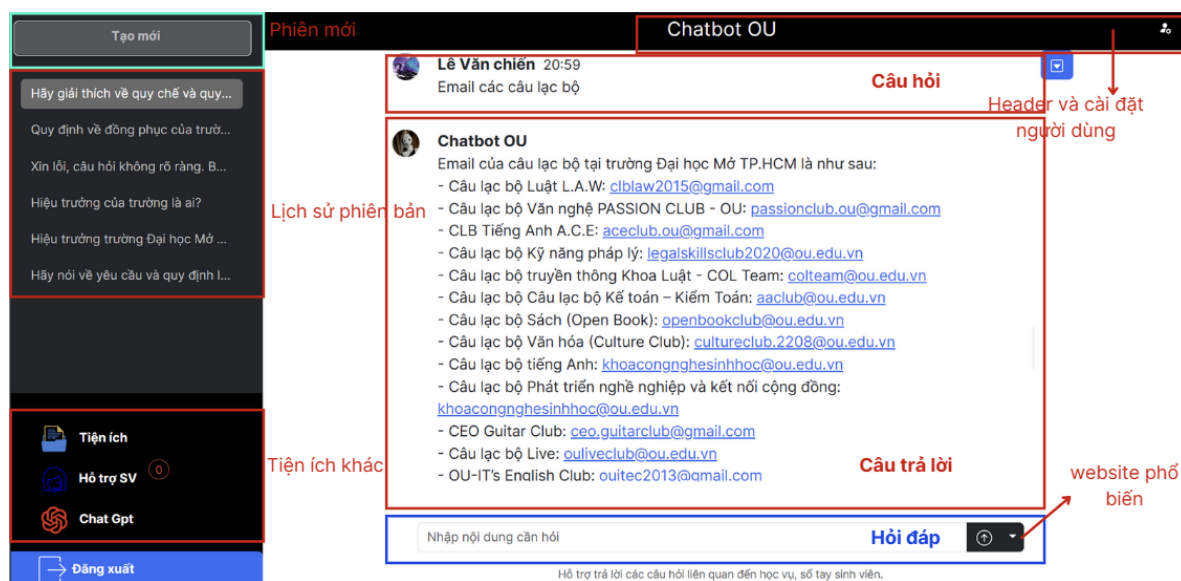
Các yếu tố chính bao gồm:

- Header: Hiển thị tên trang và cài đặt thông tin người dùng.
- Side Bar: Hiển thị lịch sử các phiên bản đã sử dụng trước đó và cho phép tạo

phiên bản mới. Ngoài ra chứa các nội dung hỗ trợ như tiện ích (Các dịch vụ hỗ trợ sinh viên khác như thông tin tuyển sinh), Chat với nhân viên để nhận được hỗ trợ, hoặc sử dụng Chat gpt.

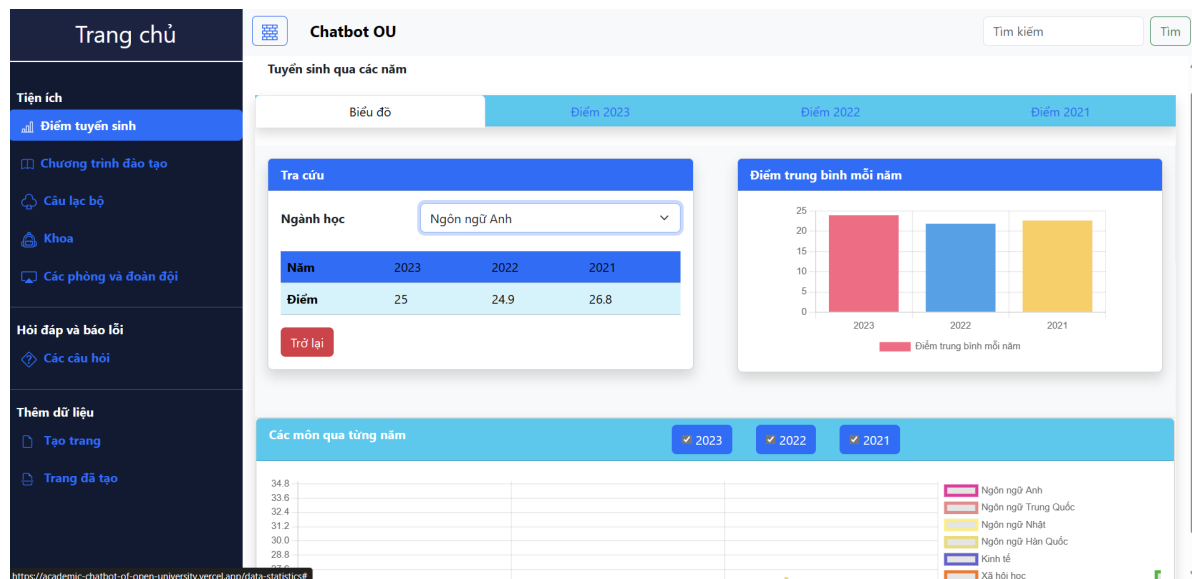
- Body: Hiển thị đoạn chat bao gồm câu hỏi của người dùng và câu trả lời.

Cấu trúc chi tiết:



Hình 37: Giao diện trang chủ Chatbot chi tiết

Trang tiện ích: Trang Tiện Ích không chỉ là trang thông tin đơn thuần nó là một người bạn đồng hành đáng tin cậy cho sinh viên trong hành trình học tập của họ. Với mục tiêu tối ưu hóa trải nghiệm học tập và nâng cao sự tiện lợi, trang này được thiết kế để đáp ứng nhu cầu thông tin đa dạng của sinh viên.



Hình 38: Giao diện trang tiện ích

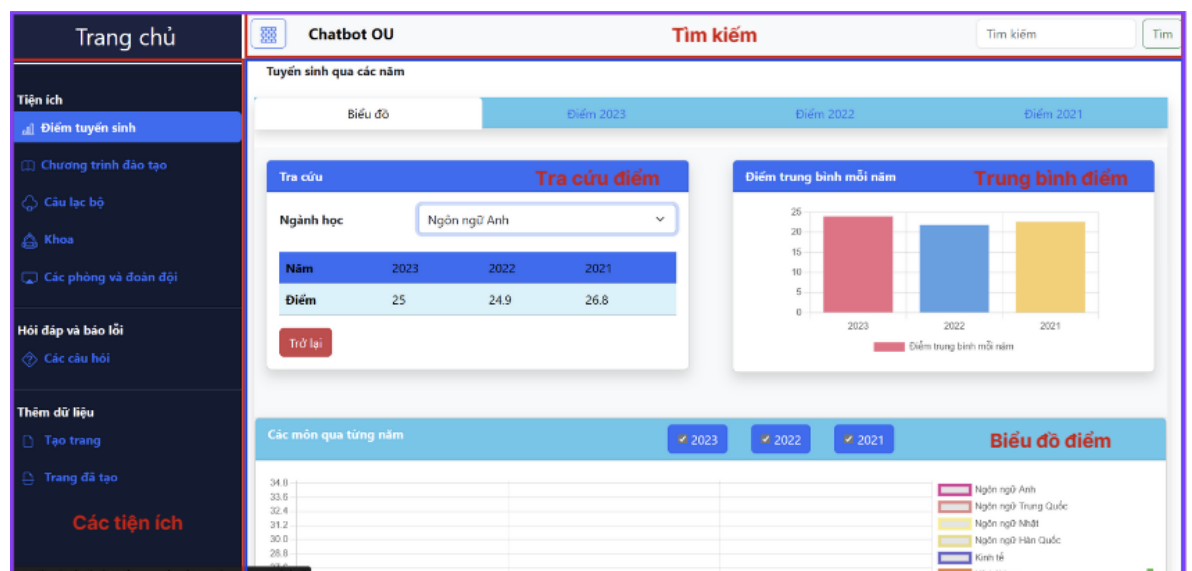
Các yếu tố chính bao gồm:

Side Bar: Hiển thị danh sách các tiện ích.

Header: Hiển thị tên trang và chức năng tìm kiếm cho một số tiện ích.

Body: Thông tin và chức năng của mỗi tiện ích.

Cấu trúc chi tiết



Hình 39: Giao diện trang tiện ích chi tiết

Các chức năng

Thông Tin Tuyển Sinh: Một cổng thông tin đầy đủ, cung cấp dữ liệu cập nhật về điểm chuẩn, quy trình nhập học, và các hướng dẫn cần thiết để sinh viên mới có thể bắt đầu hành trình học tập một cách thuận lợi ngoài ra sinh viên có thể sử dụng công cụ tính điểm bên trong để xác định các ngành đủ điều kiện theo học.

Câu Lạc Bộ và Hoạt Động Ngoại Khóa: Một danh mục toàn diện về các câu lạc bộ, tổ chức sinh viên, giúp sinh viên tìm thấy cộng đồng và tham gia vào các hoạt động phong phú ngoài giờ học.

Khoa: Thông tin liên lạc các khoa trong của trường, sinh viên có thể dễ dàng tìm kiếm khoa theo chức năng hoạt động của khoa đó và các nhiệm vụ do khoa đó đảm nhận.

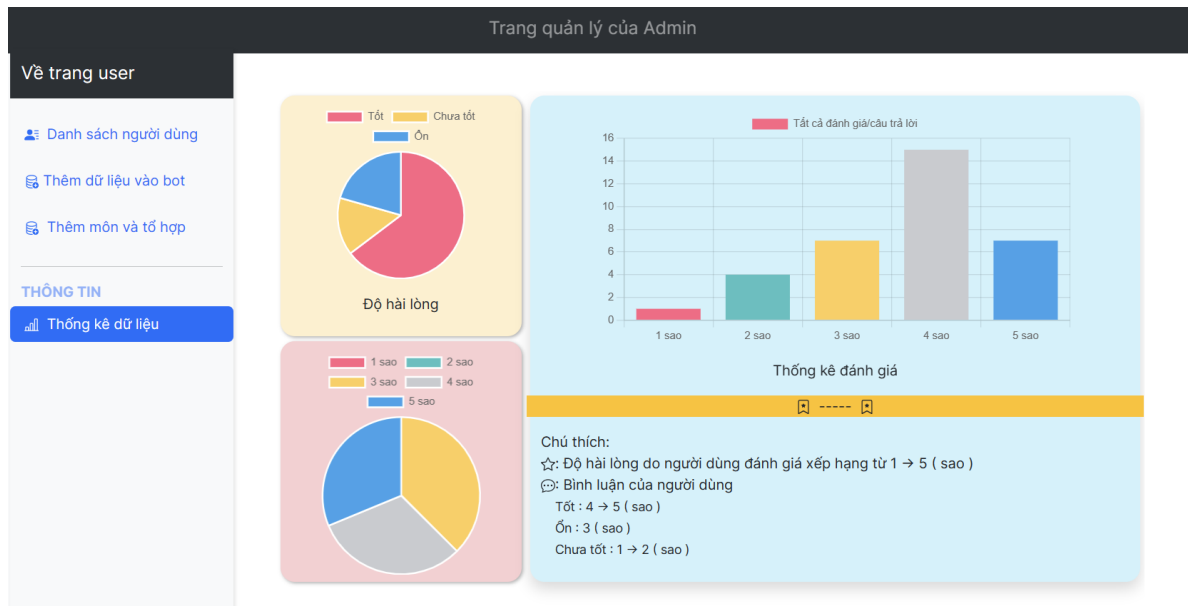
Văn phòng và đoàn đội: cung cấp thông tin về địa chỉ, số điện thoại liên lạc và chức năng nhiệm vụ của từng phòng.

Chương Trình Đào Tạo: Cung cấp về các khóa học, chương trình đào tạo, và các tùy chọn học thuật, cho phép sinh viên lập kế hoạch học tập của mình một cách hiệu quả và định hình tương lai nghề nghiệp.

Hỏi đáp và báo lỗi: Là một diễn đàn nơi sinh viên có thể tạo các blog để báo lỗi hoặc tham gia trao đổi với những người khác về hệ thống. Quản lý sẽ kiểm tra và sửa các lỗi nếu có.

Tạo trang: Chức năng dành cho các đội nhóm hoặc khoa của trường nhằm giúp tạo các thông báo nhanh chóng, cho các sinh viên sử dụng hệ thống.

Trang quản lý: Là nơi điều khiển hệ thống quản trị website, nơi quản trị viên có thể thực hiện kiểm soát và phân tích dữ liệu. Được thiết kế đơn giản hóa quản lý hằng ngày, cung cấp các công cụ mạnh mẽ trực quan cho nhiệm vụ chính.



Hình 40: Giao diện trang quản lý

Các yếu tố chính bao gồm:

Side Bar: Hiển thị danh sách các chức năng.

Header: Hiển thị tên trang.

Body: Thông tin và chức năng của mỗi chức năng.

Các chức năng

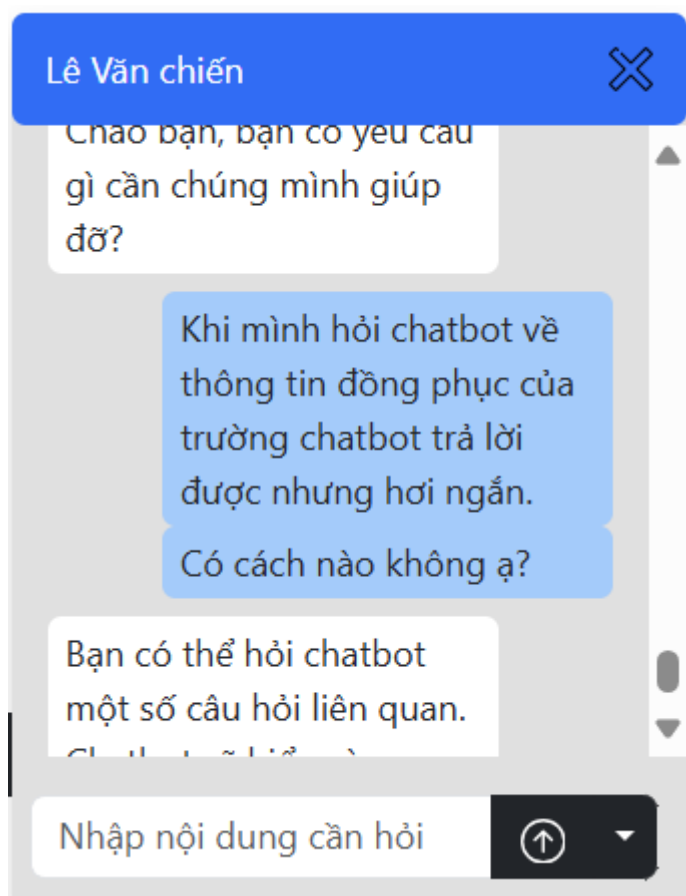
Danh sách người dùng: Trang Quản Lý cung cấp một Danh Sách Người Dùng động, nơi hiển thị thông tin chi tiết của từng người dùng. Điểm nổi bật của tính năng này là khả năng cho phép quản trị viên không chỉ quan sát mà còn trả lời tin nhắn một cách nhanh chóng, tạo điều kiện cho giao tiếp hai chiều và tăng sự trực quan giữa người dùng và quản trị viên.

Thêm dữ liệu và Chatbot: Với chức năng Thêm Dữ Liệu và Chatbot, quản trị viên có thể bổ sung dữ liệu trực tiếp vào hệ thống, giúp Chatbot trở nên thông minh hơn, nhanh nhẹn hơn trong việc trả lời và tương tác. Điều này không chỉ nâng cao khả năng phục vụ của Chatbot mà còn mang lại trải nghiệm người dùng cá nhân hóa và thân thiện hơn.

Thêm môn và tổ hợp: Tính năng Thêm Môn và Tổ Hợp là một công cụ linh hoạt, cho phép quản trị viên mở rộng cơ sở dữ liệu học thuật của nền tảng. Qua đó, họ có thể bổ xung các môn học mới và tổ hợp xét tuyển, đáp ứng sự thay đổi của chương trình giáo dục.

Thống kê dữ liệu: Là một bảng điều khiển trực quan, thể hiện đánh giá của Chatbot qua các thông số như số sao, bình luận, và tỉ lệ đánh giá. Đây là công cụ đặc lực giúp quản trị viên theo dõi hiệu suất và nhận phản hồi từ người dùng, từ đó không ngừng cải thiện và phát triển Chatbot. Mô tả quy trình nghiệp vụ

Giao diện Tính Năng Hỗ Trợ: Ngoài những giao diện chính của trang, chúng em cung cấp các giao diện phụ tiện lợi dành cho những chức năng quan trọng, bao gồm cả bảng chat linh hoạt.



Hình 41: Giao diện chat với nhân viên

Giao diện Chat Trực Tiếp với Nhân Viên: Đây là tính năng đặc biệt giúp kết nối người dùng với đội ngũ nhân viên của chúng em, tạo điều kiện cho một cuộc trò chuyện trực tiếp và hiệu quả, nhằm đáp ứng mọi yêu cầu và giải đáp thắc mắc một cách nhanh chóng.

Chương 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Trong nghiên cứu này, chúng em đã phát triển và tối ưu hóa một hệ thống chatbot học vụ dựa trên mô hình ngôn ngữ PhoBERT, một phiên bản ngôn ngữ tiếng Việt của kiến trúc BERT, để giải quyết bài toán trả lời các câu hỏi của sinh viên về vấn đề học vụ. Chúng em đã đạt được nhiều kết quả đáng kể và đưa ra các kỹ thuật cải tiến để nâng cao hiệu suất của chatbot.

Phương pháp "Clause is all you need" đã chứng minh hiệu quả trong việc cải thiện khả năng truy vết và cung cấp câu trả lời chính xác từ nguồn kiến thức. Kỹ thuật này đã giúp chatbot đạt được độ chính xác cao hơn và giảm tỷ lệ câu hỏi không thể trả lời. Thêm vào đó, việc loại bỏ từ viết tắt (Remove Acronyms) cũng đã giúp cải thiện hiệu suất của chatbot, làm tăng tỷ lệ trả lời chính xác.

Phương pháp Information Augmentation đã cải thiện khả năng truy vết dữ liệu của chatbot, với độ tương đồng cosine và tỷ lệ truy vết đúng tăng đáng kể. Việc kết hợp MultiQueryRetriever và Parent Document Retriever cũng đã giúp nâng cao chất lượng và độ đầy đủ của thông tin trả về, giúp chatbot cung cấp câu trả lời chi tiết và chính xác hơn.

Cuối cùng, mô hình PhoBERT đã được fine-tuning thành công cho tác vụ Reading Comprehension bằng tiếng Việt, với kết quả đánh giá tích cực qua các chỉ số Exact Match và ROUGE-metric. Mặc dù vẫn còn một số hạn chế, nghiên cứu của chúng em đã đặt nền móng vững chắc cho việc phát triển các hệ thống chatbot học vụ hiệu quả và đáng tin cậy.

6.2. Hạn chế của nghiên cứu

Mặc dù đã đạt được nhiều kết quả đáng khích lệ, nghiên cứu của chúng em vẫn gặp phải một số hạn chế quan trọng sau:

Hạn chế về dữ liệu huấn luyện: Một trong những hạn chế chính là dữ liệu huấn luyện chưa đủ đa dạng và phong phú. Bộ dữ liệu hiện tại chủ yếu bao gồm các

câu hỏi và câu trả lời từ báo chí, truyện, sách giáo khoa và một phần từ bộ dữ liệu SQuAD được dịch sang tiếng Việt. Điều này có thể dẫn đến việc mô hình không thể hiểu và xử lý tốt các câu hỏi thuộc nhiều ngữ cảnh khác nhau hoặc những câu hỏi có cấu trúc phức tạp hơn.

Hiệu suất của mô hình trên các câu hỏi có ngữ cảnh phức tạp: Mặc dù mô hình PhoBERT đã được fine-tuning và đạt kết quả tốt, nhưng nó vẫn gặp khó khăn trong việc trả lời các câu hỏi có ngữ cảnh phức tạp hoặc chứa nhiều mệnh đề thay thế. Các kỹ thuật như "Rewrite question according the Agent và Summary Conversation" đã cải thiện hiệu suất, nhưng vẫn còn nhiều câu hỏi mà mô hình không thể trả lời chính xác.

Giới hạn trong khả năng trích xuất thông tin từ nguồn bên ngoài: Phương pháp Open-book QA cho phép truy cập vào nguồn thông tin rộng lớn, nhưng việc truy vấn và trích xuất thông tin vẫn còn hạn chế. Kết quả thực nghiệm cho thấy tỷ lệ truy vấn đúng chưa đạt mức tối ưu, đặc biệt khi so sánh với phương pháp Close-book QA. Việc tìm kiếm thông tin từ các nguồn dữ liệu lớn và đa dạng đòi hỏi các kỹ thuật truy vấn và trích xuất thông tin hiệu quả hơn.

Độ chính xác của việc loại bỏ từ viết tắt: Mặc dù việc loại bỏ từ viết tắt đã giúp cải thiện hiệu suất của chatbot, nhưng tỷ lệ trả lời chính xác khi sử dụng ngôn ngữ thông thường chỉ cao hơn khoảng 7% so với khi sử dụng từ viết tắt. Điều này cho thấy rằng việc xử lý các từ viết tắt và ngôn ngữ tự nhiên phức tạp vẫn cần được cải thiện thêm.

Hạn chế của các kỹ thuật tăng cường dữ liệu: Mặc dù các kỹ thuật như "Clause is all you need" và "Information Augmentation" đã cải thiện đáng kể khả năng truy vết và chất lượng câu trả lời, nhưng vẫn còn nhiều câu hỏi mà các kỹ thuật này chưa thể xử lý một cách tối ưu. Việc áp dụng các kỹ thuật này đòi hỏi một lượng lớn tài nguyên tính toán và thời gian xử lý.

Khả năng xử lý ngôn ngữ tự nhiên và hiểu ngữ cảnh: Một trong những thách thức lớn nhất của chatbot là khả năng xử lý ngôn ngữ tự nhiên và hiểu ngữ cảnh

trong các cuộc trò chuyện. Mặc dù đã sử dụng các kỹ thuật tiên tiến, nhưng chatbot vẫn gặp khó khăn trong việc hiểu và trả lời các câu hỏi có ngữ cảnh phức tạp hoặc có nhiều nghĩa.

6.3. Hướng phát triển trong tương lai

Hướng phát triển trong tương lai của chatbot hỗ trợ học vụ sẽ tập trung vào việc mở rộng khả năng cá nhân hóa, tăng cường tính tương tác và tối ưu hóa khả năng xử lý thông tin. Điều này không chỉ giúp chatbot đáp ứng nhu cầu thông tin mà còn tạo ra một trải nghiệm tương tác tự nhiên và cá nhân hóa, hỗ trợ sinh viên trong quá trình học tập và phát triển.

Cá nhân hóa chatbot không chỉ giúp tăng cường trải nghiệm người dùng mà còn thúc đẩy sự tương tác hai chiều, tạo điều kiện cho chatbot học hỏi và thích ứng với từng cá nhân. Chatbot có thể được lập trình để nhớ các sở thích học tập và mục tiêu cá nhân của sinh viên, từ đó đề xuất các nguồn tài nguyên học tập phù hợp và lịch trình ôn tập cá nhân hóa.

Liên kết tiện ích sinh viên là một phần không thể thiếu trong việc phát triển chatbot học vụ. Chatbot cần được tích hợp chặt chẽ với hệ thống thông tin của trường học để cung cấp thông tin chính xác và cập nhật về điểm số, thống kê môn học, và các thông tin học vụ khác. Chatbot có thể được kết nối với cổng thông tin sinh viên, cho phép truy cập vào lịch thi, thông báo mới nhất từ trường, và thậm chí là đăng ký môn học.

Hiệu chỉnh dữ liệu là một khía cạnh quan trọng khác, cho phép chatbot liên tục cải thiện và phát triển. Việc tạo điều kiện cho người dùng đánh giá và góp ý sẽ giúp chatbot nhanh chóng cập nhật thông tin và sửa chữa các lỗi. Chatbot có thể có một hệ thống phản hồi người dùng, nơi sinh viên có thể báo cáo sự cố hoặc đề xuất cải tiến.

6.4. Ứng dụng thực tiễn

Nghiên cứu này có tiềm năng ứng dụng thực tiễn rộng rãi trong các trường

học và tổ chức giáo dục. Hệ thống chatbot học vụ có thể được triển khai để hỗ trợ sinh viên trong việc tiếp cận thông tin học vụ, đăng ký môn học, quản lý lịch thi và nhận thông báo quan trọng từ trường học. Điều này không chỉ giúp giảm tải công việc cho nhân viên hành chính mà còn nâng cao trải nghiệm học tập của sinh viên, giúp họ tiếp cận thông tin một cách nhanh chóng và thuận tiện.

Ngoài ra, hệ thống chatbot còn có thể được tích hợp vào các ứng dụng di động và trang web của trường học, tạo điều kiện cho sinh viên truy cập thông tin mọi lúc, mọi nơi. Điều này sẽ góp phần tạo nên một môi trường học tập thông minh, hiện đại và hiệu quả hơn.

TÀI LIỆU THAM KHẢO

IBM, "What is Natural Language Processing," [Online]. Available: <https://www.ibm.com/topics/natural-language-processing>. [Accessed: 20-Sep-2023].

phamdinhhkhanh, "Bài 36 – BERT model," 23-May-2020. [Online]. Available: <https://phamdinhhkhanh.github.io/2020/05/23/BERTModel.html>. [Accessed: 02-Dec-2023].

L. Tunstall, L. von Werra, and T. Wolf, Natural Language Processing with Transformers-Building Language Applications with Hugging Face, Revised edition, Sebastopol, California: O'Reilly, 2022.

Python Software Foundation, "Langchain API Reference," 2024. [Online]. Available: https://api.python.langchain.com/en/latest/langchain_api_reference.html. [Accessed: 05-Dec-2023].

Nextjs.org, "The React Framework for the Web", 2023. [Online]. Available: <https://nextjs.org/docs> [Accessed: 2024].

PHỤ LỤC

A. Mã nguồn và Demo

1. Mã nguồn PhoBERT-reader:

- Vui lòng xem tại đây: [PhoBERT Mã nguồn](#)
- Mô tả: Mã nguồn cho mô hình PhoBERT được fine-tuning để trả lời câu hỏi bằng tiếng Việt.

2. Demo PhoBERT:

- Vui lòng xem tại đây: [PhoBERT Demo](#)
- Mô tả: Trang demo cho phép thử nghiệm mô hình PhoBERT trên bộ dữ liệu tiếng Việt.

3. Mã nguồn và Demo chatbot sử dụng Information Augmentation (IA):

- Vui lòng xem tại đây: [Chatbot Mã nguồn và Demo](#)
- Mô tả: Mã nguồn và demo của chatbot sử dụng kỹ thuật Information Augmentation để cải thiện khả năng truy vấn và trả lời câu hỏi.

4. Mã nguồn frontend website hệ thống:

- Vui lòng xem tại đây: [FrontEnd \(Next.js\)](#)
- Mô tả: Sử dụng Next.js xây dựng các giao diện và tiện ích của chatbot, chịu trách nhiệm nhận các yêu cầu từ phía người dùng.

5. Mã nguồn backend website hệ thống:

Tên và liên kết	Mô tả
backendchatbot	Mã nguồn backend chính của chatbot.
server-connect-apiModel	Chịu trách nhiệm kết nối api giữa server và model.
server-websocket	Websocket server giúp người dùng nhận tin trực tiếp với quản trị hệ thống theo thời gian thực.

B. Công bố khoa học

C. Hướng dẫn cài đặt và chạy dự án Chatbot

Để chạy đầy đủ dự án chatbot, bạn cần cài đặt các thành phần sau (chi tiết được hướng dẫn trong Readme.md) :

1. FrontEnd (Next.js)

- Repository: [FrontEnd Repository](#)
- Mô tả: Mã nguồn phần FrontEnd của dự án chatbot, xây dựng bằng Next.js.

2. BackEnd (Python)

Các repository cần thiết cho phần BackEnd bao gồm:

- [backendchatbot](#)
 - Mô tả: Mã nguồn cho phần backend của chatbot.
- [server-connect-apiModel](#)

- Mô tả: API kết nối giữa server và model.
- [server-websocket](#)
 - Mô tả: Websocket server cho dự án chatbot.

D. Trải nghiệm trực tiếp

Bạn có thể trải nghiệm trực tiếp dự án qua website Chatbot của Đại học Mở:

- [Chatbot OU Website](#)
- Mô tả: Trang web cho phép trải nghiệm trực tiếp các tính năng của chatbot được phát triển trong dự án.